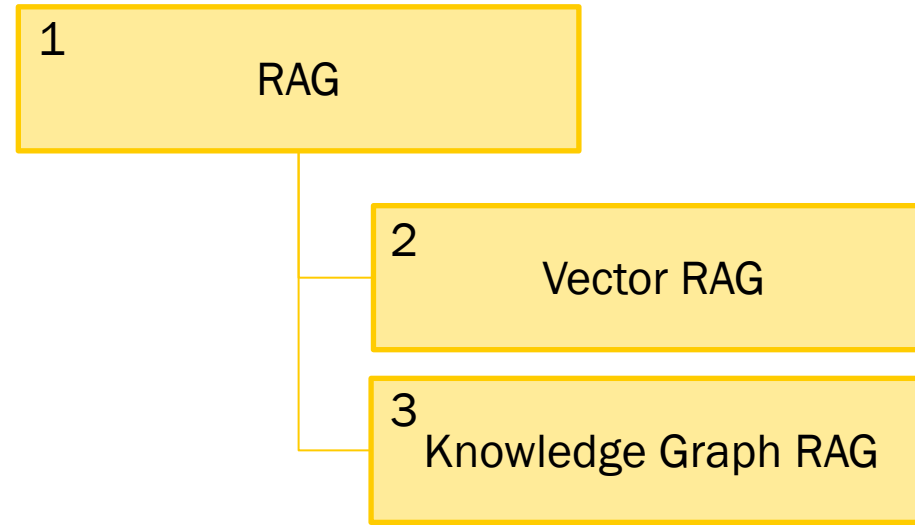


Intelligente Informationssysteme 4 – Agentic Infrastructure

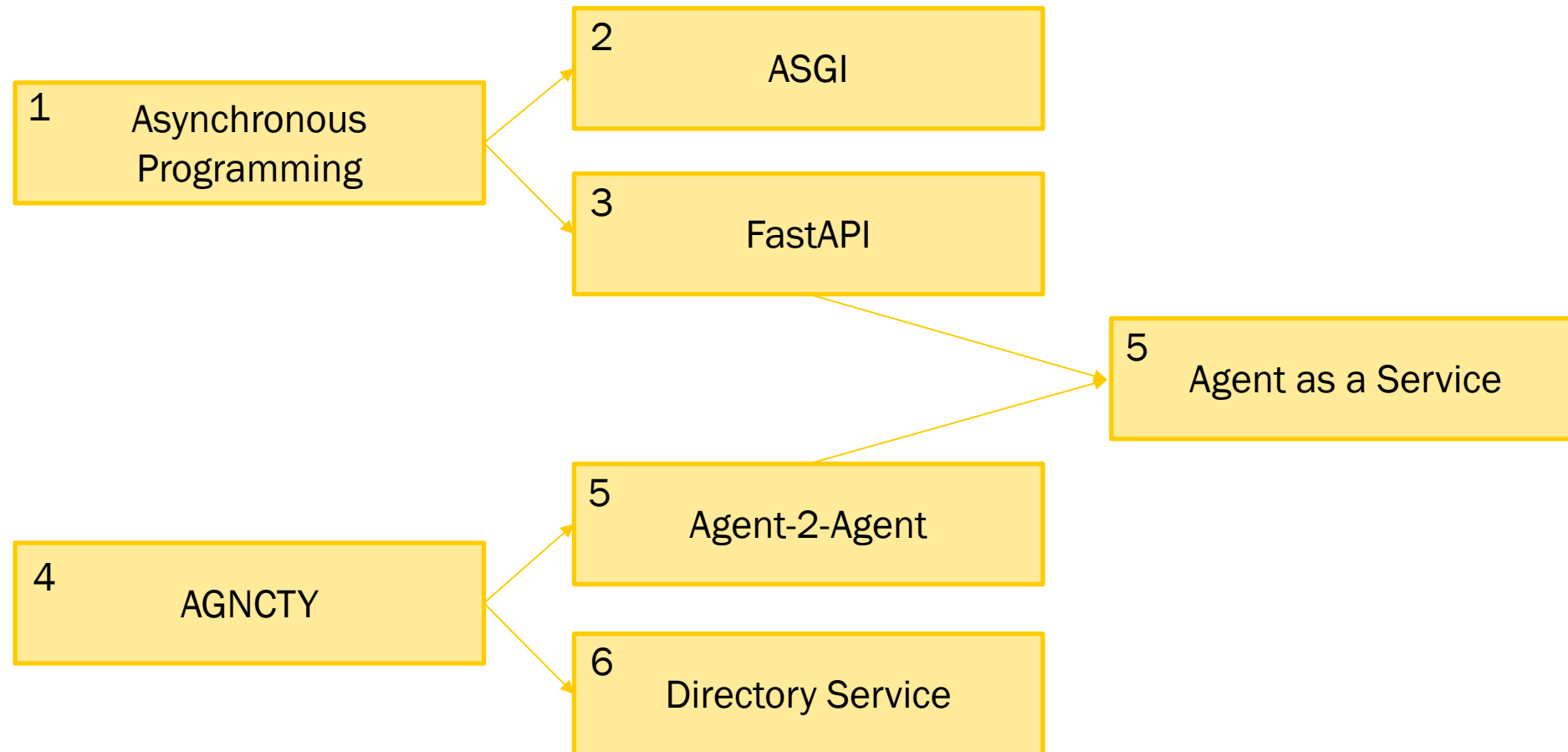
Dominik Neumann



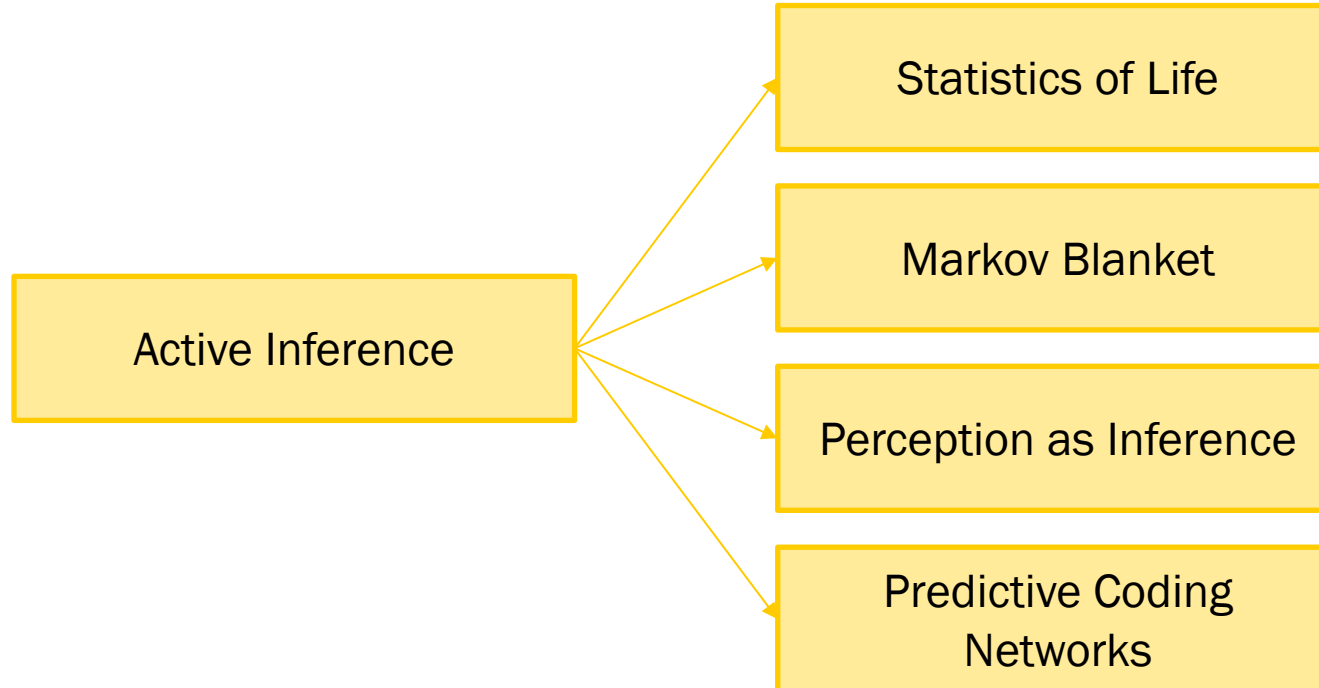
Block 4a Knowledge Representaion



Block 4b Agentic Infrastructure



Block 4c Active Inference



Intelligente Informationssysteme 4a Retrieval Augmented Generation

Dominik Neumann



Knowledge Representation Hypothesis



Intelligence depends on internal representations of the world that are causally responsible for behavior.

Brian Smith - Knowledge Representation Hypothesis:

- any system that exhibits intelligent behavior must contain internal structures that represent knowledge about the world,
- and are used causally to generate the system's behavior.

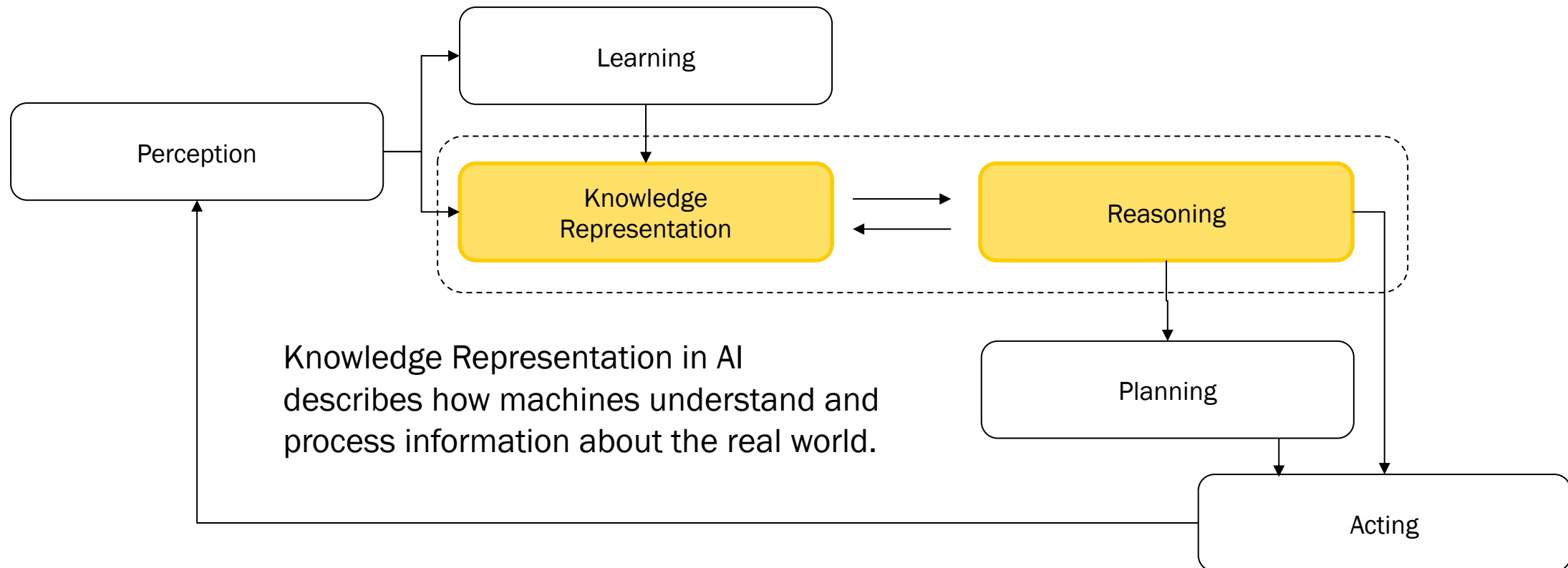
In short: intelligence requires knowledge representations, and those representations must matter to how the system acts.

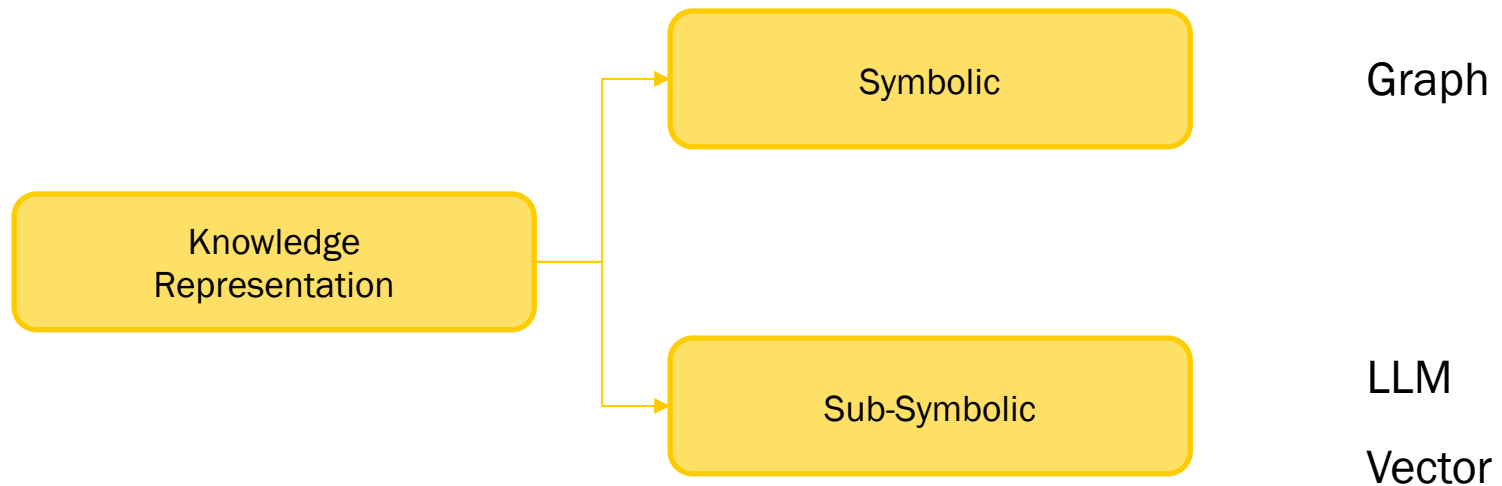
Smith breaks this into two essential claims:

1. **Representational claim:** An intelligent system has internal states (symbols, data structures, models, etc.) that *correspond* to aspects of the world - facts, objects, relations, goals.
2. **Causal efficacy claim:** These representations are not merely descriptive; they **play a functional role** in producing behavior. The system reasons, plans, or acts *because of* them.



Knowledge Representation and Reasoning serves as the foundation for how AI systems interpret and interact with the world around them.

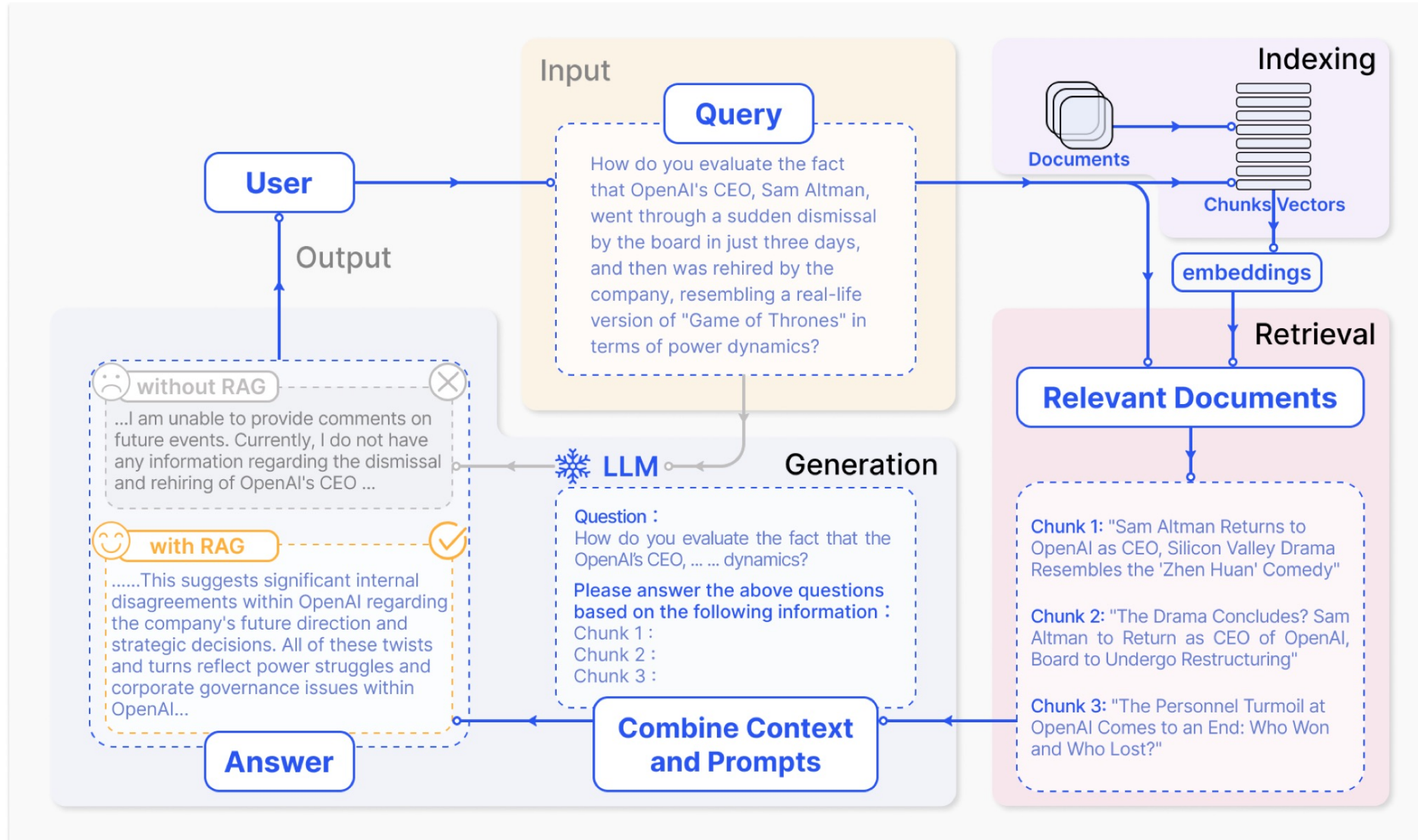




Short Introduction into Retrieval Augmented Generation

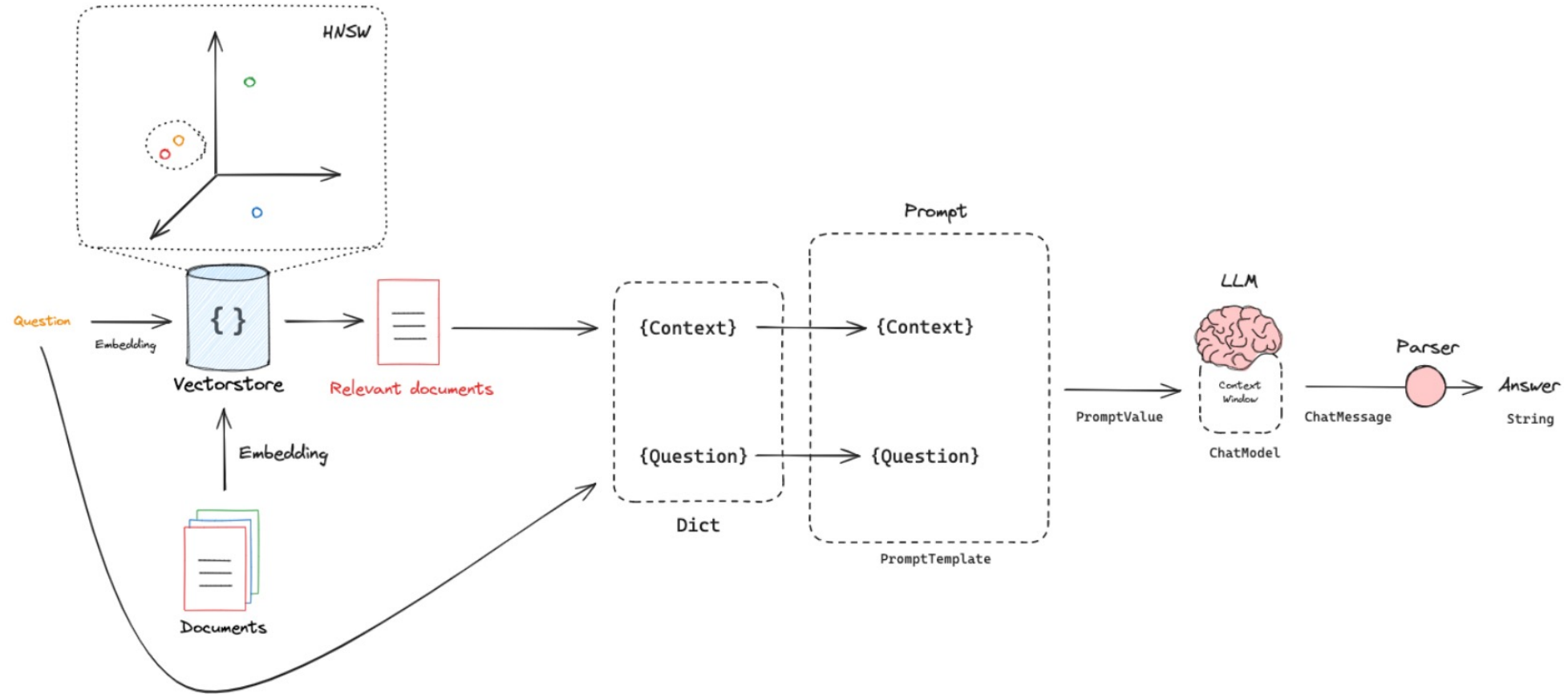


Retrieval Augmented Generation



Source: <https://arxiv.org/pdf/2312.10997>

Retrieval Augmented Generation



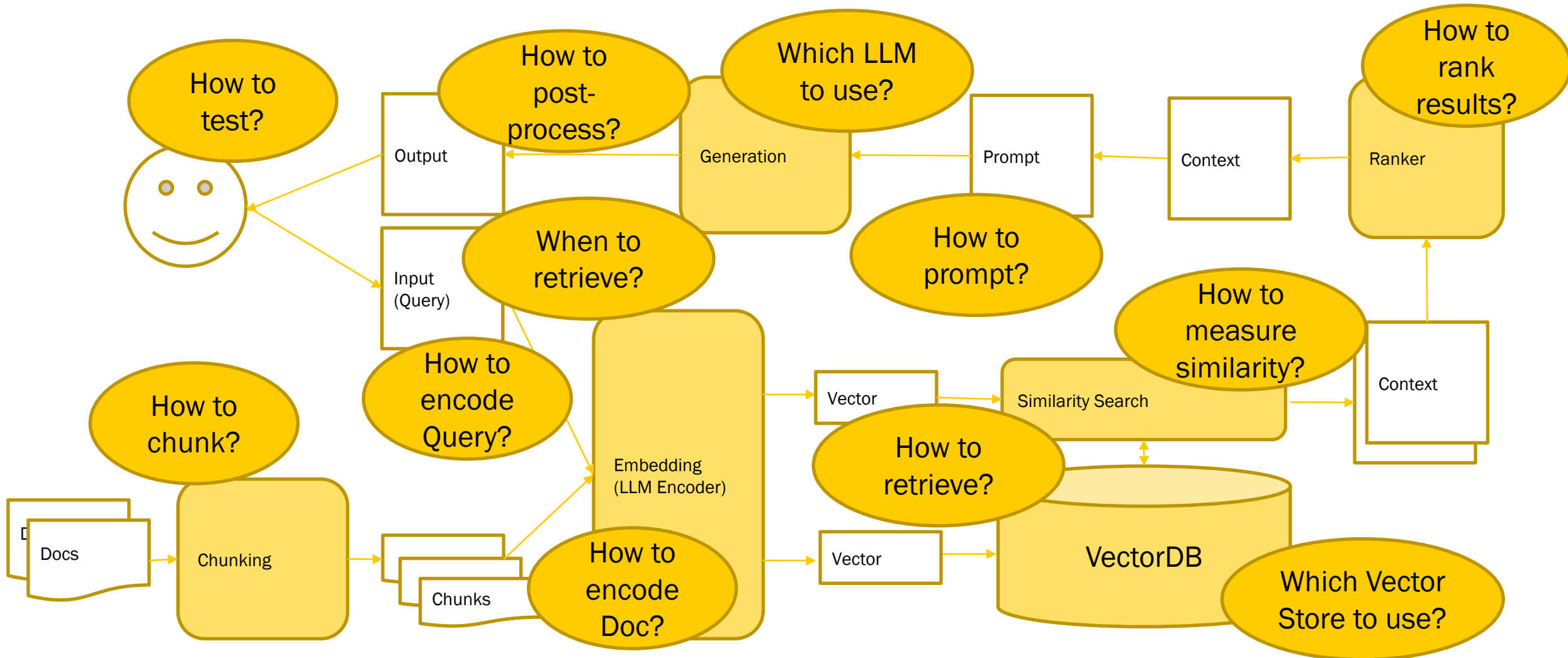
Source: https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_1_to_4.ipynb

Start with simple RAG

- Vectore Database we use ChromaDB
 - Understand Vector Databases: `00_Vectorstore.ipynb`
- RAG Framework we use LlamaIndex
 - Understand RAG `01_Simple_RAG.ipynb`



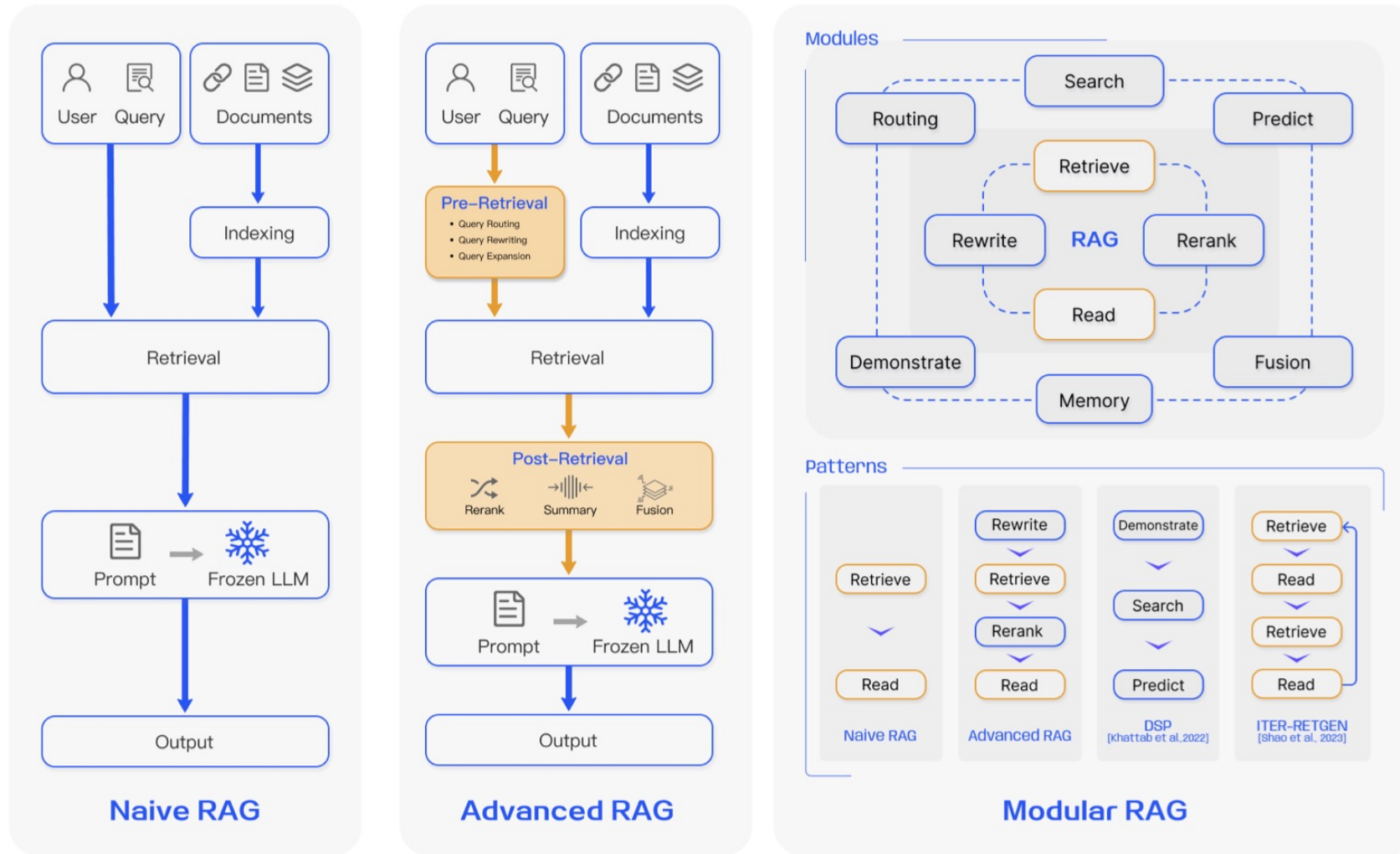
What are the challenges with RAG solutions?



Advanced Retrieval Augmented Generation



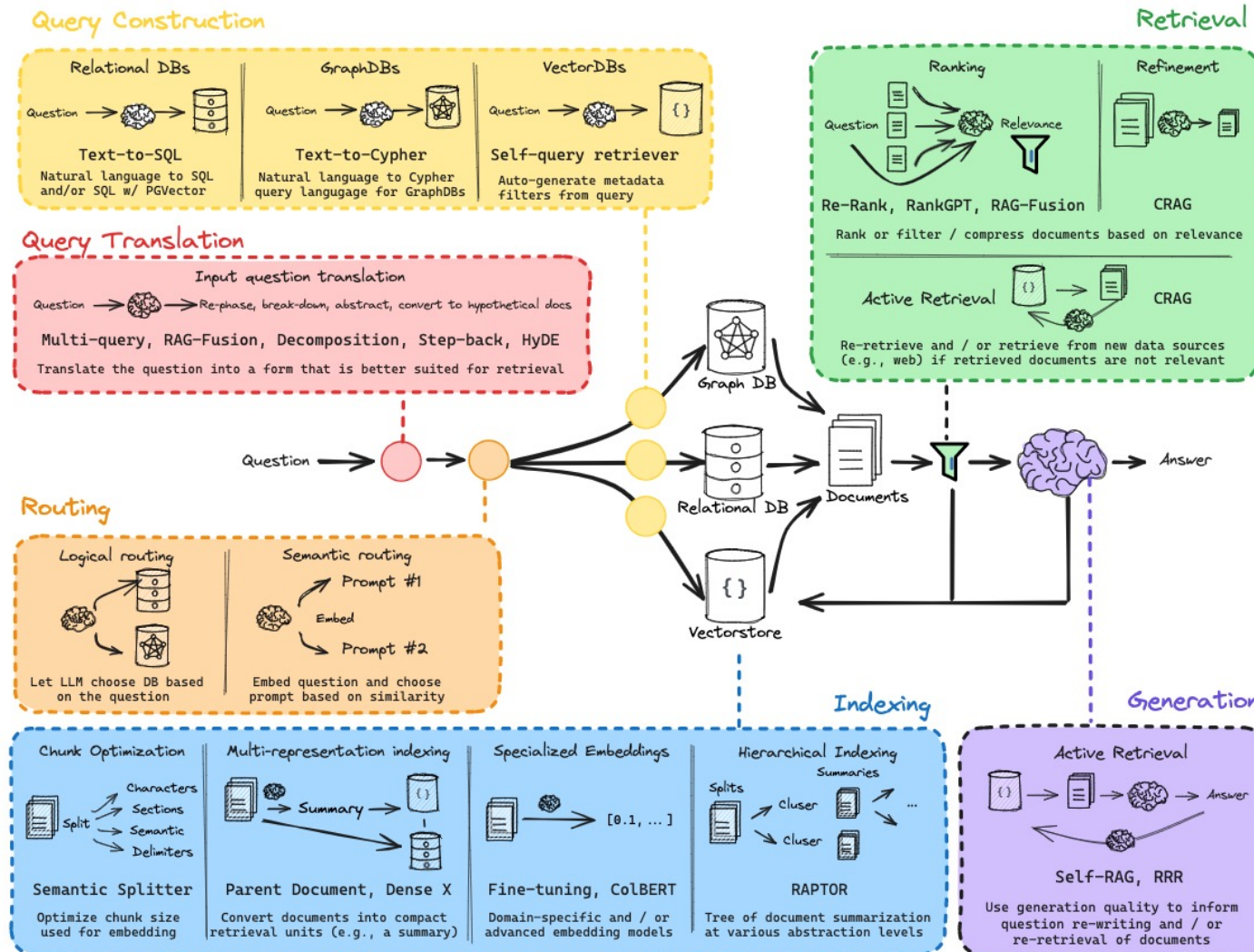
From simple to advanced RAG



Source: <https://arxiv.org/pdf/2312.10997>

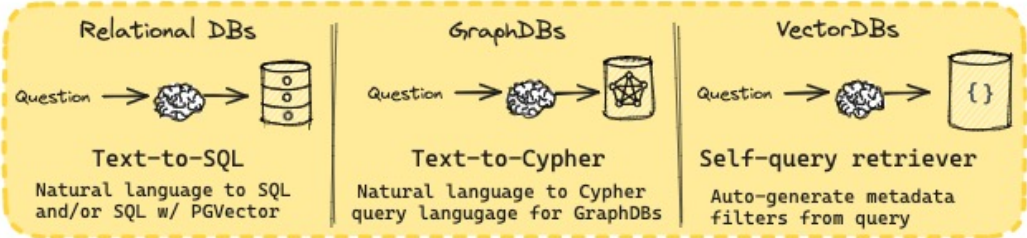
Retrieval Augmented Generation

1. Query Transformation
2. Routing
3. Query construction
4. Retrieval
5. Generation

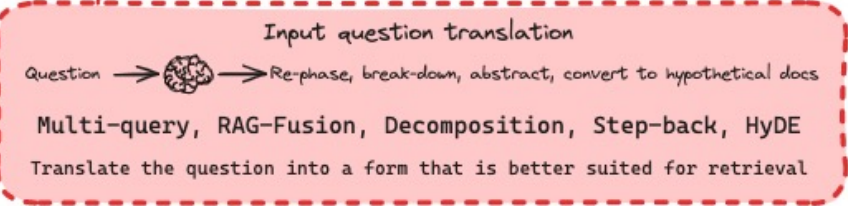


Source: Langchain https://docs.google.com/presentation/d/1C9laAwHoWcc4RSTqo-pCoN3hOnCgqV2JEYZUJunv_9Q

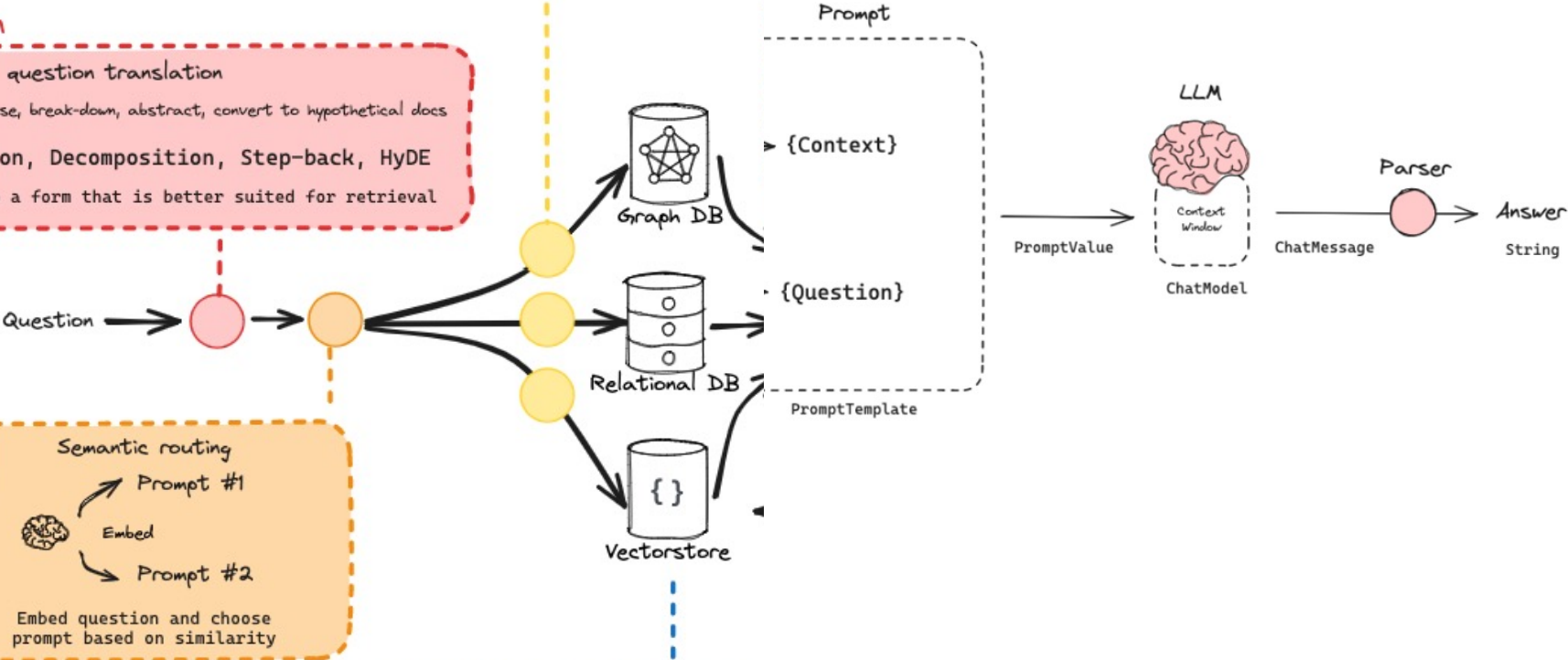
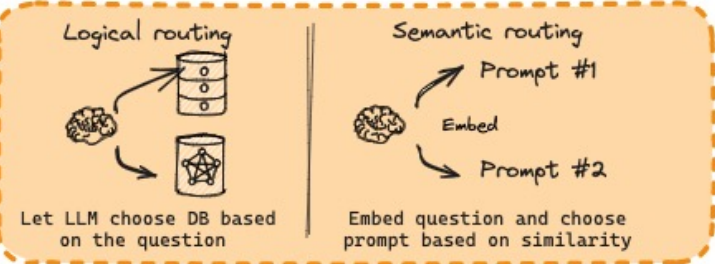
Query Construction

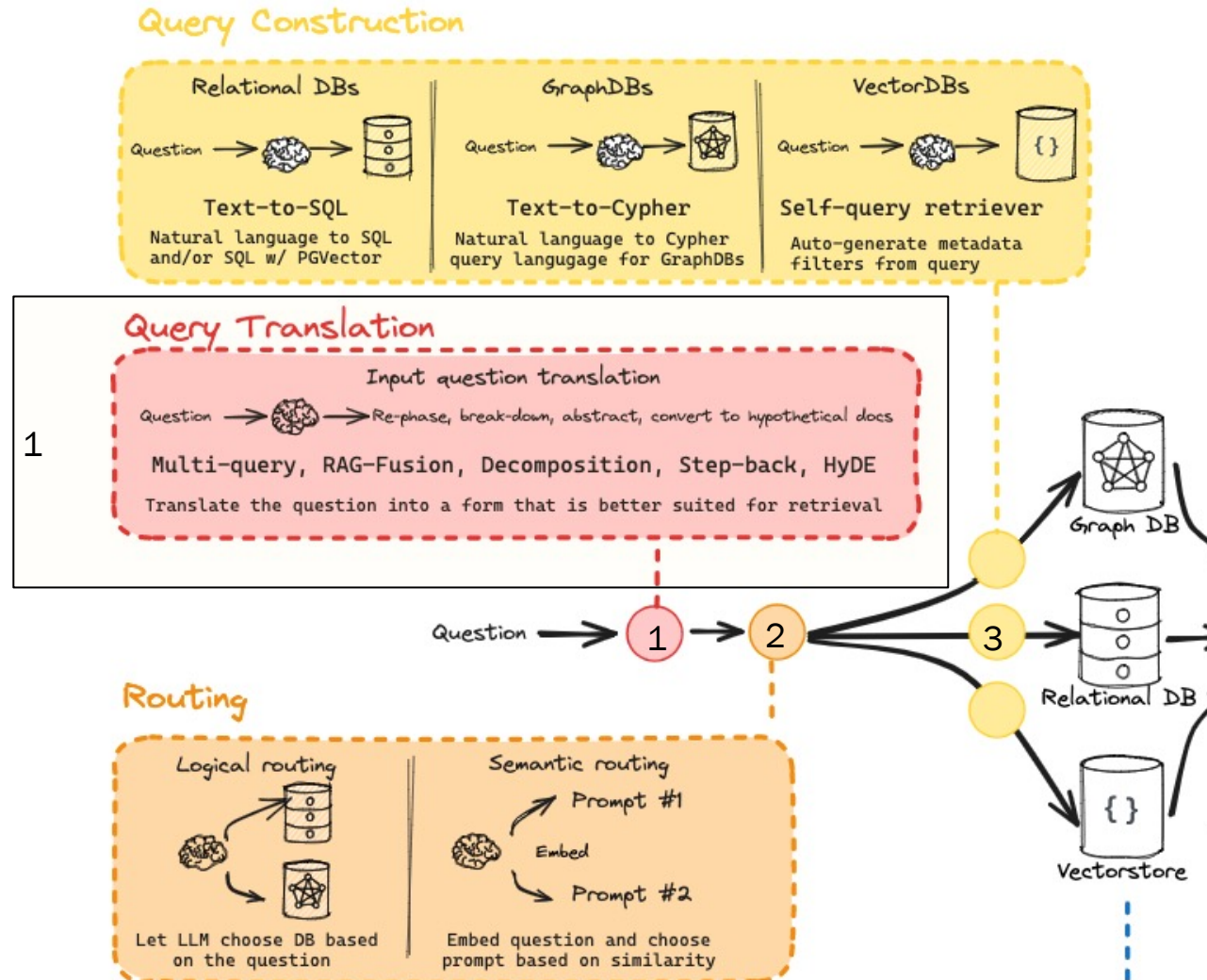


Query Translation



Routing

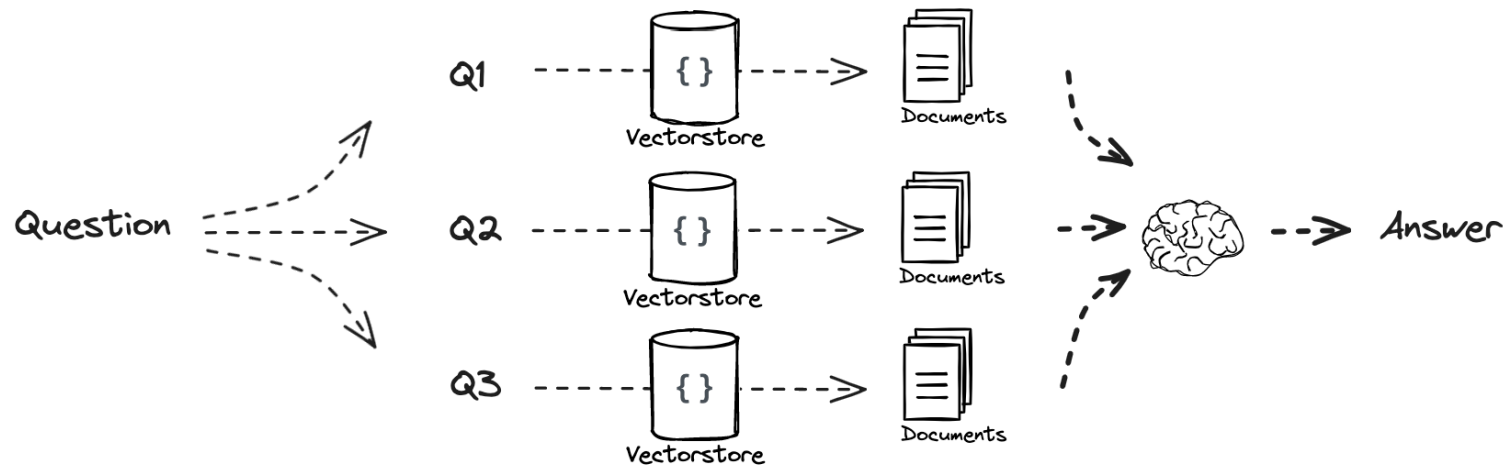




Goal: Translate the input question in a form that is better suited for retrieval:

- Multi-Query
- RAG-Fusion
- Decomposition
- Step-back
- HyDE

Query Translation: Multi-Query

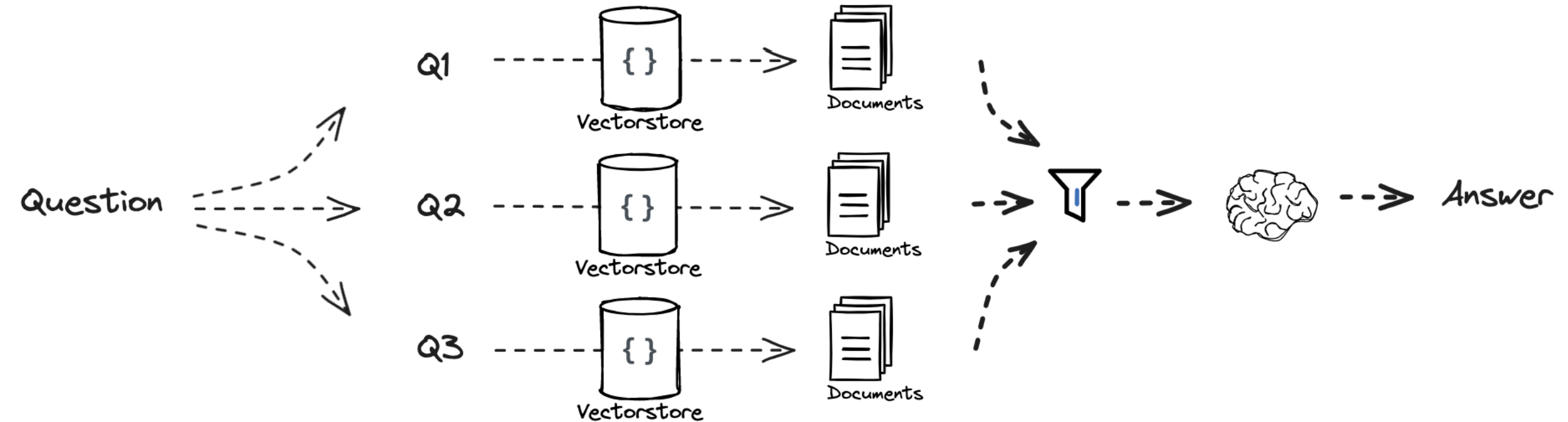


Prompt:

""You are an AI language model assistant. Your task is to generate five different versions of the given user question to retrieve relevant documents from a vector database. By generating multiple perspectives on the user question, your goal is to help the user overcome some of the limitations of the distance-based similarity search. Provide these alternative questions separated by newlines. Original question: {question}""

Source: https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_5_to_9.ipynb

Query Transition: RAG Fusion

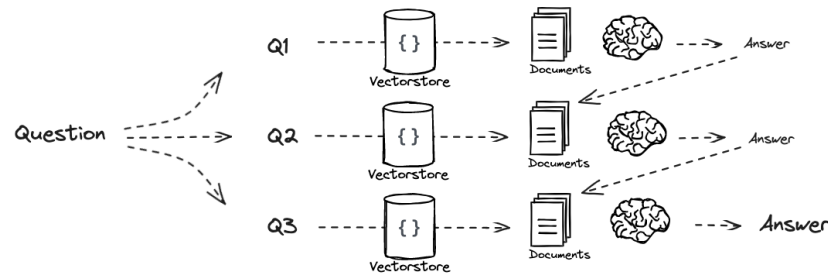


Source: https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_5_to_9.ipynb

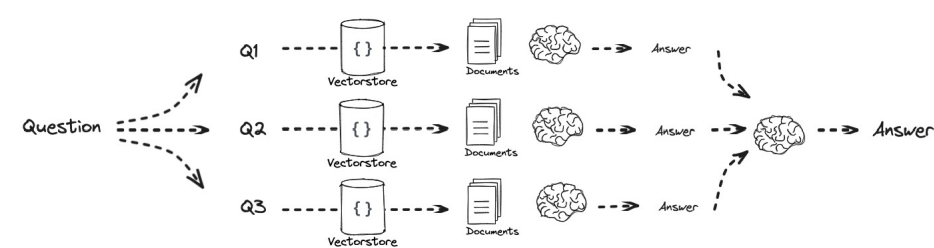
Query Translation: Decomposition

Decompose the question into subqueries and

answer recursively



or individually



Prompt:

""You are a helpful assistant that generates multiple sub-questions related to an input question.

The goal is to break down the input into a set of sub-problems / sub-questions that can be answers in isolation.

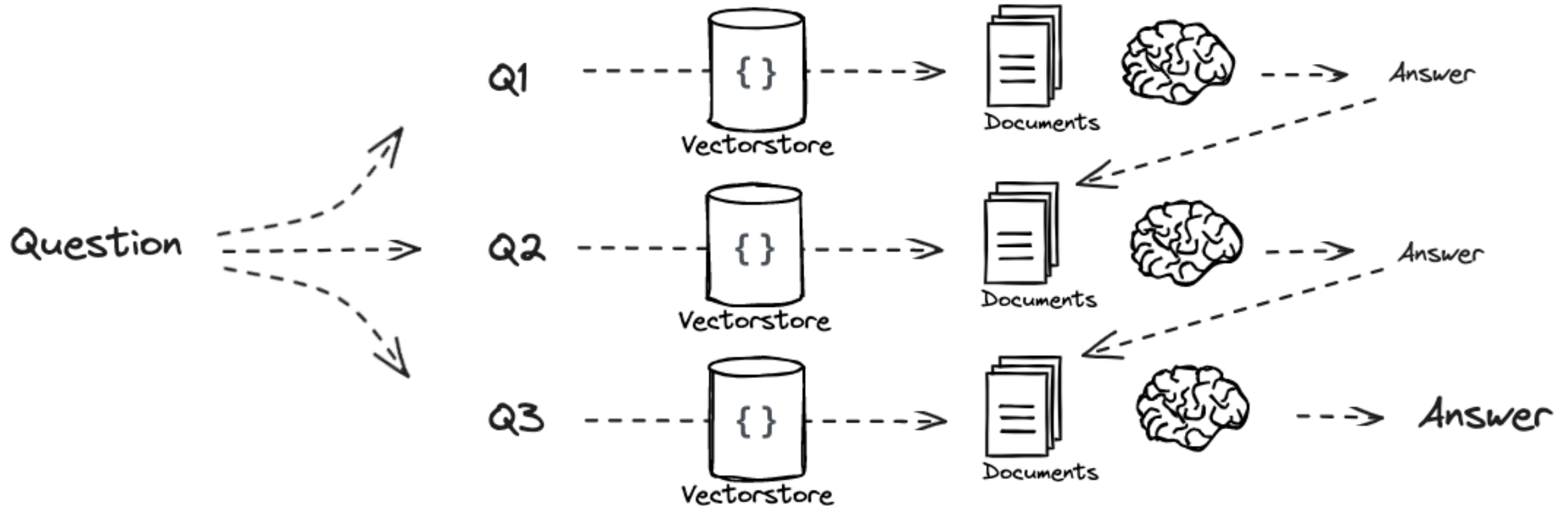
Generate multiple search queries related to: {question}

Output (3 queries):""

Source: https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_5_to_9.ipynb

Query Transition: Decomposition

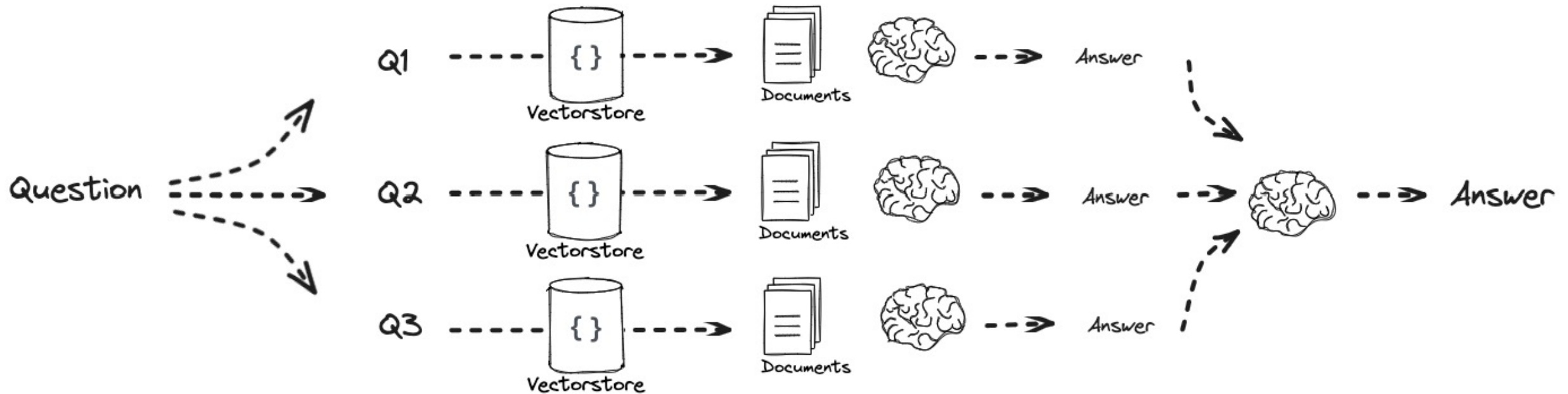
Answer recursively



Source: https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_5_to_9.ipynb
<https://arxiv.org/pdf/2205.10625.pdf>
<https://arxiv.org/abs/2212.10509.pdf>

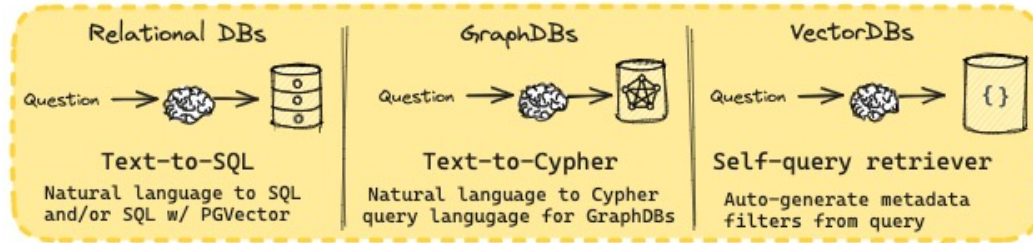
Query Transition: Decomposition

Answer individually

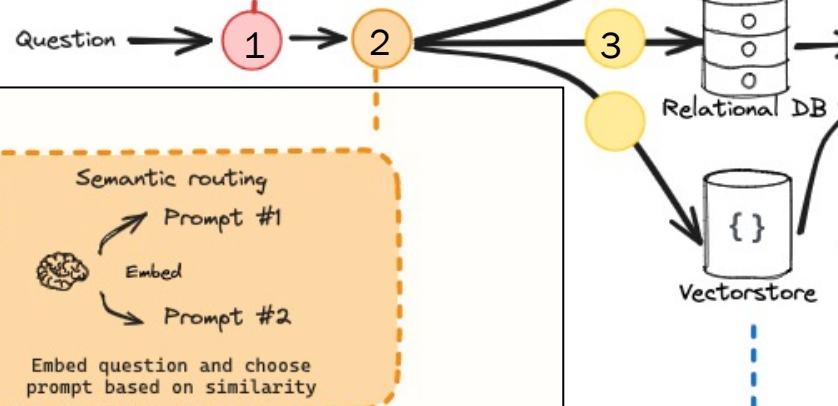
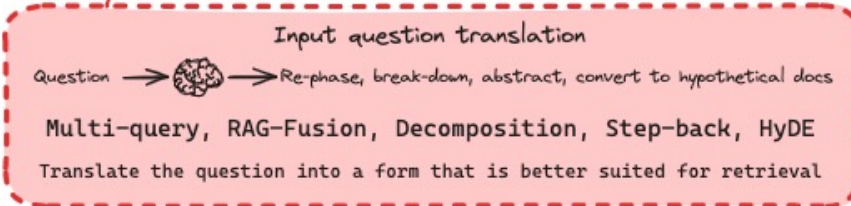


Source: https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_5_to_9.ipynb

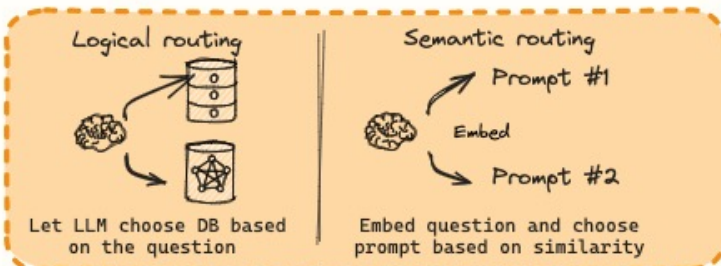
Query Construction



Query Translation



Routing

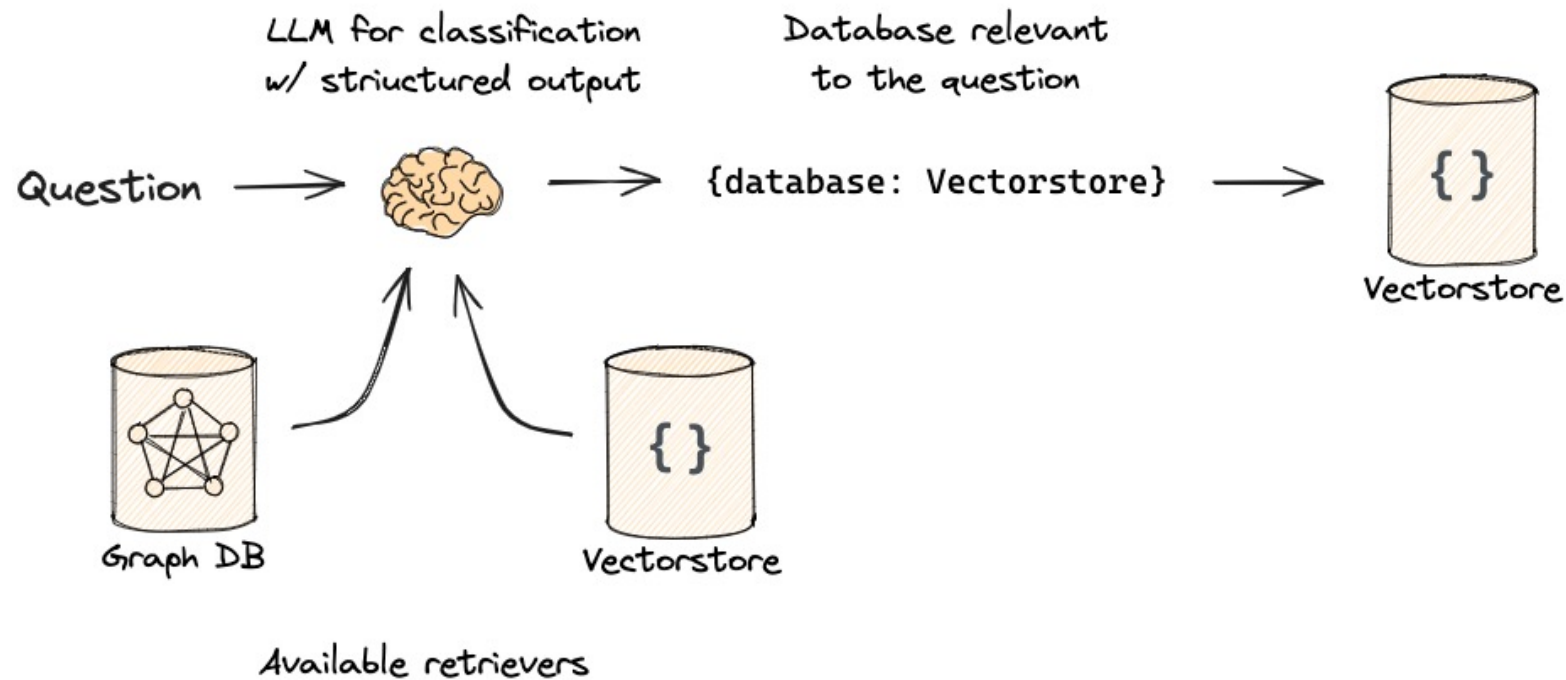


Goal:

- Logical Routing: Find the right Datastore based on question
- Semantic Routing: Choose the right prompt based on similarity

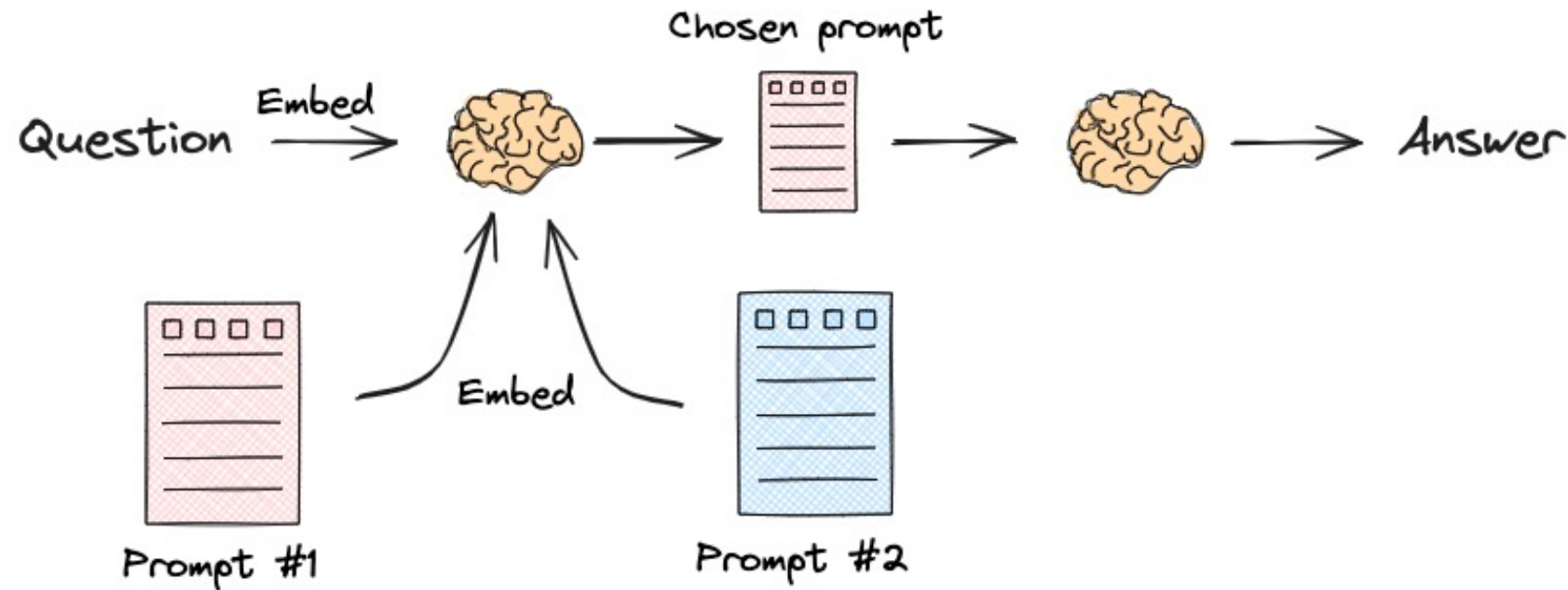
Logical Routing

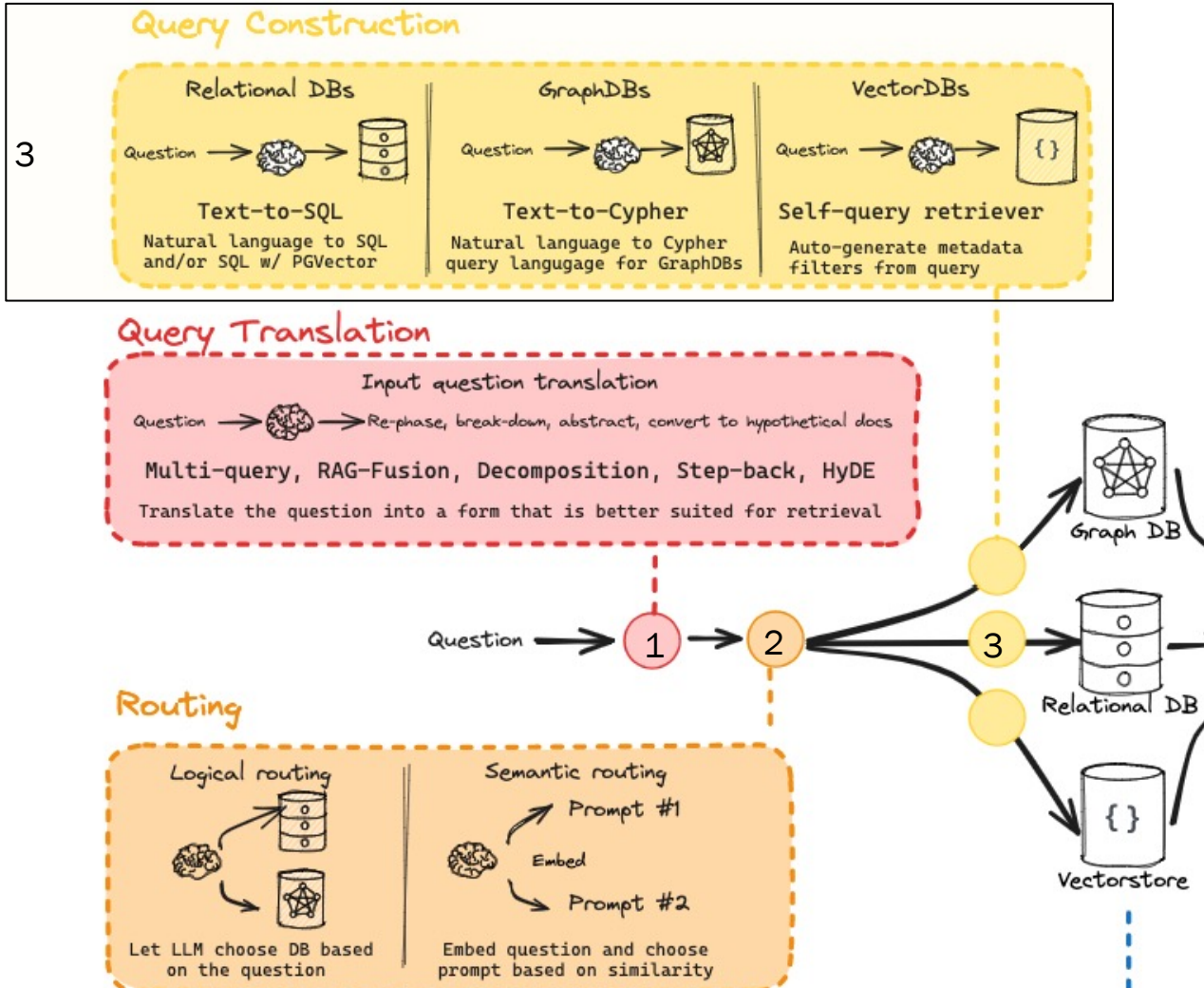
In case we have different available retrievers: where to retrieve?



Semantic Routing

In case we have multiple prompts: which prompt should we use?





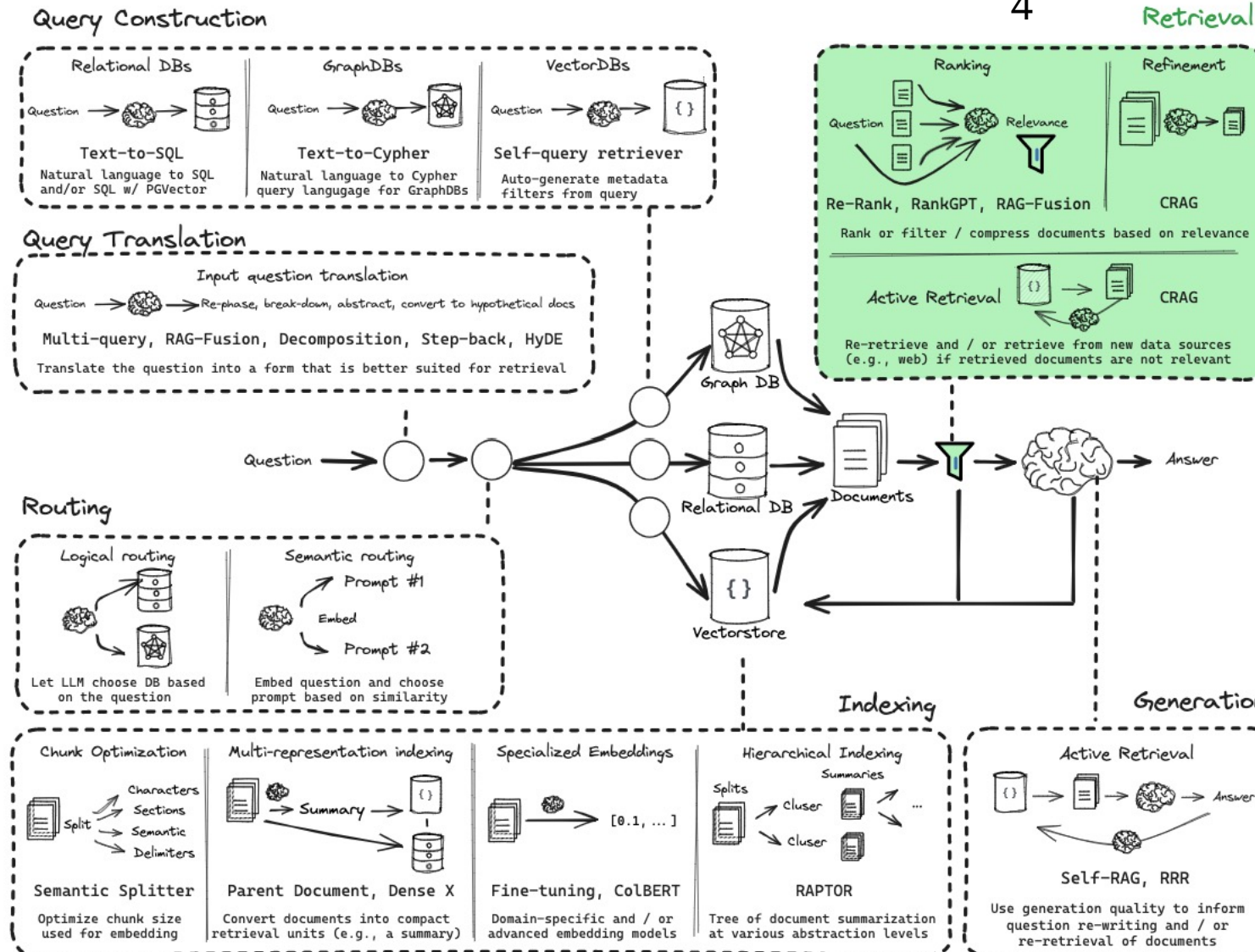
Goal:

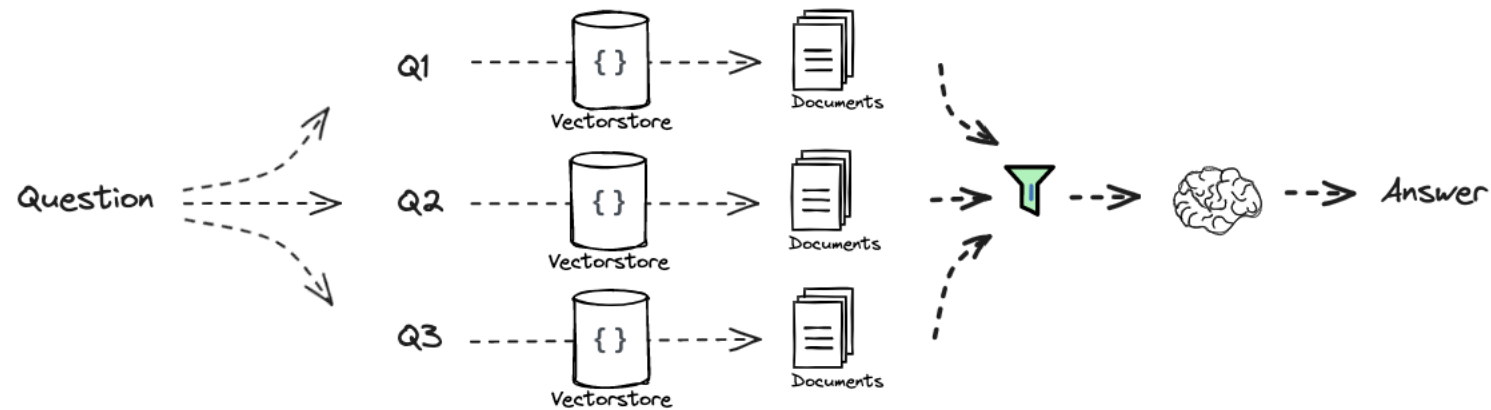
Extract the information from question we need to structure our query:

- Text-to-SQL
- Text-to-Cypher
- Extract Metadata, Entities, ...

4

Retrieval





<https://medium.com/@sahin.samia/what-is-reranking-in-retrieval-augmented-generation-rag-ee3dd93540ee>

<https://medium.com/@rossashman/the-art-of-rag-part-3-reranking-with-cross-encoders-688a16b64669>

1. Neural Rerankers:

- **Cross-Encoders**: These models jointly encode the query and each retrieved document, assessing their relevance through a single forward pass. This approach captures intricate interactions between the query and documents, leading to precise relevance scoring. However, it can be computationally intensive.
- **Bi-Encoders with Interaction Layers**: While bi-encoders independently encode queries and documents, incorporating interaction layers allows for modeling complex relationships, balancing efficiency and accuracy.

2. Traditional Scoring Techniques:

- **BM25**: A probabilistic retrieval model that ranks documents based on term frequency and inverse document frequency, effectively handling exact term matches.
<https://homepages.dcc.ufmg.br/~nivio/cursos/ri10/transp/slideschap04c.pdf>
https://en.wikipedia.org/wiki/Okapi_BM25
- **TF-IDF**: Evaluates the importance of terms within documents relative to the entire corpus, aiding in identifying key terms that distinguish relevant documents.

3. Hybrid Approaches:

- **Combining Neural and Traditional Methods**: Integrating neural rerankers with traditional models like BM25 can leverage the strengths of both, enhancing reranking performance.



4. Graph-Based Reranking:

- Document Graphs: Constructing graphs where nodes represent documents and edges denote semantic relationships enables the identification of clusters of relevant documents, improving reranking outcomes.

5. Fine-Tuning LLMs for Reranking:

- Instruction Fine-Tuning: Training LLMs with specific instructions for context ranking and answer generation allows a single model to perform both tasks, streamlining the reranking process.

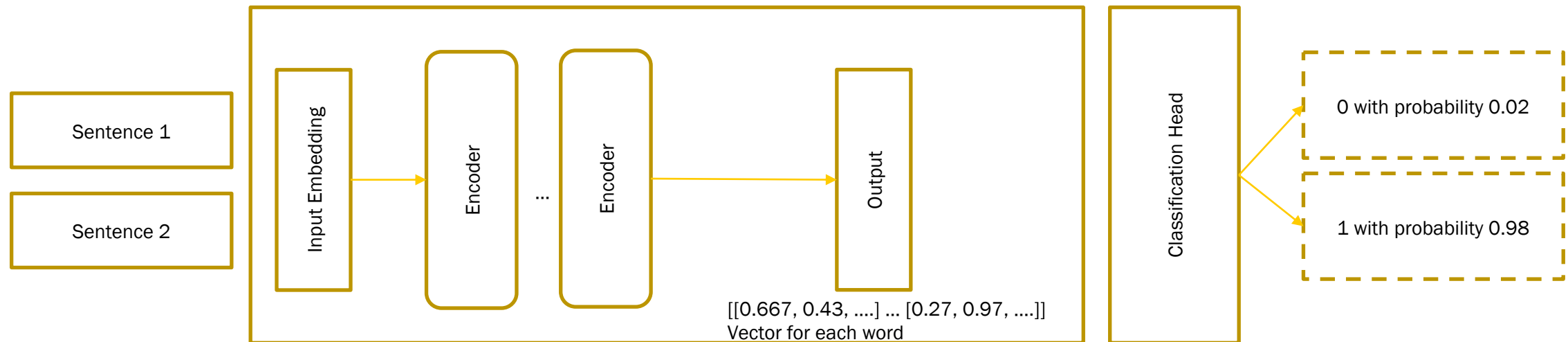
6. Fusion-Based Techniques:

- Fusion-in-Decoder (FiD): This method processes multiple retrieved documents simultaneously within the decoder, enabling the model to synthesize information from various sources effectively.

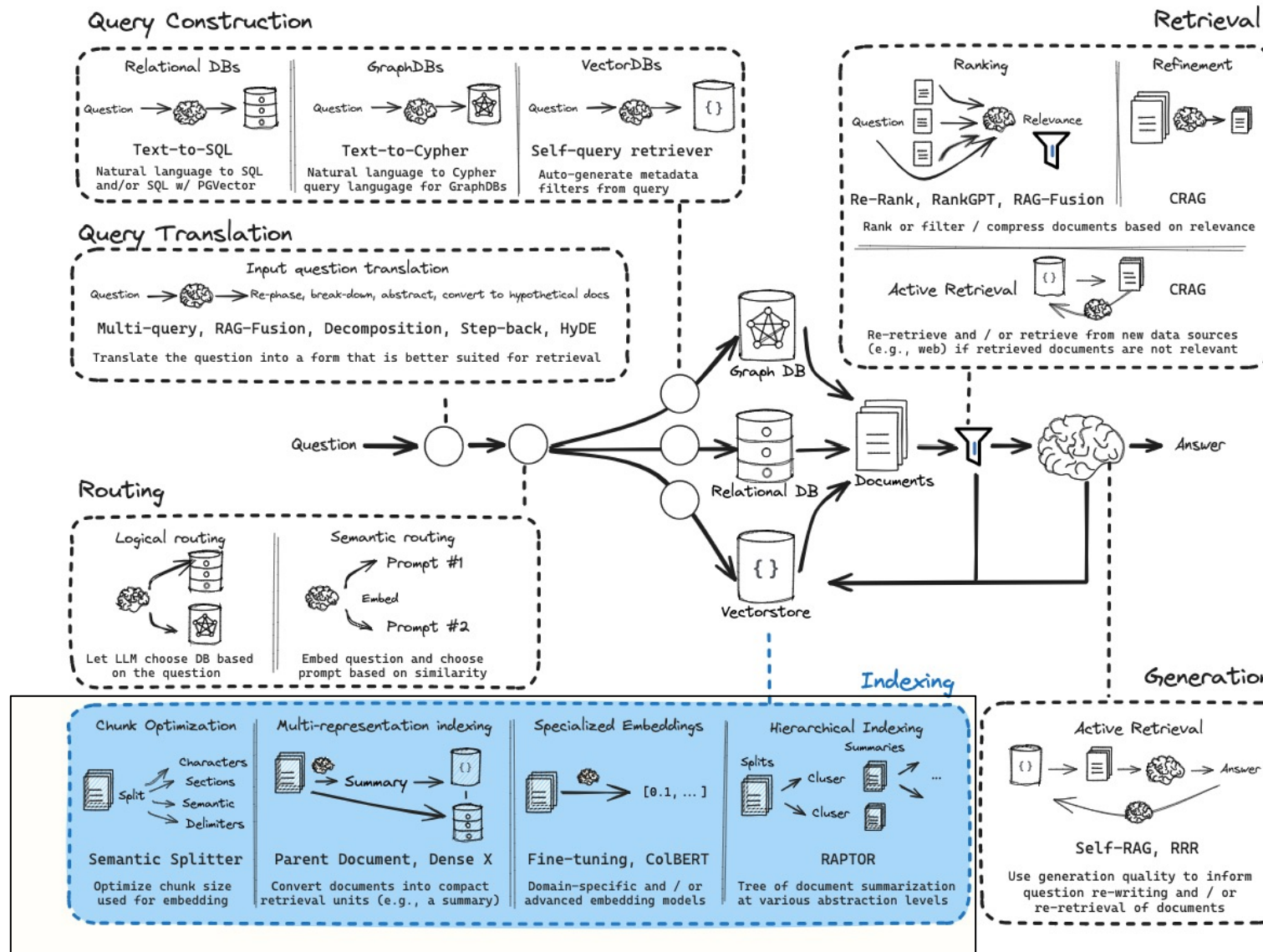


Encoder-Only Models for Semantic Similarity Computation

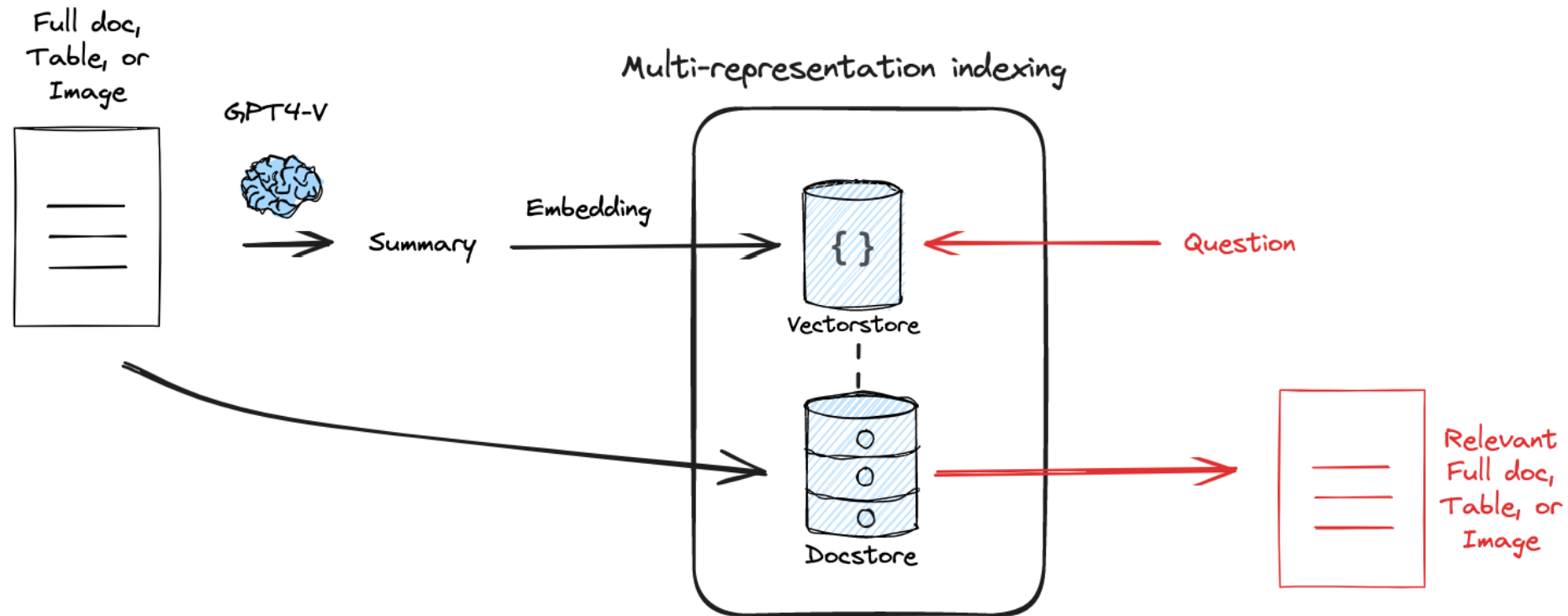
- CrossEncoder is used to measure similarity between pairs of sentences
- The input of the model always consists of a data pair, for example two sentences, and outputs a value between 0 and 1 indicating the similarity between these two sentences
- can be used as Re-Ranker in Retrieval-Augmented Generation (RAG)



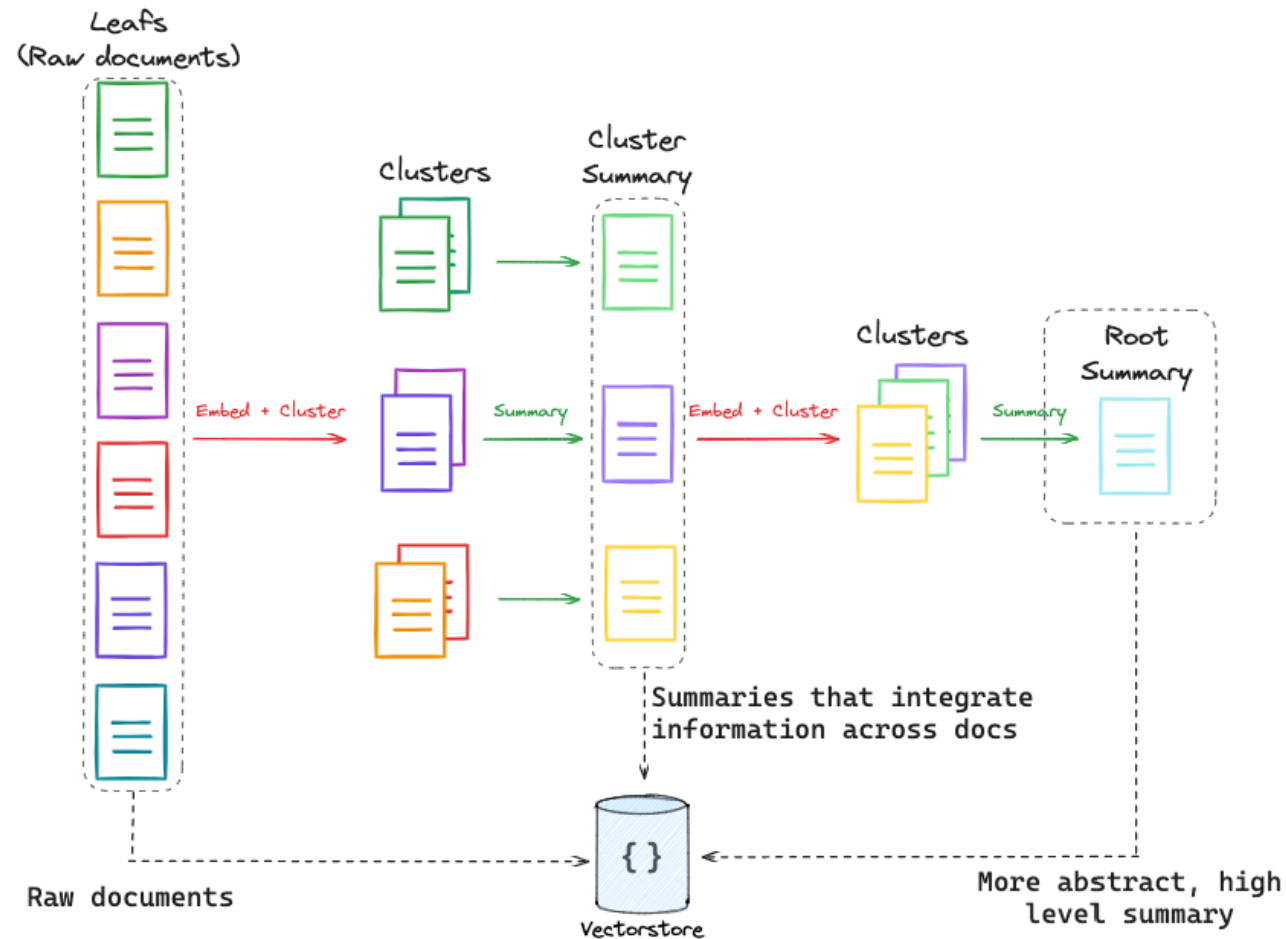
https://www.sbert.net/examples/applications/retrieve_rerank/README.html



Multi-Representation Indexing



https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_12_to_14.ipynb



https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_12_to_14.ipynb

Symbolic Knowledge Representation with Graph RAG



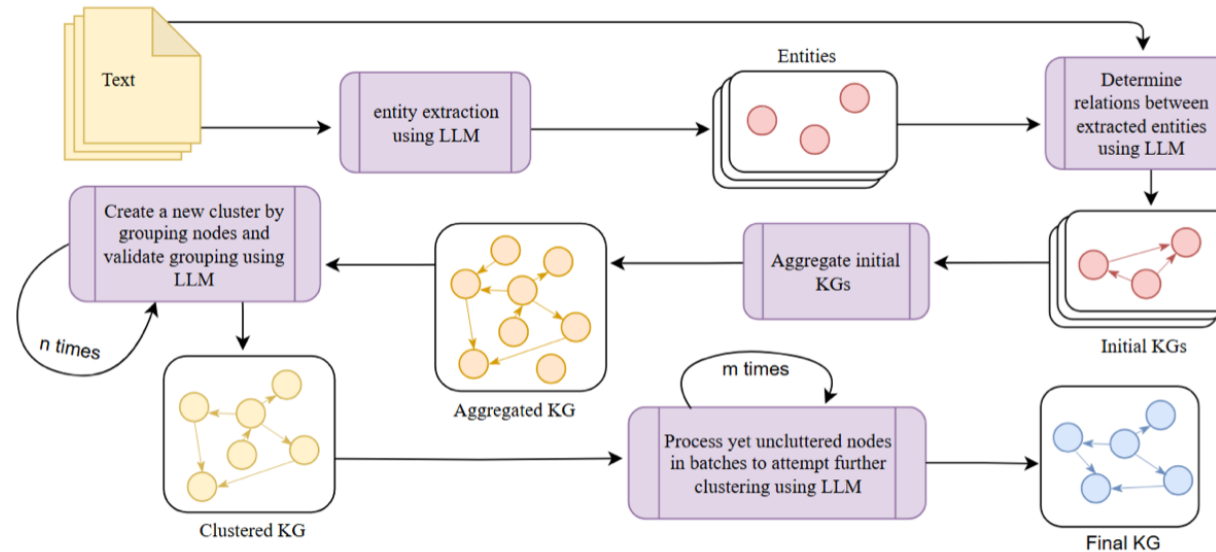
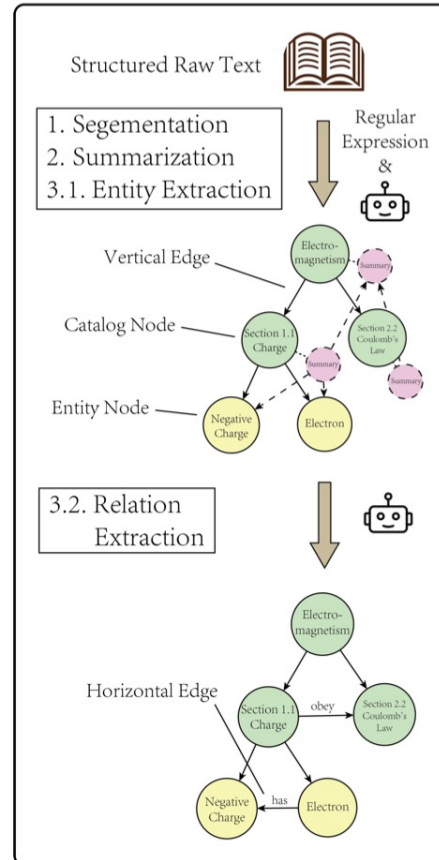


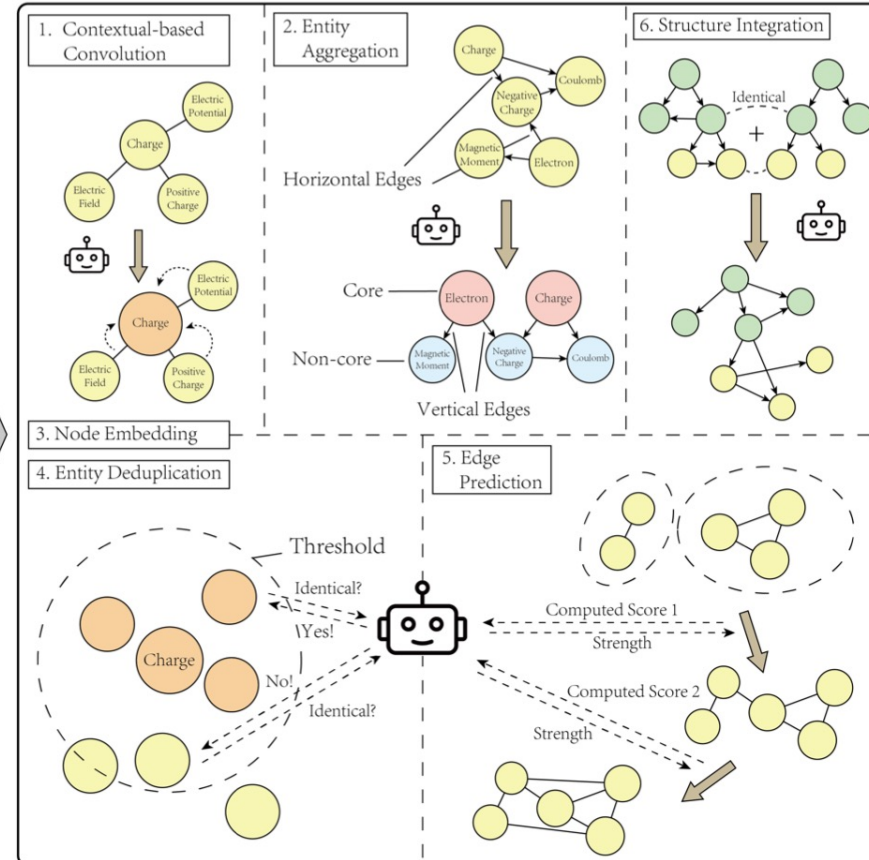
Figure 1: KGGen extraction method

Source: <https://arxiv.org/pdf/2502.09956>
<https://github.com/stair-lab/kg-gen/>

Initial Construction



Iterative Expansion

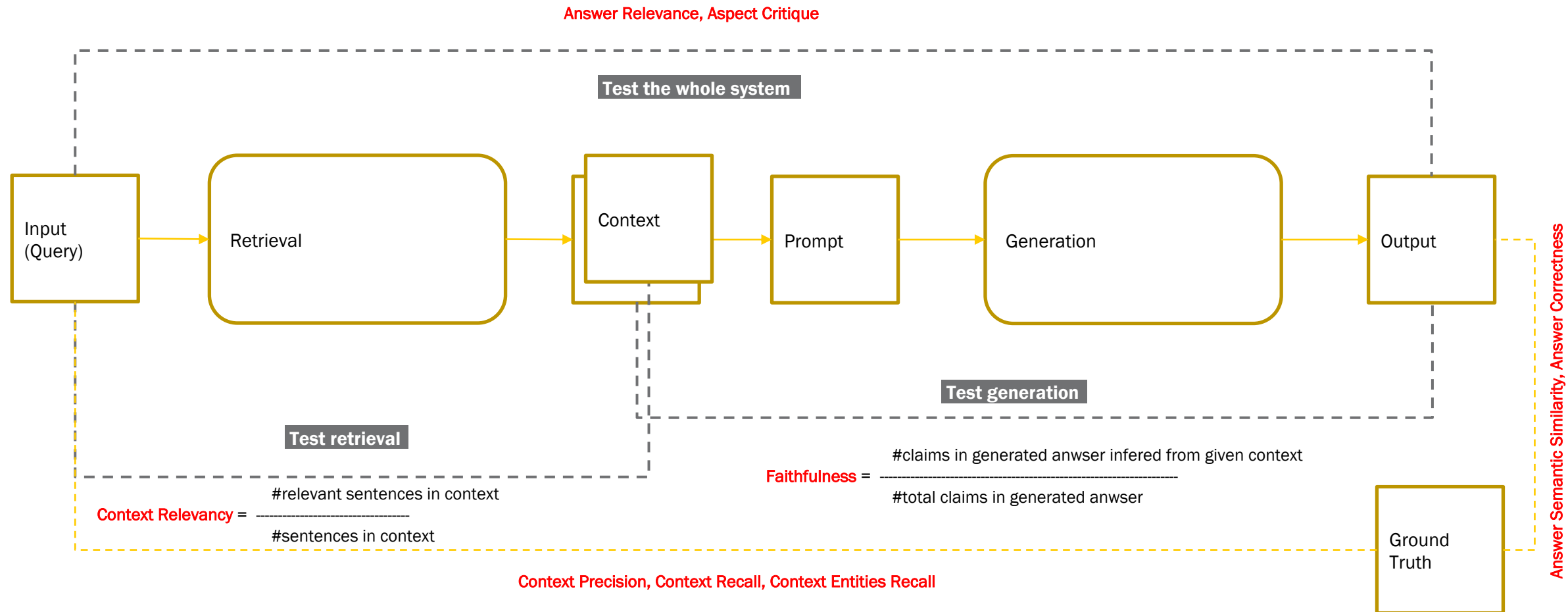


Source: <https://aclanthology.org/2025.acl-long.907.pdf>

RAG Evaluation & Tuning



<https://docs.ragas.io/en/latest/>



Intelligente Informationssysteme 4b Agentic Infrastructure

Dominik Neumann



04.01 Asynchronous Programming

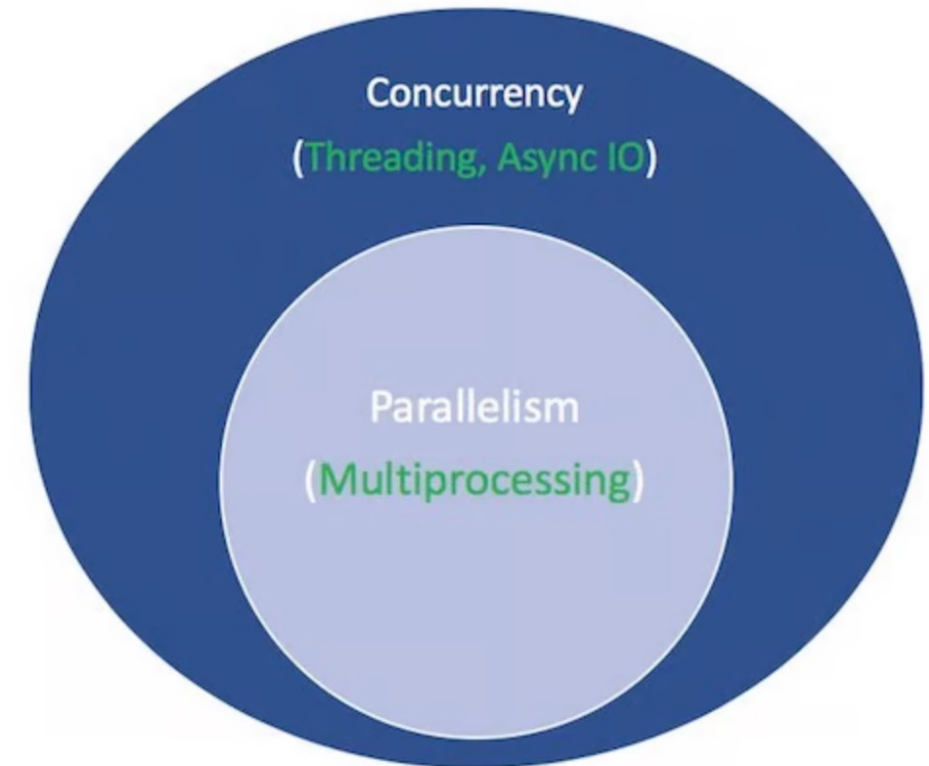


Concurrency versus Parallelism

- **Parallelism** consists of executing multiple operations at the same time.
- **Multiprocessing** is a means of achieving parallelism that entails spreading tasks over a computer's central processing unit (CPU) cores. Multiprocessing is well-suited for CPU-bound tasks, such as tightly bound for loops and mathematical computations.
- **Concurrency** is a slightly broader term than parallelism, suggesting that multiple tasks have the ability to run in an overlapping manner. Concurrency doesn't necessarily imply parallelism.
- **Threading** is a concurrent execution model in which multiple threads take turns executing tasks. A single process can contain multiple threads.

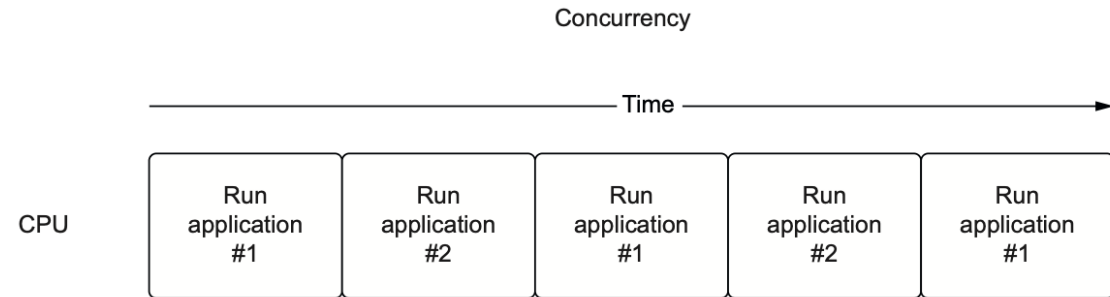
But Python GIL (Global Interpreter Lock) is a lock that allows only one thread to hold the control of the Python interpreter.

Source: <https://realpython.com/async-io-python/>

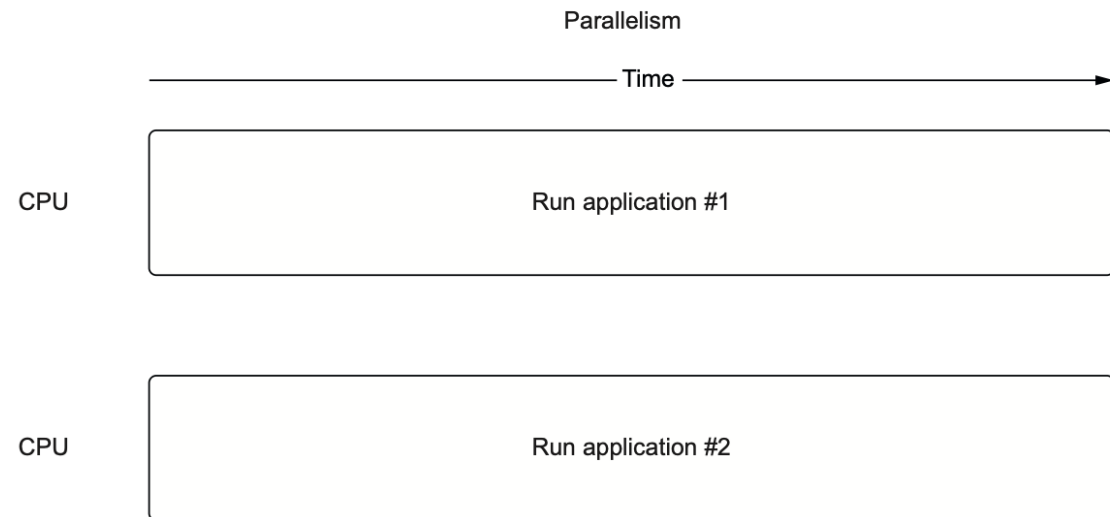


Concurrency versus Parallelism

With **concurrency**, we switch between running two applications.



With **parallelism**, we actively run two applications simultaneously.

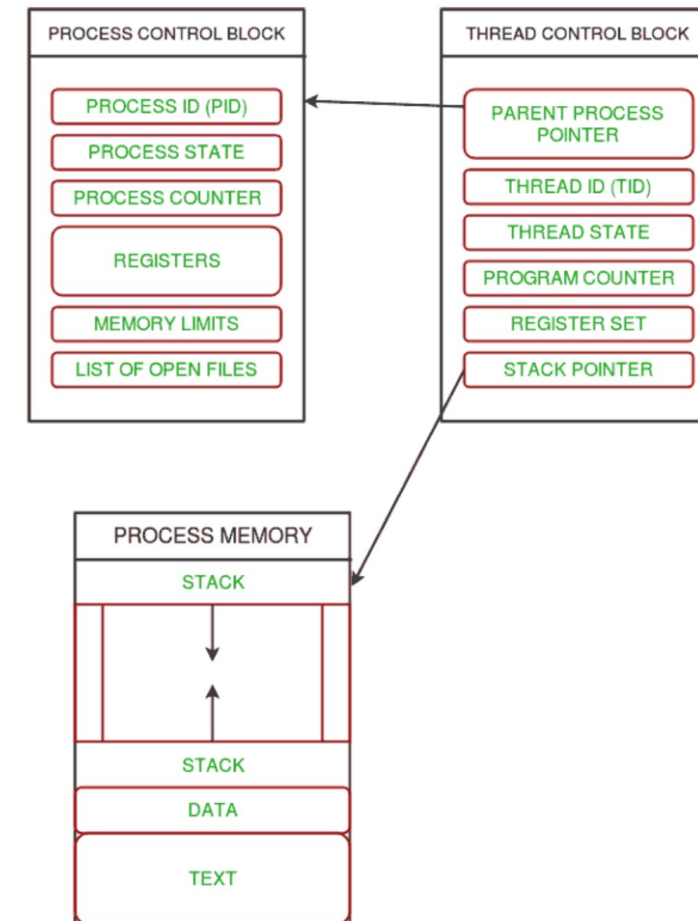
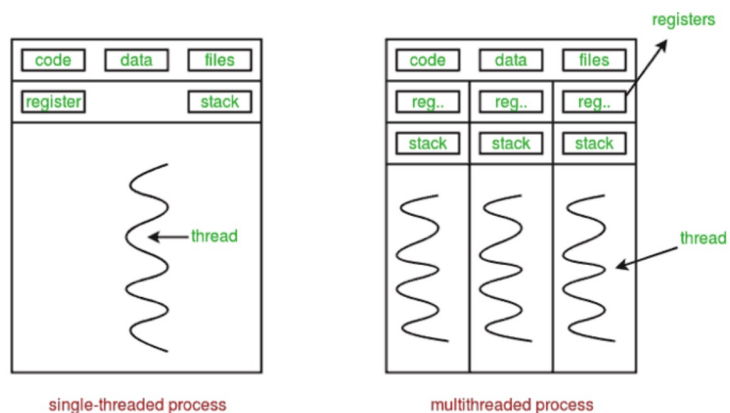


Source: Python Concurrency with asyncio



Relationship between Process and Thread:

- A process is managed by a PCB.
- Each thread is managed by a TCB and linked to its process.
- Threads share the process's code and data but have their own stacks.



Source: <https://www.geeksforgeeks.org/python/multithreading-python-set-1/>

Async I/O is a single-threaded, single-process technique that uses cooperative multitasking. Async I/O gives a feeling of concurrency despite using a single thread in a single process.

What does it mean for something to be **asynchronous**?

- Asynchronous routines can pause their execution while waiting for a result and allow other routines to run in the meantime
- Asynchronous code facilitates the concurrent execution of tasks by coordinating asynchronous routines.

Source: <https://realpython.com/async-io-python/>



Asynchronous Programming

asyncio — Asynchronous I/O

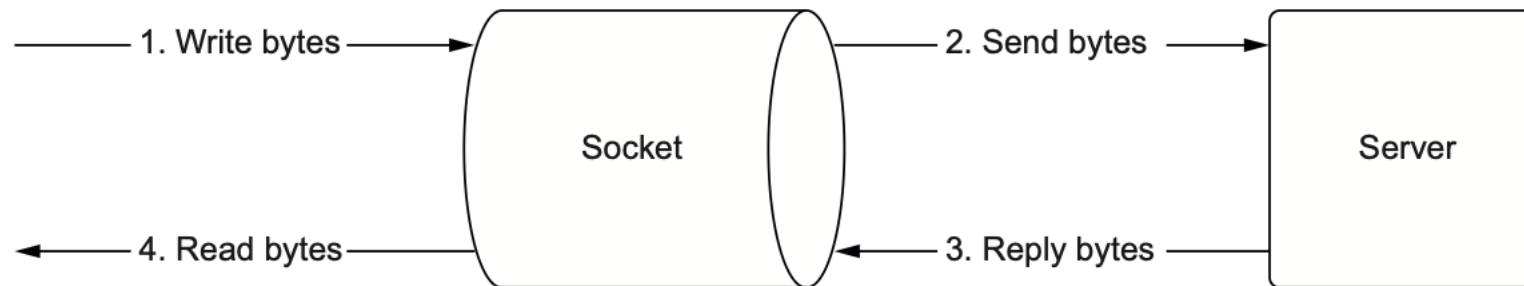
Language Level Support for Threading in Python

- **async**: modifies a function to be run asynchronously. It can be run independently of the main thread.
- **await**: joins the thread the function is running in with the main thread.
- A **Future** object is a wrapped-function that will execute some point the future. (inside the future there is a function executed in the future). You can ask the future if or not the function has been executed. It has three states:
 - Unfulfilled: the wrapped function has not completed
 - Fulfilled : the wrapped function has completed and results are available
 - Failed: : the wrapped function threw an exception

Asynchronous Programming

asyncio — Asynchronous I/O

Sockets are blocking by default. Simply put, this means that when we are waiting for a server to reply with data, we halt our application or block it until we get data to read. Thus, our application stops running any other tasks until we get data from the server, an error happens, or there is a timeout.



Source: Python Concurrency with asyncio

Asynchronous Programming

How single-threaded with asyncio concurrency works

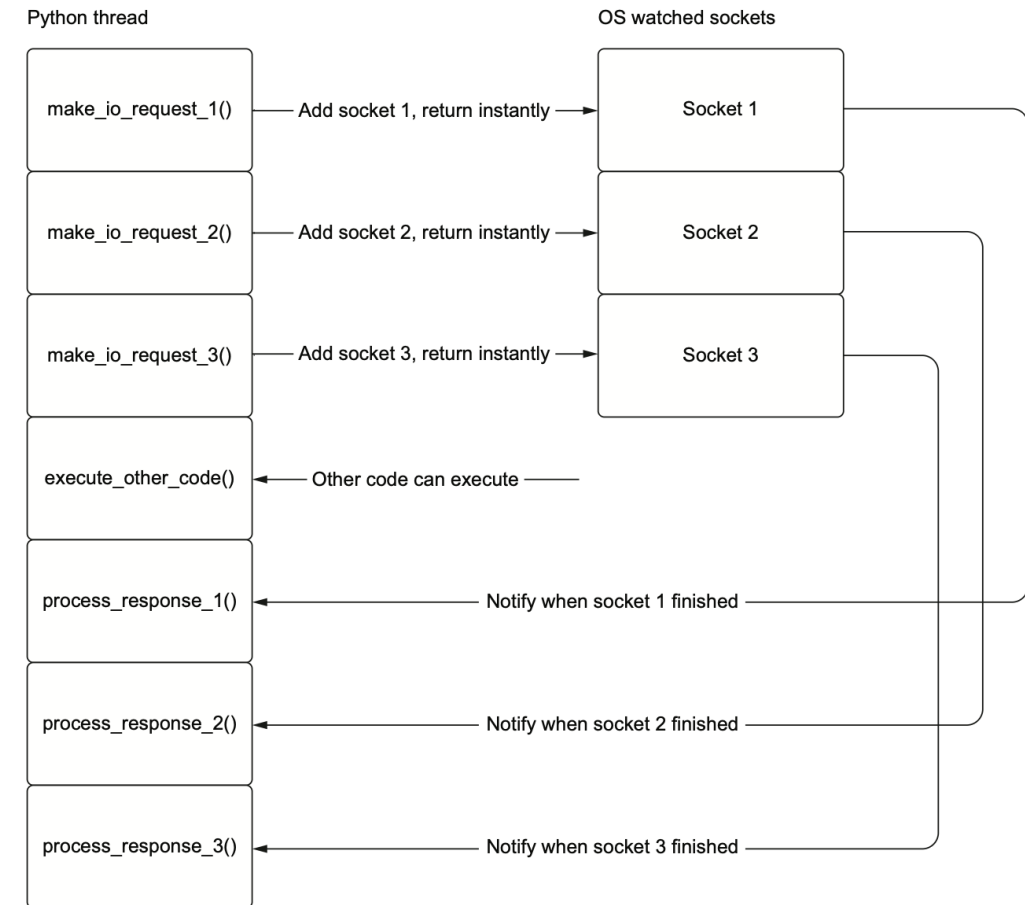
In asyncio's model of concurrency, we have only one thread executing Python at any given time.

When we hit an I/O operation, we hand it over to our operating system's event notification system to keep track of it for us.

Once we have done this handoff, our Python thread is free to keep running other Python code or add more non-blocking sockets for the OS to keep track of for us.

When our I/O operation finishes, we “wake up” the task that was waiting for the result and then proceed to run any other Python code that came after that I/O operation.

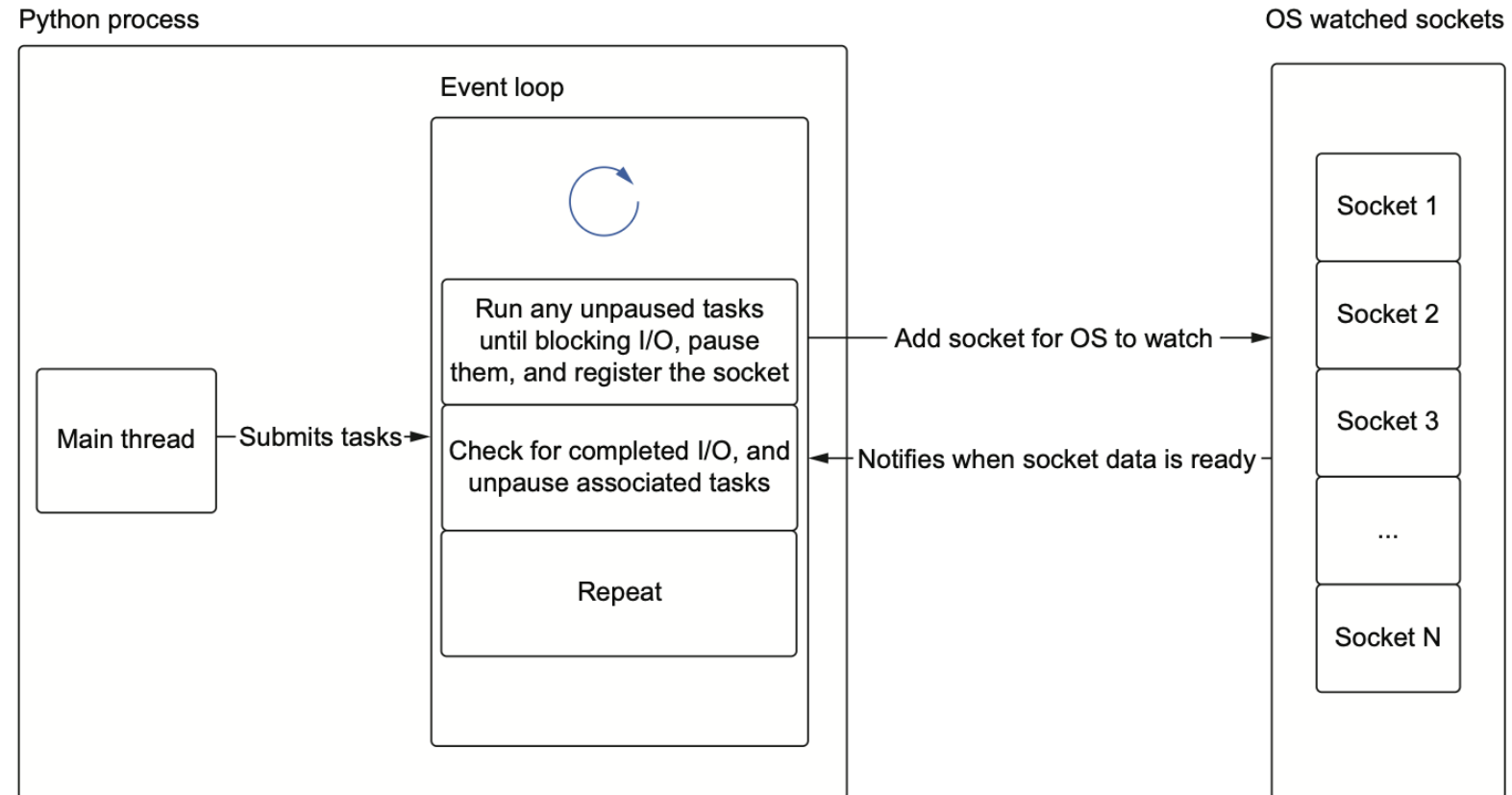
Source: **Python Concurrency with asyncio**



Asynchronous Programming

Event loop in asyncio

Asyncio is using an event loop

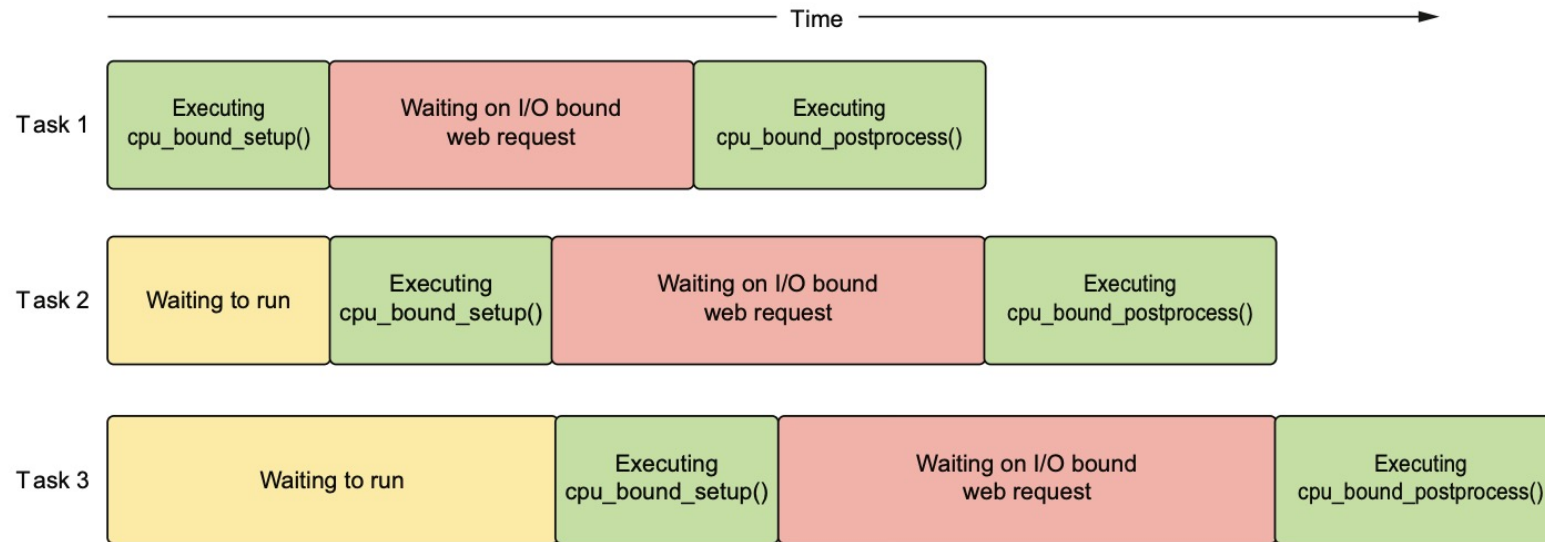


Source: Python Concurrency with asyncio

Asynchronous Programming

Event loop in asyncio

Asyncio is using an event loop



Source: Python Concurrency with asyncio

04.02 ASGI



ASGI - Asynchronous Server Gateway Interface

- WSGI was born in 2004 to decouple web servers from web frameworks with a standardized interface: PEP-333.
- WSGI is now the de facto standard for web application deployment.
- ASGI is a way to abstract frameworks from web servers



04.03 Agentic Infrastructure



BeeAI Framework

BeeAI Framework is an open-source framework for building production-grade multi-agent systems. It is hosted by the Linux Foundation under open governance, ensuring transparency, community-driven development, and enterprise-grade stability.

<https://framework.beeai.dev/introduction/welcome>

Key Features

Feature	Description
Production Optimization	Built-in caching, memory optimization, and resource management for scalable deployment
Agents with Constraints	Preserve your agent's reasoning abilities while enforcing deterministic rules instead of suggesting behavior
Dynamic Workflows	Use simple decorators to design multi-agent systems with advanced patterns like parallelism, retries, and replanning
Declarative Orchestration	Define complex agent systems in YAML for more predictable and maintainable orchestration
Pluggable Observability	Integrate with your existing stack in minutes with native OpenTelemetry support for real-time monitoring, auditing, and detailed tracing
MCP and A2A Native	Build MCP-compatible components, equip agents with MCP tools, and interoperate with any MCP or A2A agent system
Provider Agnostic	Supports 10+ LLM providers including Ollama, Groq, OpenAI, Watsonx.ai, and more with seamless switching
Python and TypeScript Support	Complete feature parity between Python and TypeScript implementations lets teams build with the tools they already know and love

Open infrastructure for deploying agents from any framework:

Agent Stack is open, self-hostable infrastructure for deploying AI agents built with any framework. Hosted by the Linux Foundation and built on the Agent2Agent Protocol (A2A), it gives you everything needed to move agents from local development to shared production environments—without vendor lock-in.

<https://agentstack.beeai.dev/introduction/welcome>

What Agent Stack Provides

Everything you need to deploy and operate agents in production:

- **Self-hostable server** to run your agents
- **Web UI** for testing and sharing deployed agents
- **CLI** for deploying and managing agents
- **Runtime services** your agents can access:
 - **LLM Service** — Switch between 15+ providers (Anthropic, OpenAI, watsonx.ai, Ollama) without code changes
 - **Embeddings & vector search** for RAG and semantic search
 - **File storage** — S3-compatible uploads/downloads
 - **Document text extraction** via **Docling**
 - **External integrations** via MCP protocol (APIs, Slack, Google Drive, etc.) with OAuth
 - **Secrets management** for API keys and credentials
- **SDK** (`agentstack-sdk`) for standardized A2A service requests
- **HELM chart** for Kubernetes deployments with customizable storage, databases, and auth

Build your agent using LangGraph, CrewAI, or your own framework—SDK handles runtime service requests automatically.

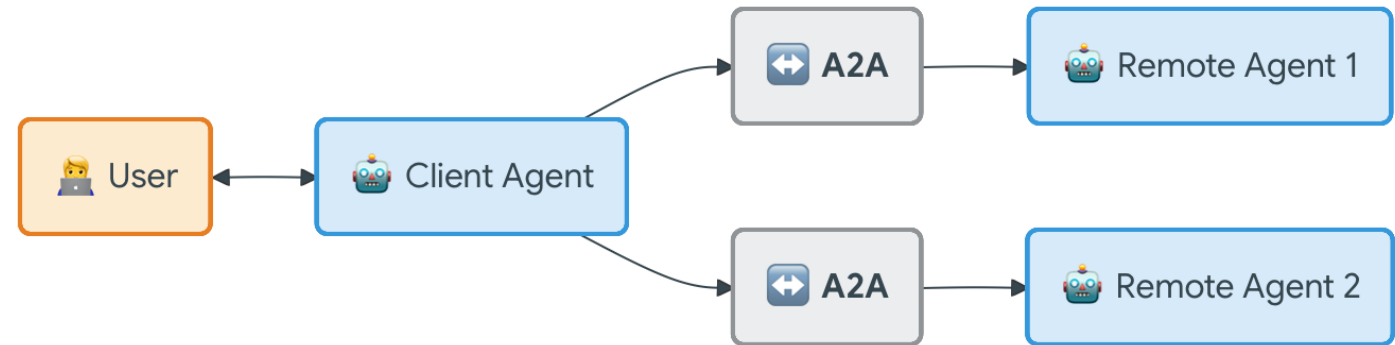
A2A Protocol

Agent2Agent (A2A) Protocol, an open standard designed to enable seamless communication and collaboration between AI agents.

Originally developed by Google and now donated to the Linux Foundation, A2A provides the definitive common language for agent interoperability in a world where agents are built using diverse frameworks and by different vendors.

<https://a2a-protocol.org/latest/>

<https://github.com/a2aproject/A2A>

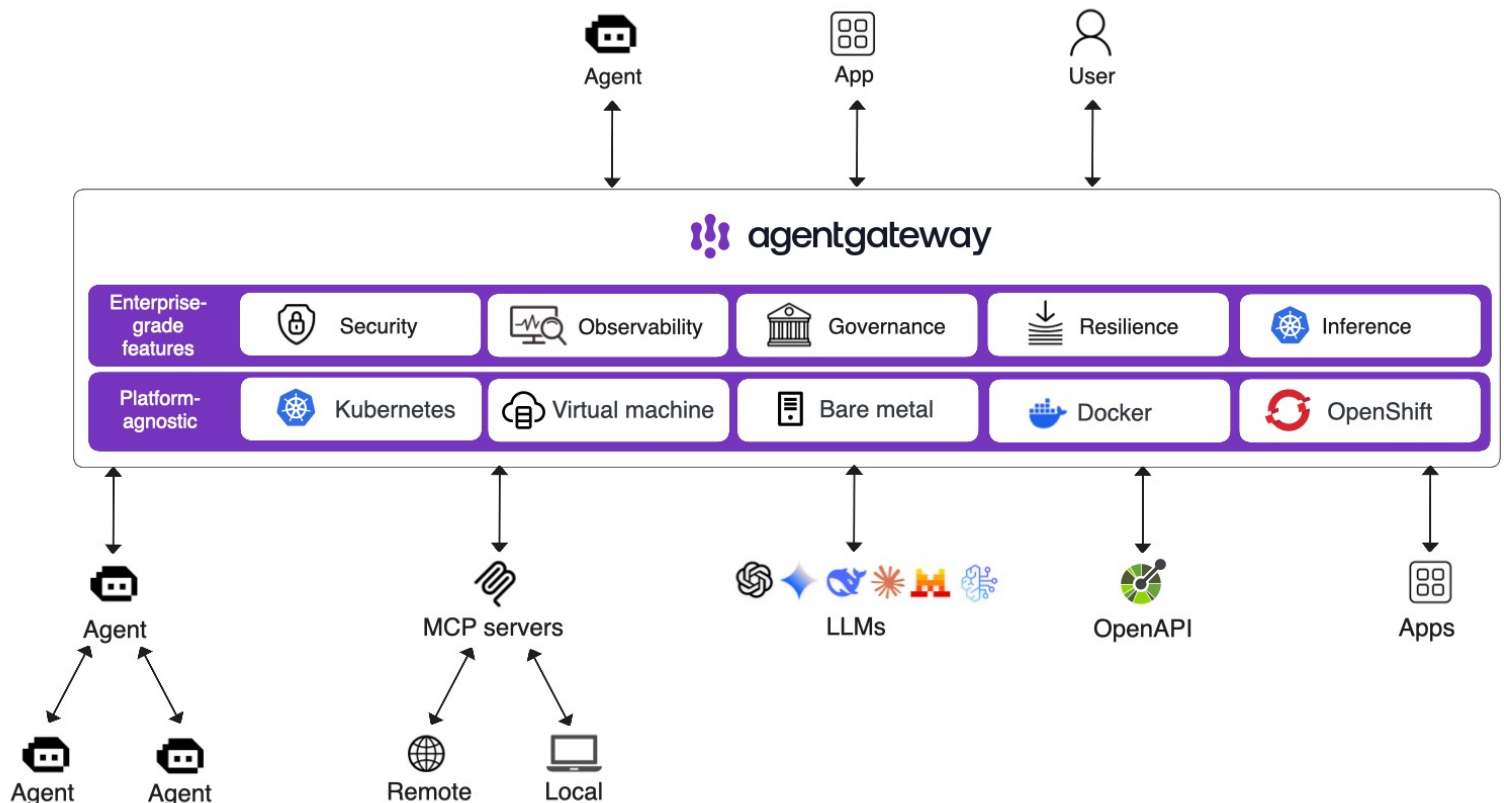


AgentGateway

Agentgateway connects AI agents, MCP tool servers, and LLM providers in any environment. These two-way connections are secure, scalable, and stateful. You get enterprise-grade security, observability, resiliency, reliability, and multi-tenancy features.

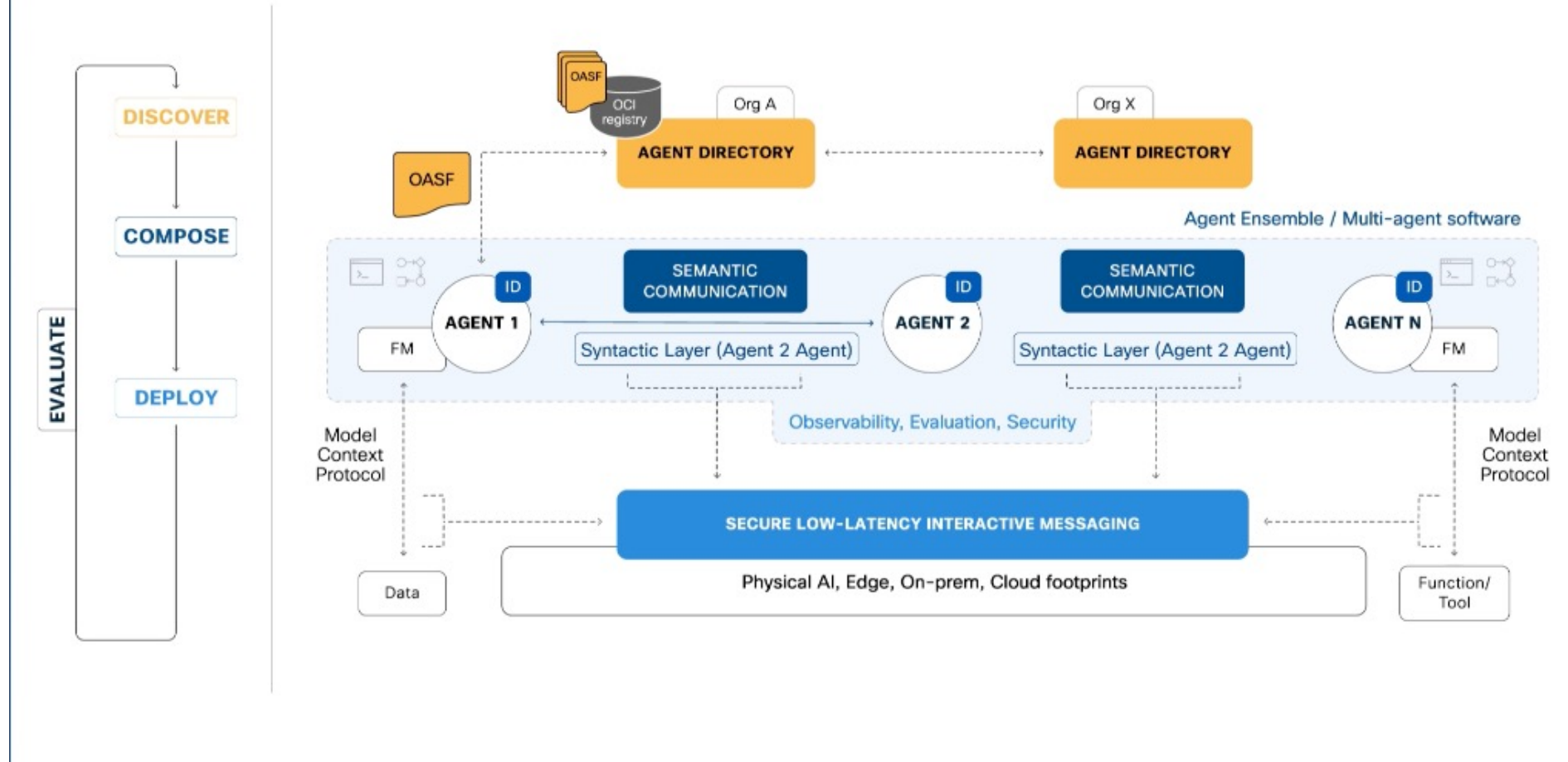
This way, agentgateway addresses common gaps with traditional gateway proxies or protocols like MCP and A2A. With agentgateway, you can quickly adopt and scale your agentic AI environments.

<https://agentgateway.dev>



Internet of Agents

<https://agntcy.org>



Layerd Architecture

Layer 3: IMPLEMENTATION

- Microservices
- API-first
- Cloud-native
- Headless

- Deliver each agent as a Microservice (MS)
- Decouple the LLM and Tools
- Use FastAPI (ASGI and asyncio) as MS framework
- Decouple Frontends: CLI or Steamlit or Chainlit

Layer 2: PROTOCOLS

- Collaboration
- Tooling
- LLM-Zugriff
- Payment
- Data Semantics

- Agent-2-Agent (A2A)
- Model Context Protocol (MCP)
- OpenAI API
- Agent Payment Protocol (A2P, google) and Agent Commerce Protocol (ACP, OpenAI)
- Open Data Model

Layer 1: INFRASTRUCTURE

- OASF
- ADS
- Security
- Messaging
- Observability

Intelligente Informationssysteme

4c – Active Inference

a biological inspiration of agentic behavior

Dominik Neumann



Active Inference is a theory of how living organisms maintain their existence by minimizing surprise via perception and action.



Perception as Inference



The Brain as a Statistical Inference Engine

Unconscious Inference

Helmholtz argued that perception is not a direct reading of sensory input.

Instead, the brain infers the most likely causes of sensory signals based on:

- Noisy, ambiguous sensory data
- Prior experience (learned regularities of the world)

He called this process **unbewusster Schluss** (“unconscious inference”).

Brain as a statistical organ: In today’s language, Helmholtz’s view implies that the brain

- Treats sensory signals as uncertain
- Uses past experience as probabilistic expectations
- Selects the most likely interpretation of the world given the evidence

This is why illusions occur: the brain’s inference is statistically reasonable but sometimes wrong.

The general rule determining the ideas of vision that are formed whenever an impression is made on the eye, with or without the aid of optical instruments, is that *such objects are always imagined as being present in the field of vision as would have to be there in order to produce the same impression on the nervous mechanism, the eyes being used under ordinary normal conditions.* To employ an illustration which has been

Handbuch der physiologischen Optik, Band 3



Helmholtz (1821 - 1894)





Giuseppe Archimboldo
The vegetable Gardener

https://de.wikipedia.org/wiki/Giuseppe_Arcimboldo

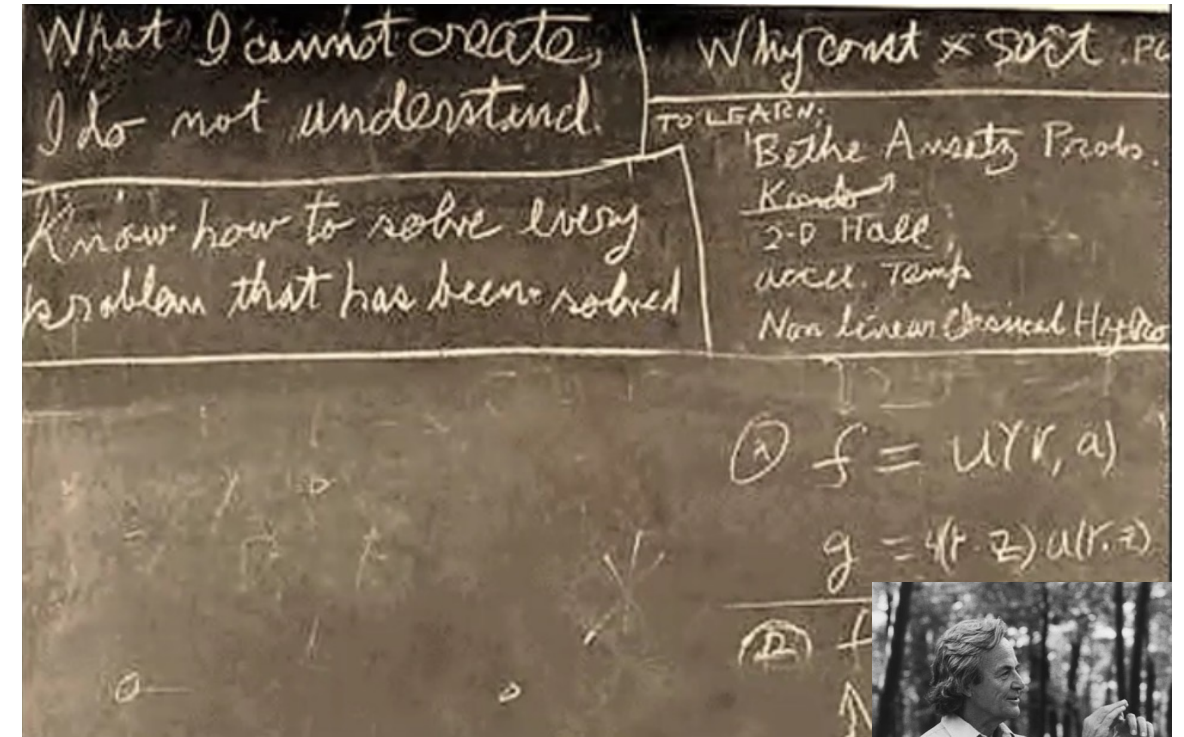
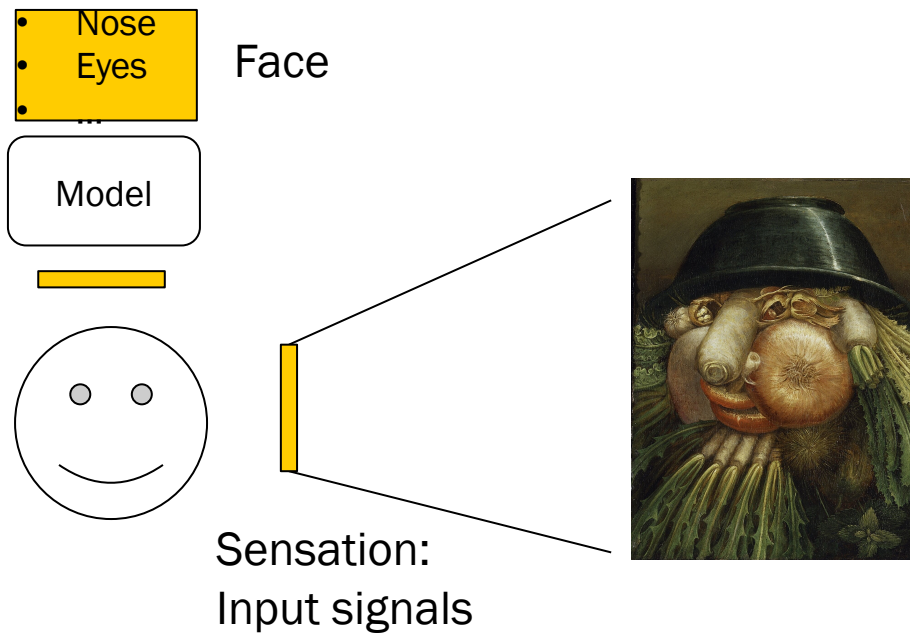


Our percepts are constrained by what we expect to see and the hypotheses that can be called upon to explain our sensory input.

Perception as Inference

What I cannot create on the inside I cannot perceive.

To Understand you need a generative model on the inside that generates the cause of your sensations.



What I cannot create, I do not understand.

Richard Feynman (1918 - 1988)

Predictive Coding Networks



Predictive coding networks (PCNs) constitute a biologically inspired framework for understanding **hierarchical computation in the brain.**

PCN offer an alternative to traditional feedforward neural networks in ML.

Introduction to Predictive Coding Networks for Machine Learning

Mikko Stenlund*

May 29, 2025

Abstract

Predictive coding networks (PCNs) constitute a biologically inspired framework for understanding hierarchical computation in the brain, and offer an alternative to traditional feedforward neural networks in ML. This note serves as a quick, onboarding introduction to PCNs for machine learning practitioners. We cover the foundational network architecture, inference and learning update rules, and algorithmic implementation. A concrete image-classification task (CIFAR-10) is provided as a benchmark-smashing application, together with an accompanying Python notebook containing the PyTorch implementation.

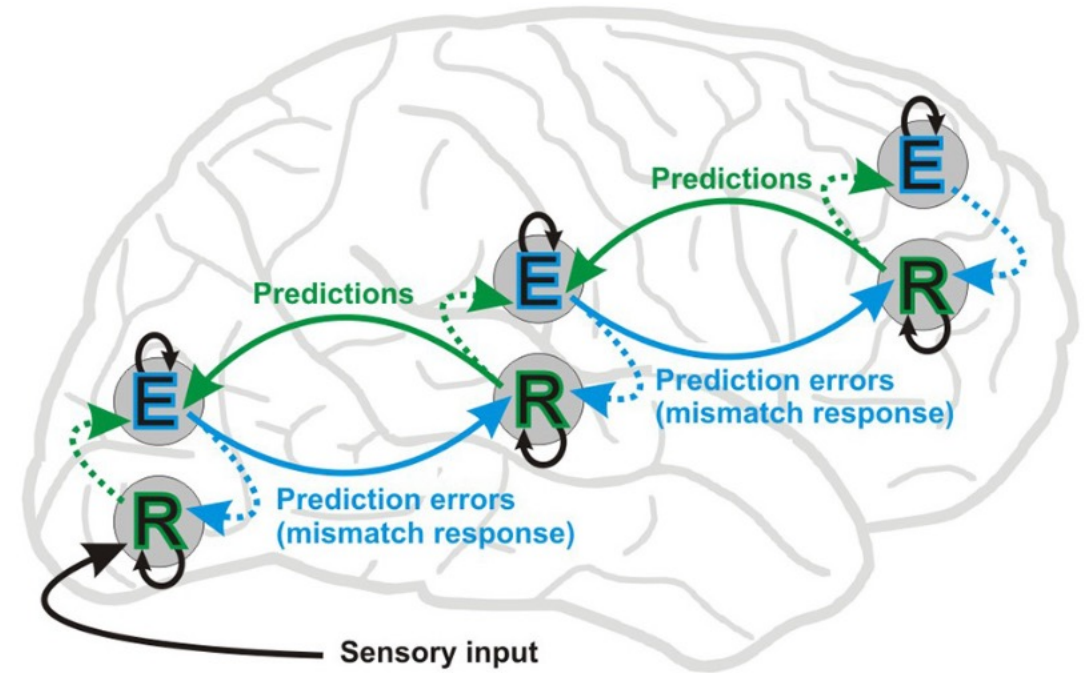
Source: <https://arxiv.org/pdf/2506.06332v1>



Predictive Coding Networks

Predictive coding is a foundational theory in **neuroscience** proposing that **the brain is a prediction machine**:

The Brain is constantly attempting to anticipate incoming sensory inputs and updating **internal representations** to minimize the difference between expectations and actual observations.



Predictive Coding Networks

Deep neural networks learn hierarchical feature representations

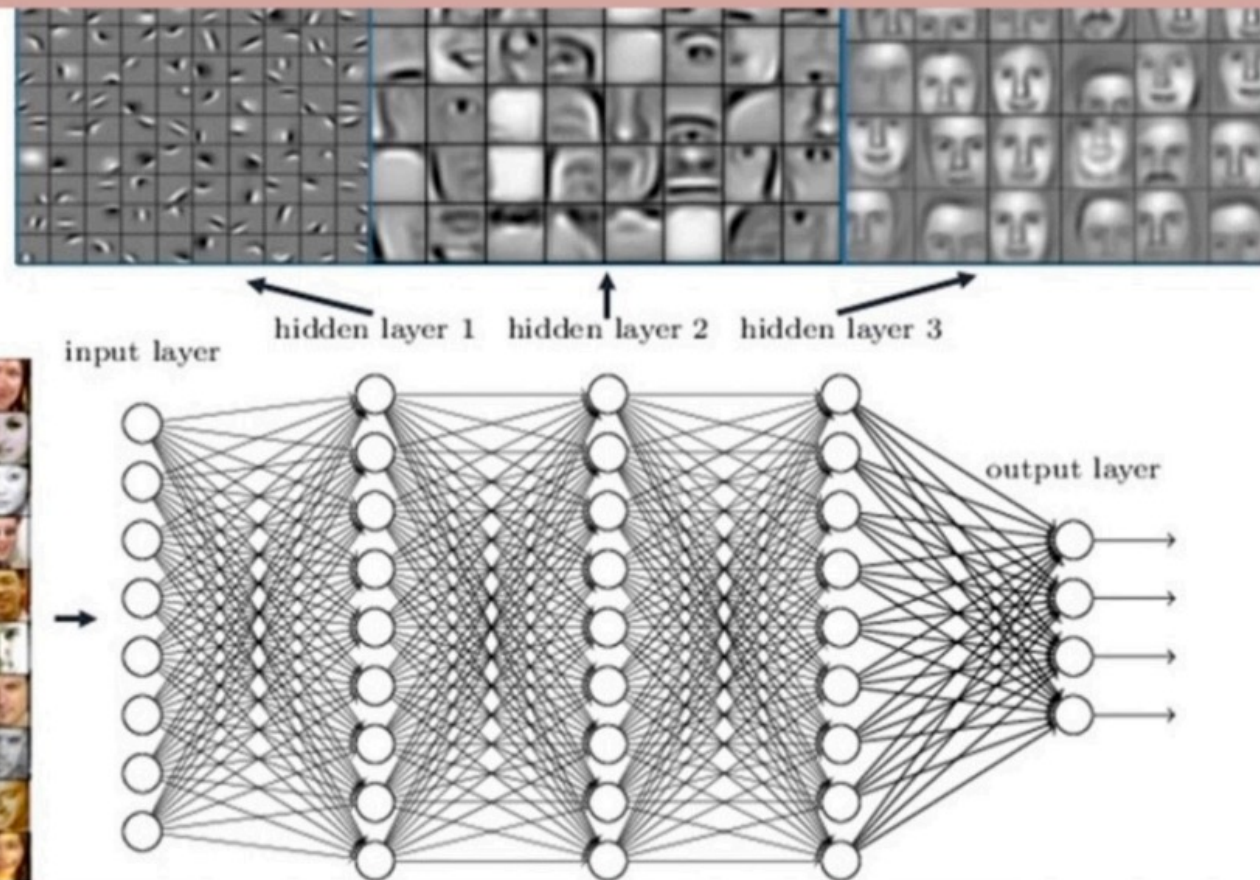
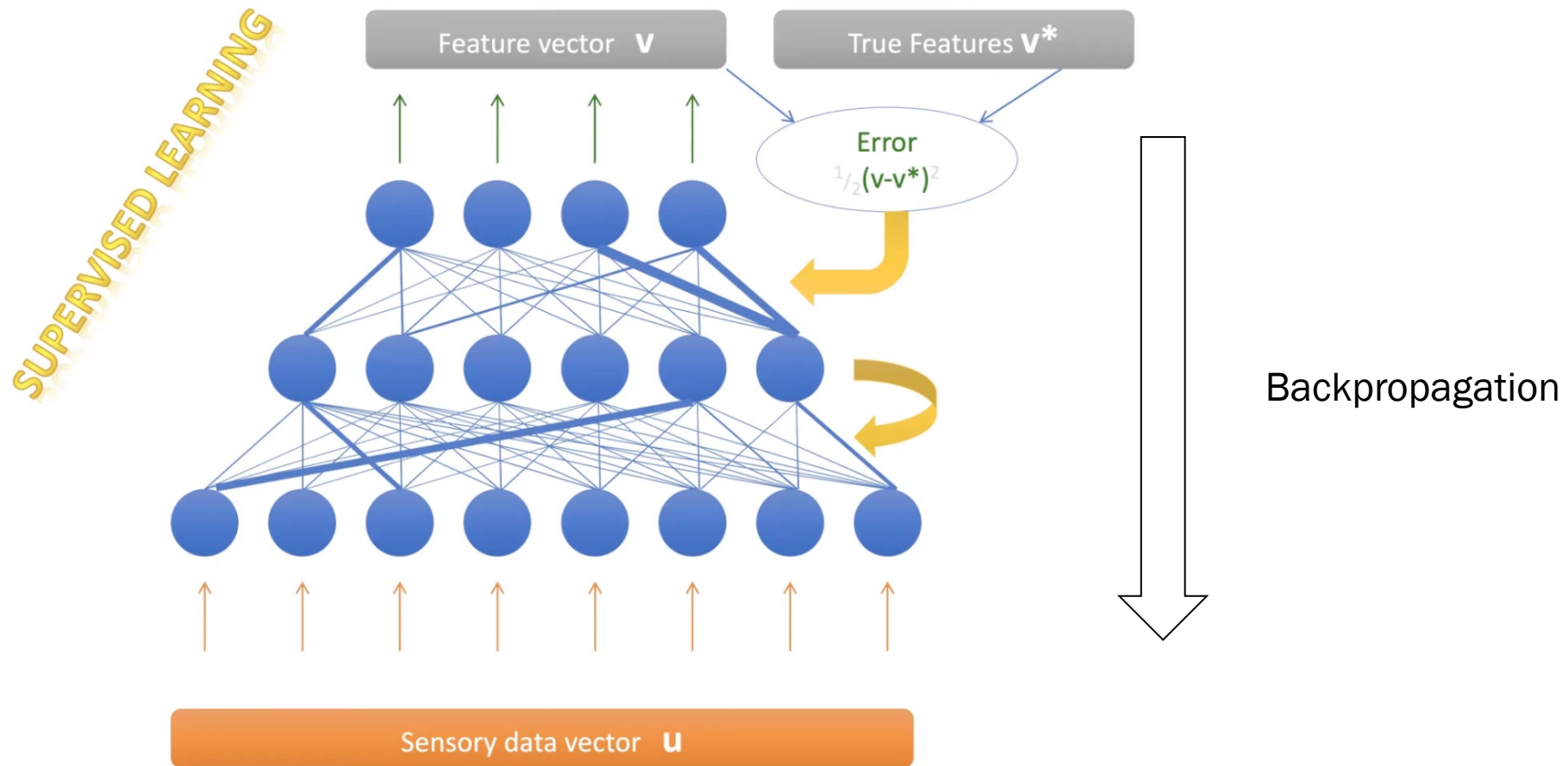


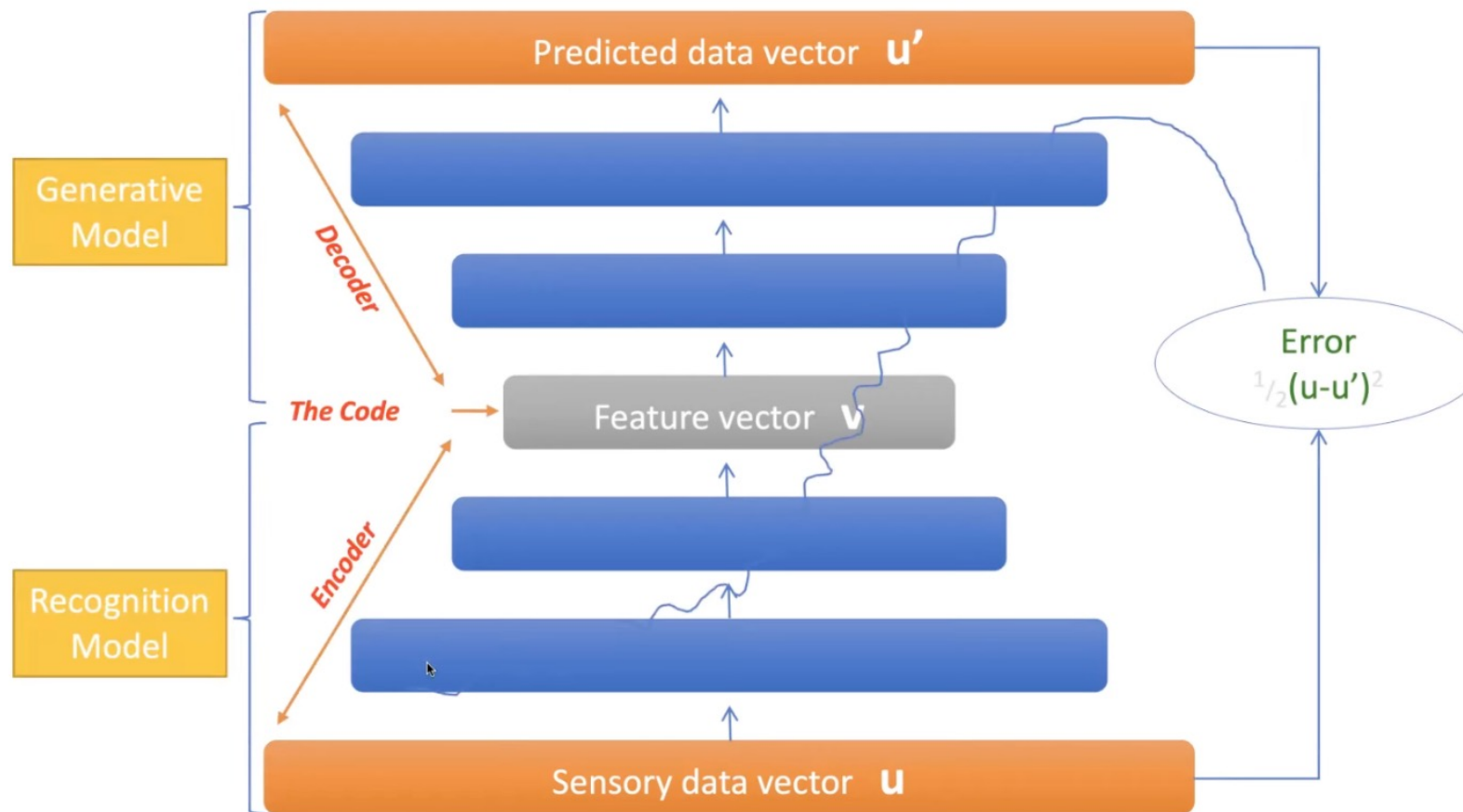
Figure 1. Deep neural networks learn hierarchical feature representations. After (LeCun et al. (2015)) [24].

Predictive Coding Networks

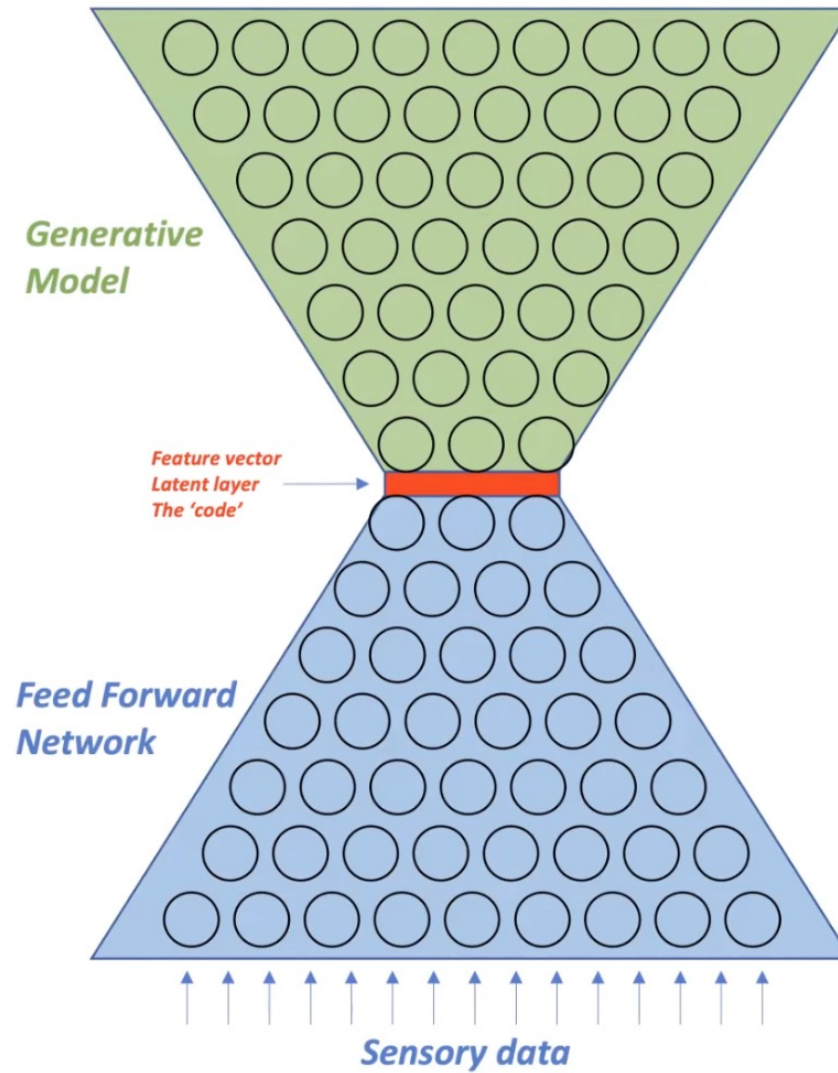


Predictive Coding Networks

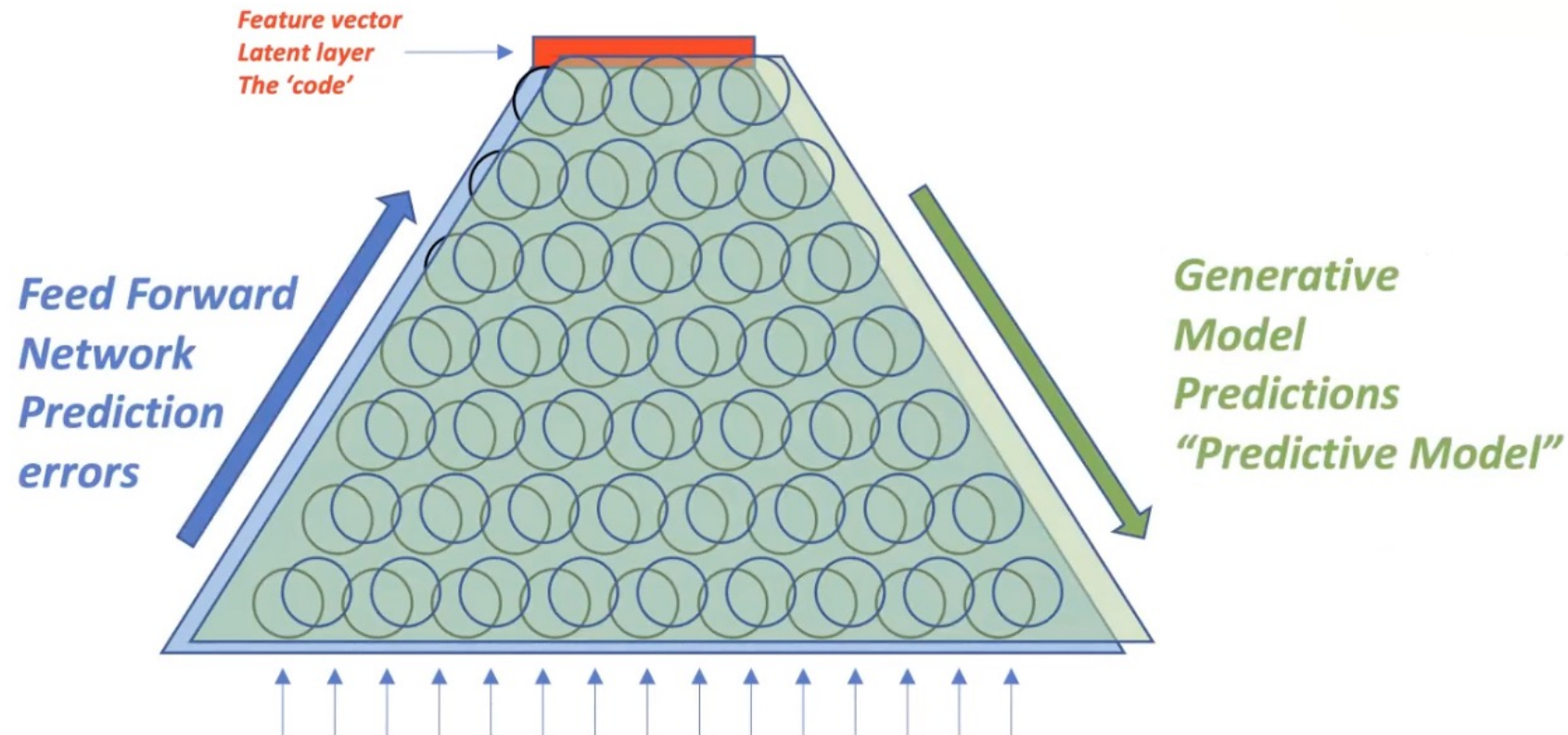
Autoencoder: Self-Supervised Learning



Predictive Coding Networks



Predictive Coding Networks

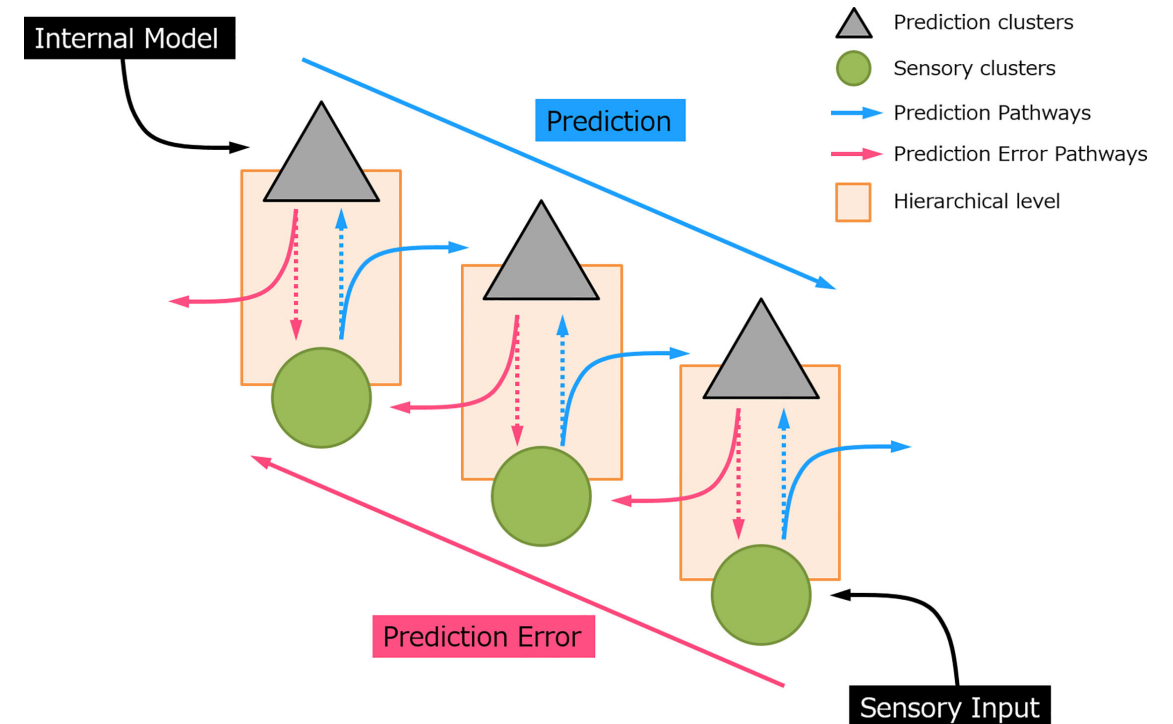


Perception as a bi-directional and hierarchical process:

In sensory processing, sensory “prediction” is actively generated by an **internal model** at the higher-order area which incorporates information from the past (memory), produces and sends information regarding the future to the lower hierarchical cascade.

This inferential process naturally allows the implementation of the Bayesian theories, where prior sensory experiences and current sensory input were used to compute the posterior perceptual estimation, that is, the prediction:

At each processing layer, neurons subtract this prediction from the sensory data and register the residual by a “prediction error” signal. In this sense, it is not necessary for an expected event to transmit to the top of the cortical hierarchy (9). Instead, only the information that deviates from predictions is further processed and passed upward to the higher order cortical areas in the form of “error”. The



Perception as a bi-directional and hierarchical process:

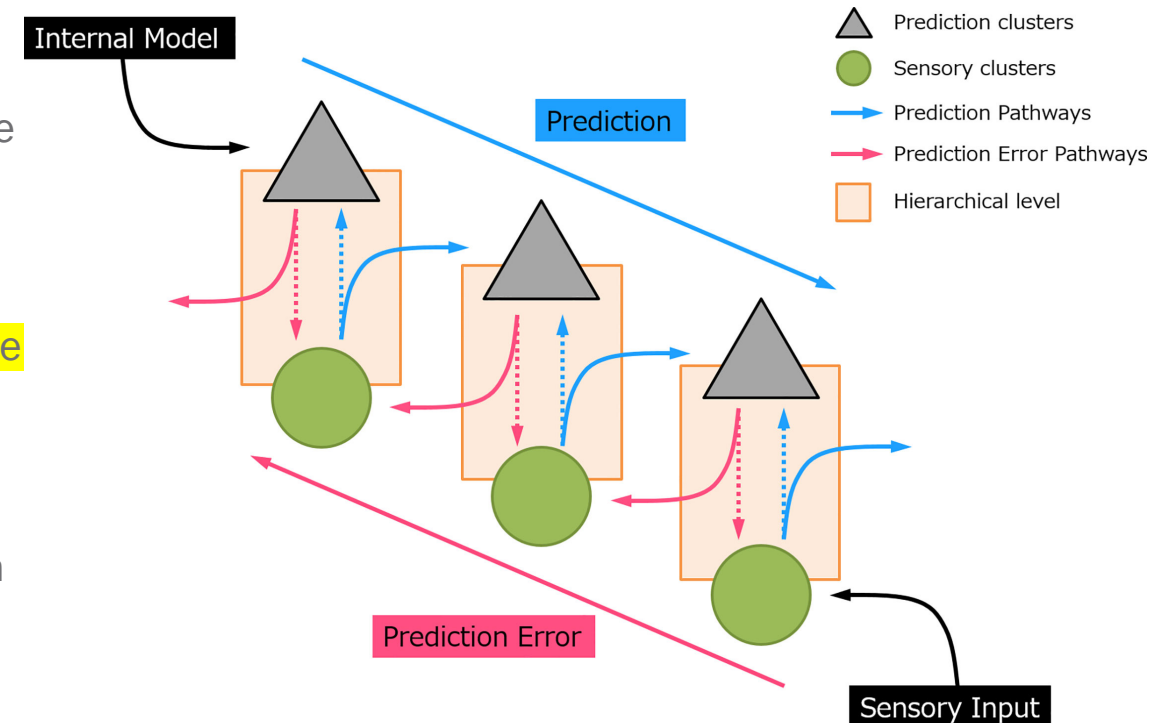
In sensory processing, sensory “prediction” is actively generated by an **internal model** at the higher layer which incorporates information from the past (memory), produces and sends information regarding the future to the lower hierarchical layer.

This inferential process naturally allows the implementation of the Bayesian theories, where **prior sensory experiences and current sensory input** were used to compute the posterior perceptual estimation, that is, the prediction:

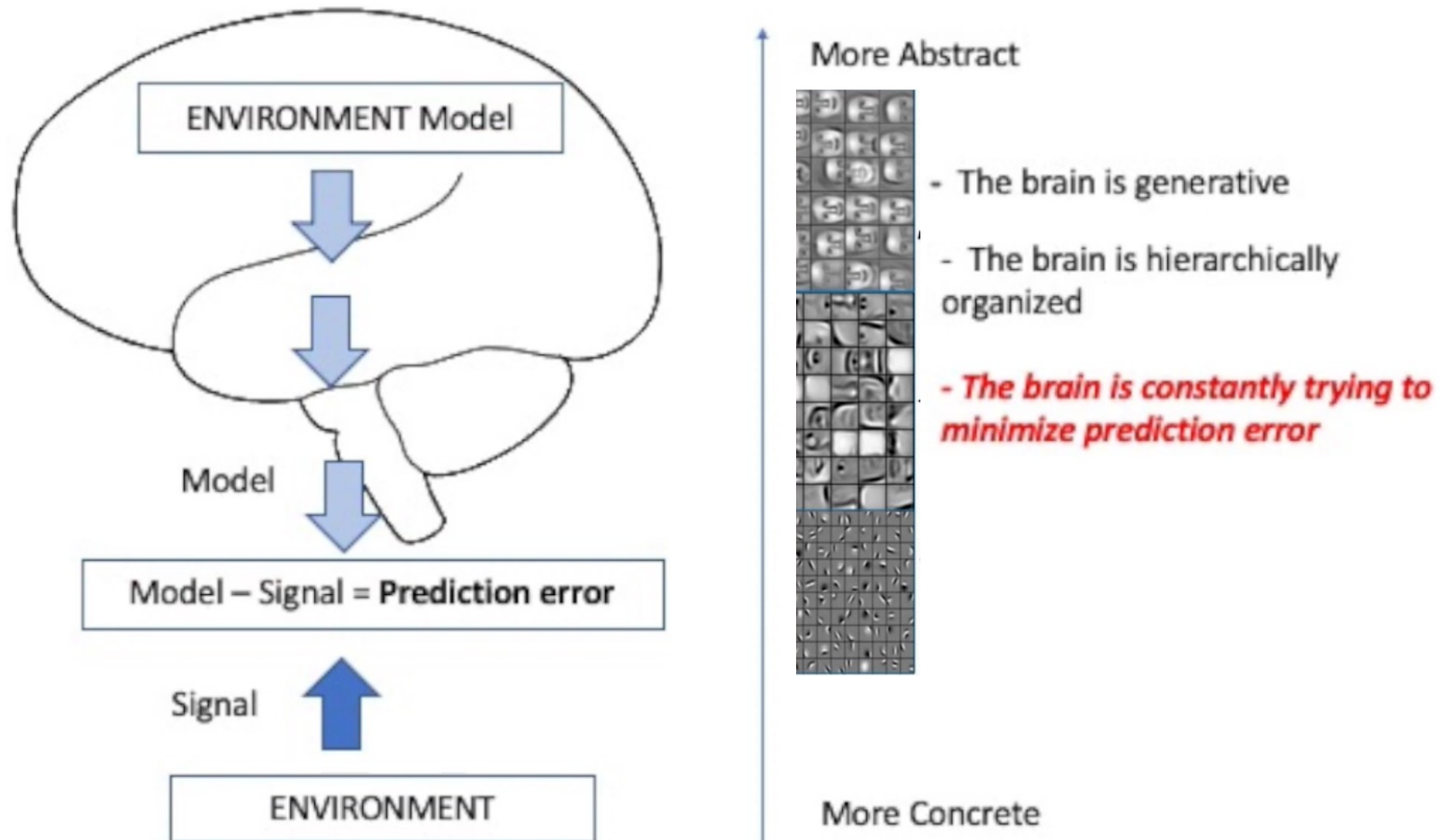
At each processing layer, neurons **subtract this prediction from the sensory data and register the residual by a “prediction error” signal.**

Only the information that deviates from predictions is further processed and passed upward to the higher-order cortical area in the form of “error”.

The greater the prediction error, the larger the neural response evoked. This prediction error signal propagates through an ascending pathway and updates the subsequent prediction.



Predictive Coding Networks



2 Network Architecture

Model. A PCN consists of $L \geq 1$ layers of latent variables $\mathbf{x}^{(l)} \in \mathbb{R}^{d_l}$, $1 \leq l \leq L$, and an input layer of variables $\mathbf{x}^{(0)} \in \mathbb{R}^{d_0}$. Each layer attempts to predict the state of the layer **below**, so the architecture includes the following **top-down** elements for $0 \leq l \leq L - 1$:

- Weights $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ from layer $l + 1$ to layer l

- Preactivations

$$\mathbf{a}^{(l)} = \mathbf{W}^{(l)} \mathbf{x}^{(l+1)} \in \mathbb{R}^{d_l}$$

- Predictions

$$\hat{\mathbf{x}}^{(l)} = f^{(l)}(\mathbf{a}^{(l)}) \in \mathbb{R}^{d_l}$$

where $f^{(l)}$ is, often a nonlinear, scalar function applied elementwise

- Prediction errors

$$\boldsymbol{\epsilon}^{(l)} = \mathbf{x}^{(l)} - \hat{\mathbf{x}}^{(l)} \in \mathbb{R}^{d_l}$$

The loss function subject to minimization is the total square prediction error, or **energy**:

$$\mathcal{L} = \frac{1}{2} \sum_{l=0}^{L-1} \|\boldsymbol{\epsilon}^{(l)}\|^2 .$$



Predictive Coding Networks

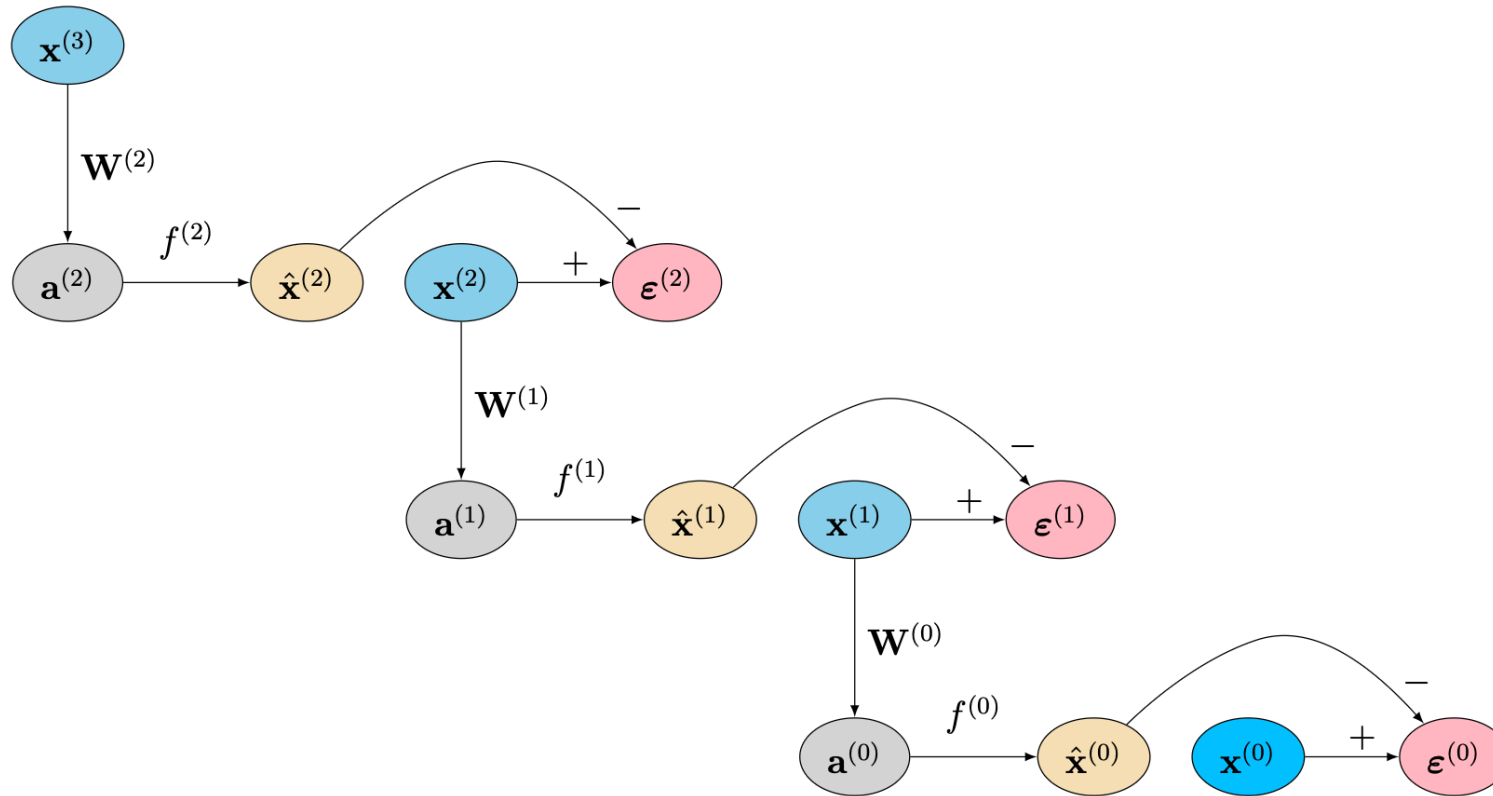


Figure 1: A graph representation of a PCN with three latent layers.

Statistics of Life



An important prerequisite for any **adaptive system** is that it has a certain degree of separation and autonomy from its environment.

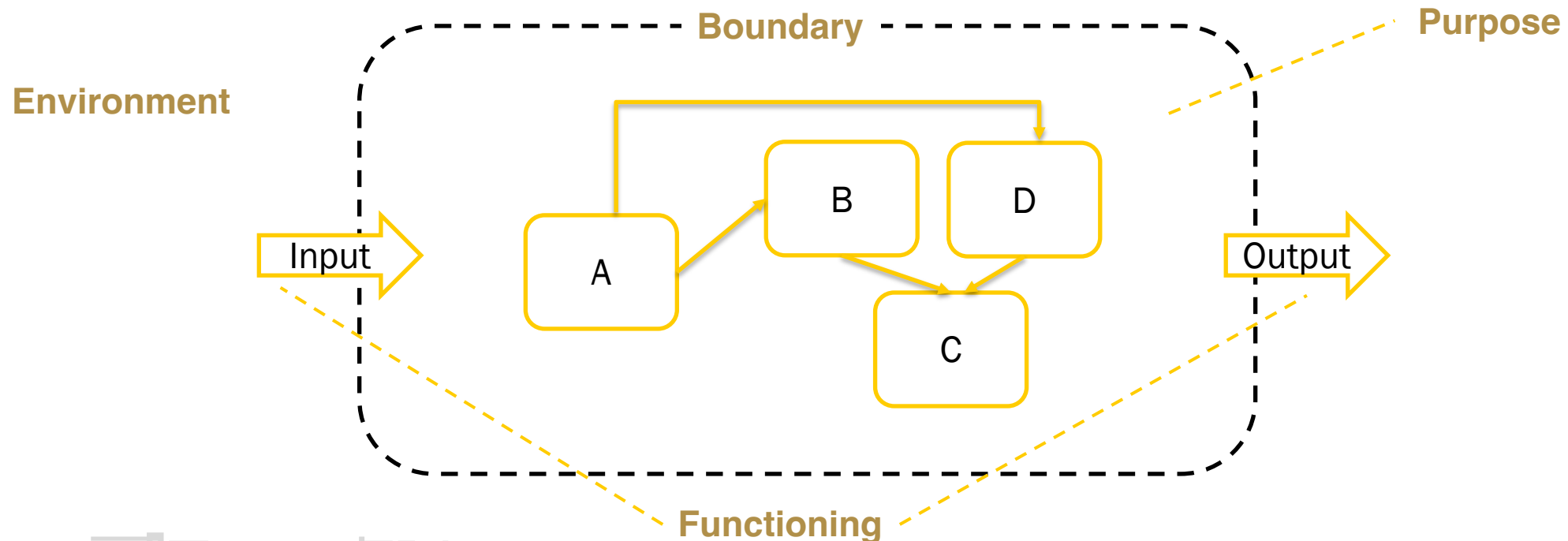


System:

A system is a group of interacting or interrelated elements that act according to a set of rules to form a unified whole.

(<https://www.merriam-webster.com/dictionary/system>)

A system, surrounded and influenced by its environment, is described by its boundaries, structure and purpose and is expressed in its functioning. Systems are the subjects of study of systems theory and other systems sciences.



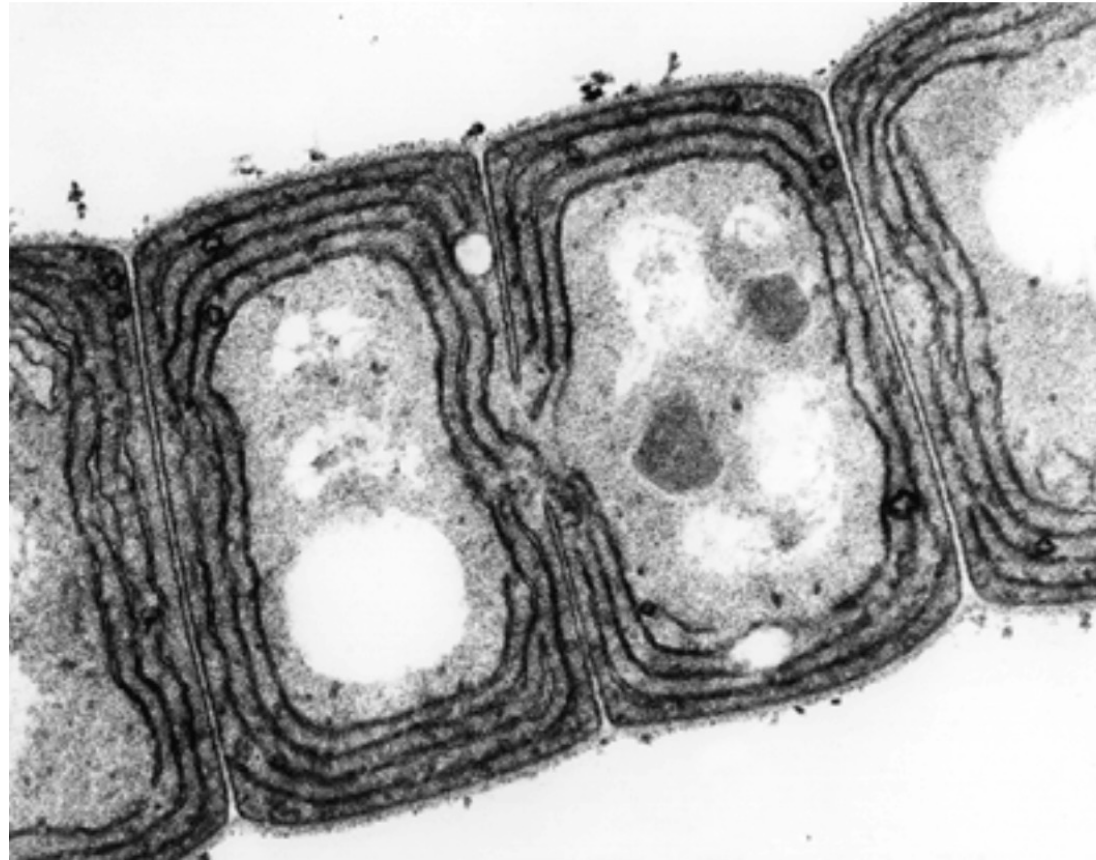
An adaptive system is a set of interacting or interdependent entities, real or abstract, forming an integrated whole that together are able to respond to environmental changes or changes in the interacting parts, in a way analogous to either continuous physiological homeostasis or evolutionary adaptation in biology.

Feedback loops represent a key feature of adaptive systems, such as ecosystems and individual organisms; or in the human world, communities, organizations, and families. Adaptive systems can be organized into a hierarchy.

Source: https://en.wikipedia.org/wiki/Adaptive_system



Cell as an Adaptive System



Zell-Membran als
Boundary.

Sie legt fest, wie Zellen
miteinander kommunizieren.

Eine Zelle existiert, weil ihre Membran definiert, was innen und was außen ist. Aber auch was hinein und hinaus darf.



Markov Blanket

Judea Pearl, Probabilistic Reasoning in Intelligent Systems:
Networks of Plausible Inference, (1988)



Conditional independence:

- describes situations in which an observation is irrelevant or redundant when evaluating the certainty of a hypothesis.
- Let A be a hypothesis, and B and C observations, and $P(C) > 0$:

A and B are said to be conditional independent, if $P(A \mid B, C) = P(A \mid C)$

Since the probability of A given C is the same as the probability of A given both B and C , this equality expresses that B contributes nothing to the certainty of A .

Interpretation: A and B are conditionally independent given C , if and only if,

- given knowledge of whether A occurs,
 - given knowledge of whether C occurs
- } provides no information on the likelihood of B occurring
- and
- given knowledge of whether B occurs,
 - given knowledge of whether V occurs
- } provides no information on the likelihood of A occurring



Markov Blanket as a Statistical Boundary

Markov Blanket:

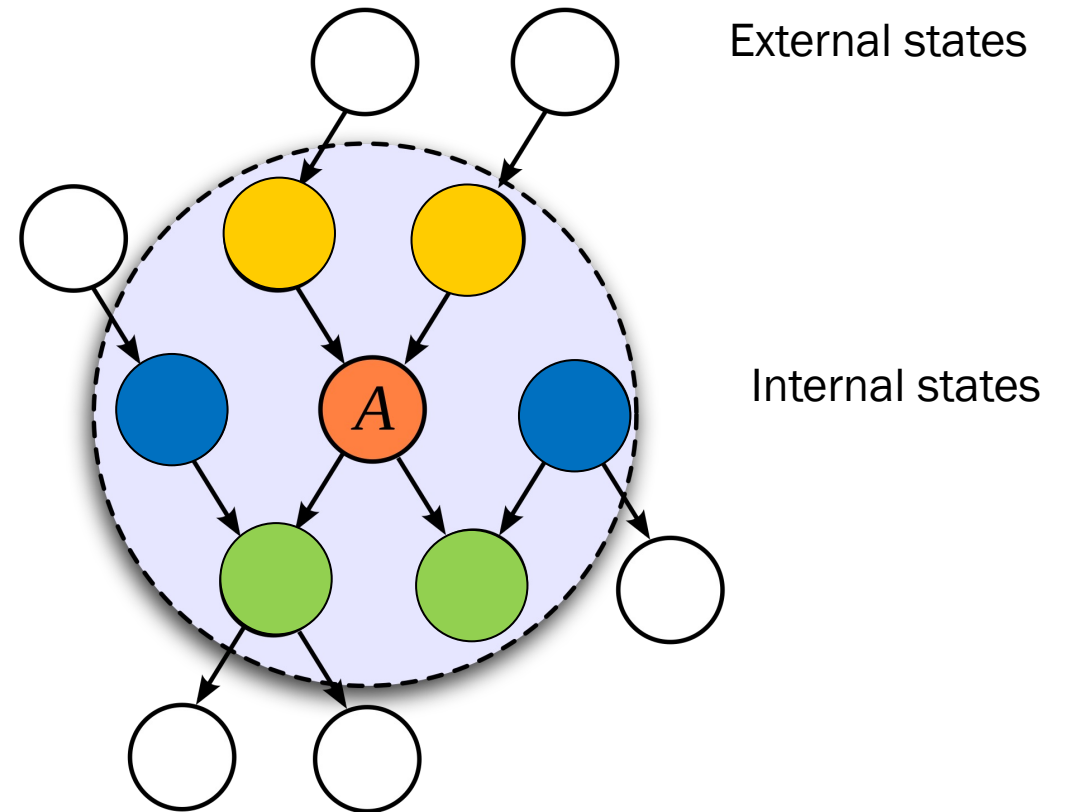
Markov blanket of a random variable is a set of random variables that renders the variable conditionally independent of all other random variables in the system.

In a Bayesian network, the Markov blanket of a node consists of ist

- **parents**,
- its **children**, and
- its **children's other parents**

Knowing the values of these nodes makes the target node conditionally independent of the rest of the network.

Identifying a Markov blanket allows for efficient inference and helps isolate relevant variables for prediction or causal reasoning.



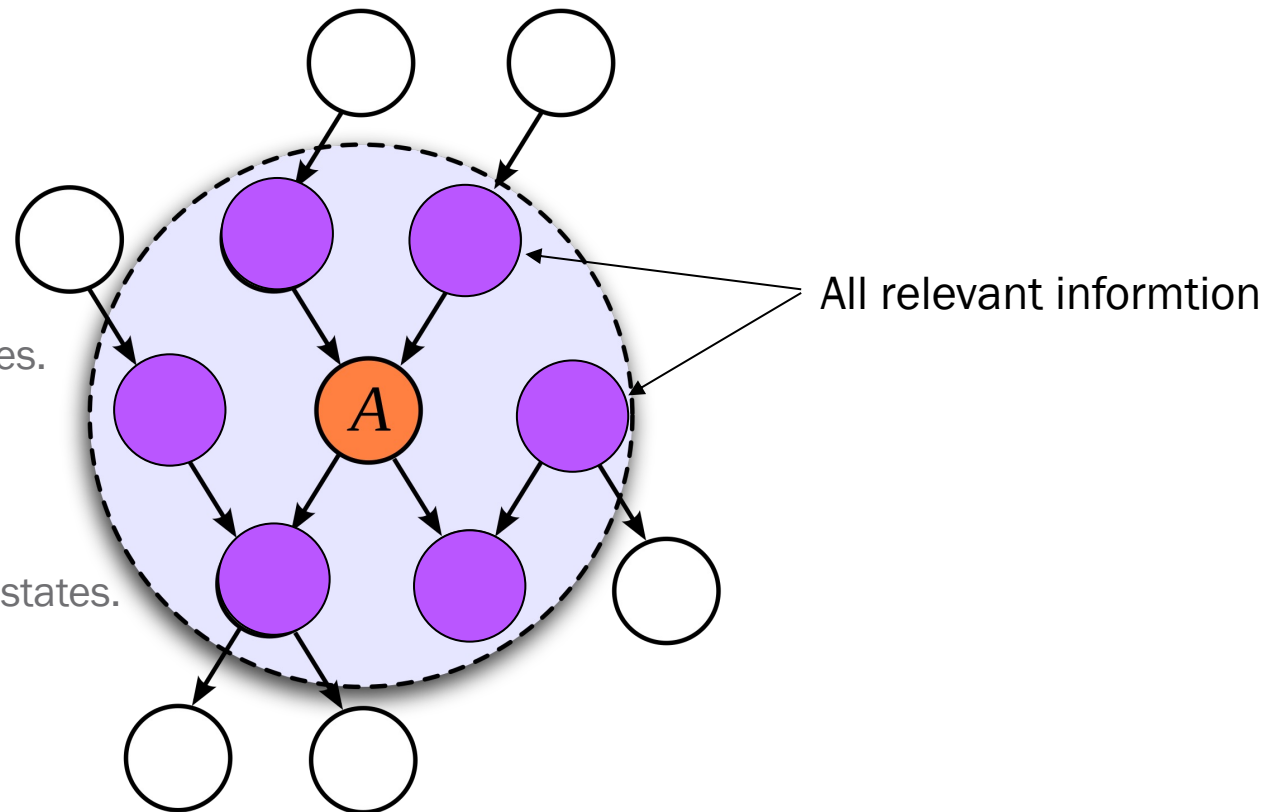
Markov Blanket as a Statistical Boundary

The **blanket states** enables me to separate the internal states from the external states of the universe.

If I want to know how my internal states were going to evolve over time given the whole universe?

I would only need to know my blanket states.

All the information in the entire universe necessary to predict me or describe my internal states is contained in the blanket states.



The probability of my internal states is conditionally independent of the external states given the blanket states.

A Markov blanket is an important recurring concept in this book (Friston 2019a, Kirchhoff et al. 2018, Palacios et al. 2020). Technically, a blanket (b) is defined as follows:

$$\mu \perp x | b \Leftrightarrow p(\mu, x | b) = p(\mu | b) p(x | b)$$

This says (in two different but equivalent ways) that a variable μ is conditionally independent of a variable x if b is known. In other words, if we know b , knowing x would give us no additional information about μ . A common example of this is a Markov chain, where the past causes the present causes the future. In this scenario, the past may only influence the future via the present. This means no additional information about the future is gained by finding out about the past (assuming we know the present).

To identify a Markov blanket in a system wherein we know the conditional dependencies, we can follow a simple rule. The blanket for a given variable comprises its *parents* (the variables it depends on), its *children* (the variables that depend on it) and, in some settings, the other *parents of its children*.

Source: Active Inference



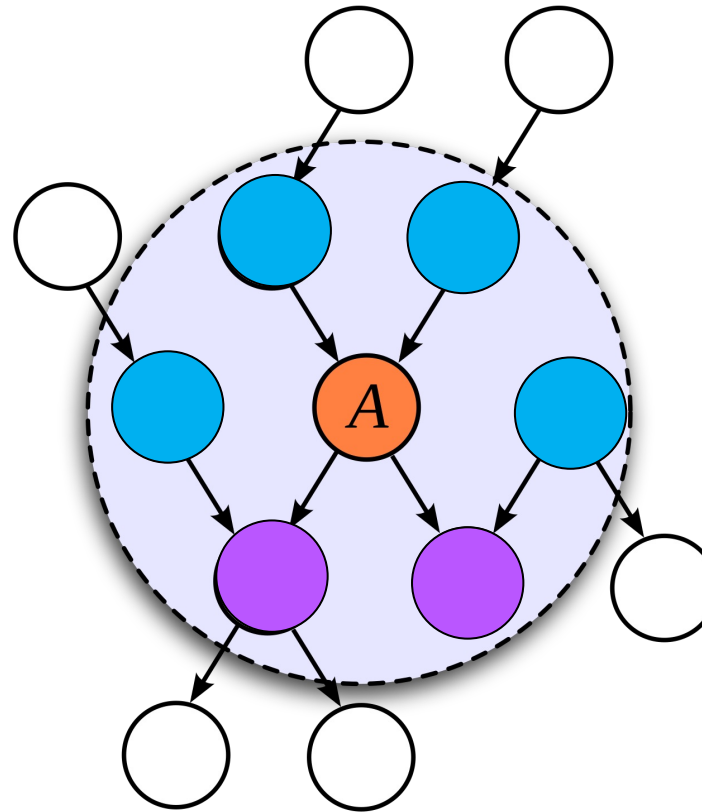
Markov Blanket as a Statistical Boundary

We can partition the blanket states into

- Sensory States
- Active States

Sensory States influence and are not influenced by internal states.

Active States are influenced by internal states, but do not influence internal states.



External states

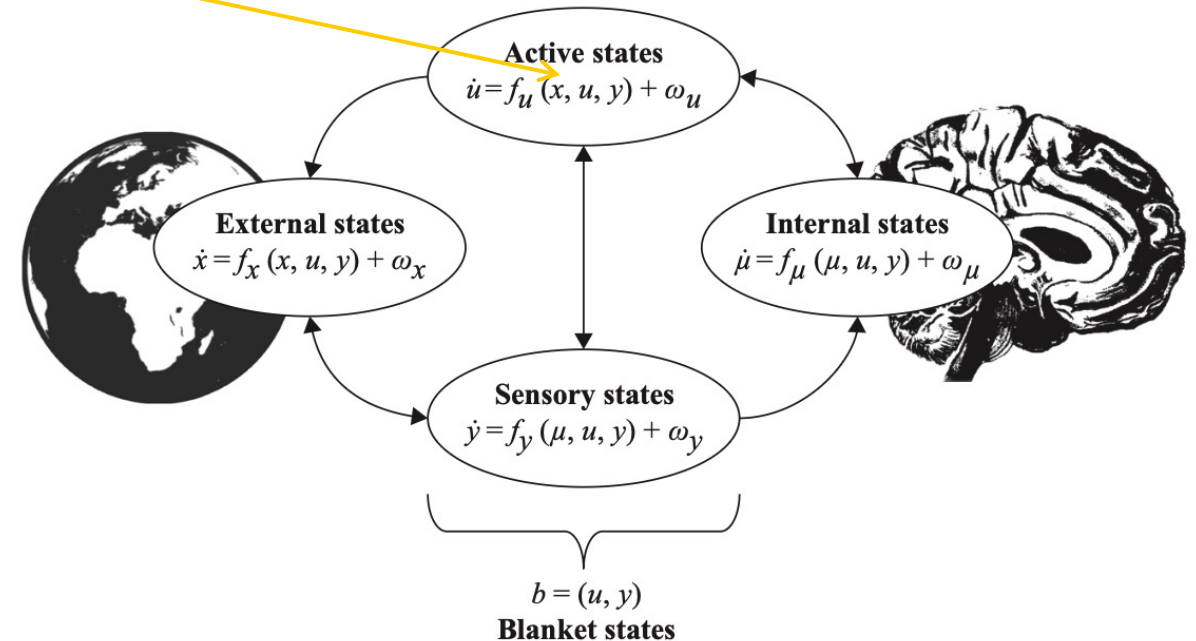
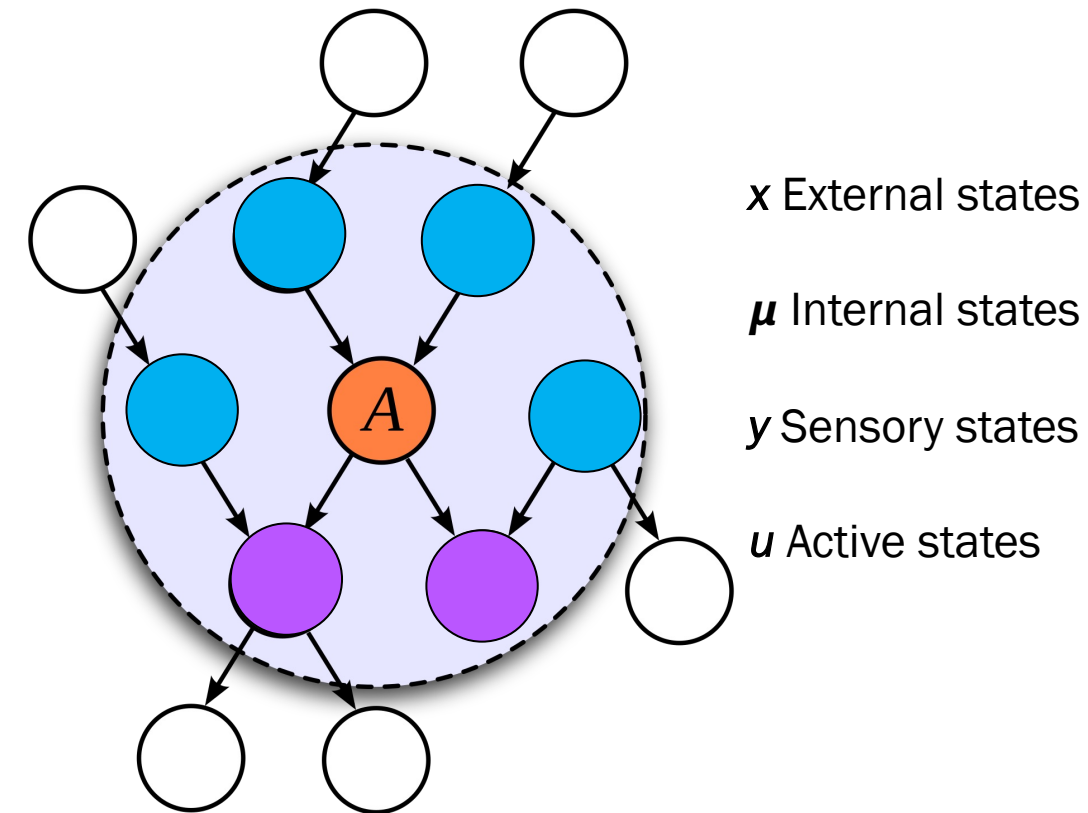
Internal States

Sensory States

Active States

Markov Blanket as a Statistical Boundary

Internal and active States are not functions of the external states!
Is x right here ?



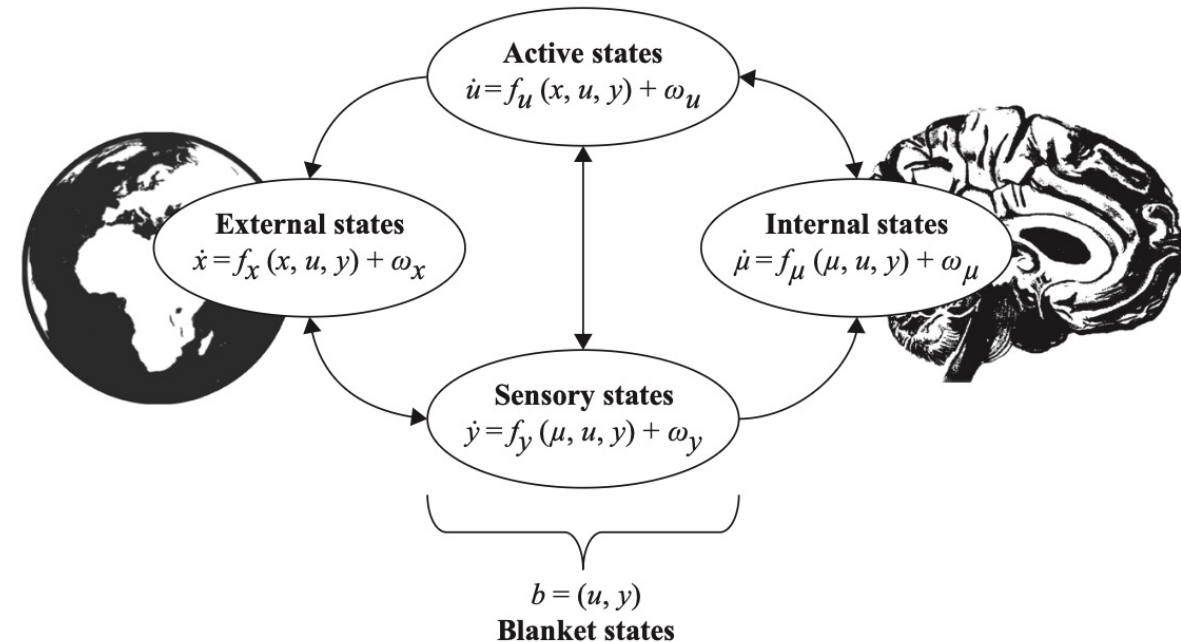
Source: Active Inference

Markov Blanket as a Statistical Boundary

A Markov blanket is the set of variables that mediate all (statistical) interactions between a system and its environment:

- if we knew about the active and sensory states, the external states would offer no additional information about internal states (and vice versa).
- internal states cannot directly change external states but can do so indirectly by changing active states.
- external states cannot directly change internal states but can do so indirectly by changing sensory states.

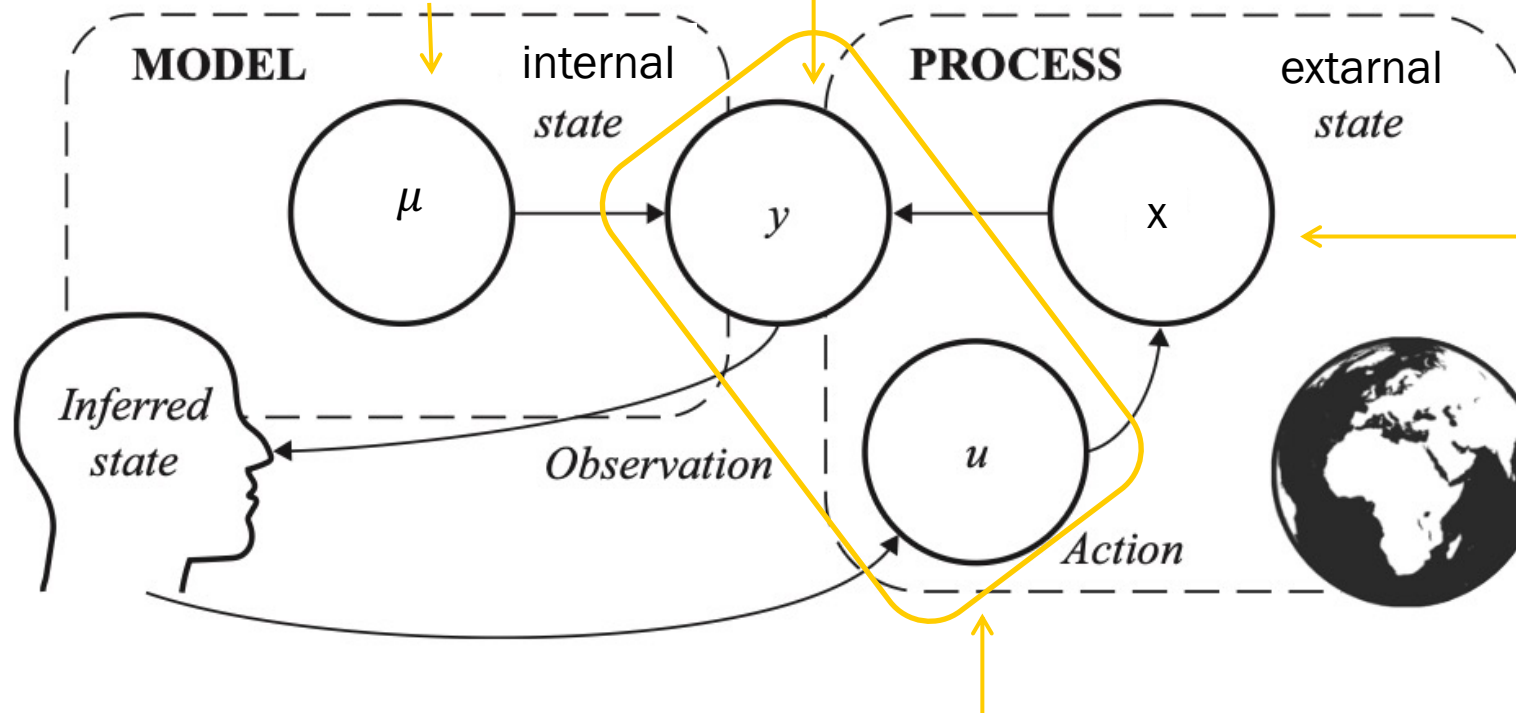
A dynamic Markov blanket, which separates an adaptive system (here, the brain) from the environment. The dynamics of each set of states are determined by a deterministic flow specified as a function (f) giving the average rate of change and additional stochastic (random) fluctuations (ω). The arrows indicate the direction of influence of each variable over the rates of change of other variables



Action - Perception Cycle

Sensory States y cause model to predict and update internal States μ .

External States x cause change in the Sensory States y .



Actuators change the states x of the external world.

Internal States μ influence Active States u .

The Math behind Active Inference

1. Grundidee von Bayesian Inference

Bayes' Theorem:

$$P(H \mid D) = \frac{P(D \mid H) \cdot P(H)}{P(D)}$$

- H = Hypothese (was wir wissen wollen)
- D = Daten (Beobachtungen)
- $P(H)$ = **Prior** (vorherige Annahme über H)
- $P(D \mid H)$ = **Likelihood** (Wahrscheinlichkeit der Daten, wenn H wahr ist)
- $P(H \mid D)$ = **Posterior** (aktualisierte Wahrscheinlichkeit für H nach Beobachtung von D)
- $P(D)$ = Normierungsfaktor (Gesamtwahrscheinlichkeit der Daten)

Kernaussage:

Bayesian Inference aktualisiert unseren Glauben über eine Hypothese, sobald neue Daten verfügbar sind.



Starts from the Helmholtz view of Perception: „perception as unconscious inference“

→ Bayesian brain hypothesis

treating not just perception but also action, planning, and learning as problems of (Bayesian) inference and by deriving a principled (variational) approximation to such otherwise intractable problems.



Bayesian Brain Hypothesis



The “**Bayesian brain**” hypothesis is a theoretical framework in neuroscience and cognitive science proposing that the brain represents information probabilistically and performs **Bayesian inference** to perceive the world, make decisions, and guide action. In **Kenji Doya’s 2007 paper**, the idea is refined by linking *different neuromodulatory systems* to distinct components of Bayesian inference and learning.

https://inc2.dec.ens.fr/sites/cognition.ens.fr/files/2018-02/Doya_2008.pdf

<https://direct.mit.edu/books/edited-volume/2884/Bayesian-BrainProbabilistic-Approaches-to-Neural>

<https://www.nature.com/articles/nn2077>

The Doya Bayesian Brain Hypothesis, associated with Katsunari Doya, posits the brain operates as a Bayesian inference machine, constantly updating internal probabilistic models (priors) with sensory evidence (likelihood) to predict and act in the world, minimizing prediction errors. It integrates **Bayes' theorem** into neuroscience, viewing perception, decision-making, and motor control as statistical inference processes, where the brain generates hypotheses about causes of sensory inputs and acts to fulfill preferences, unifying cognitive functions under a single framework.

https://en.wikipedia.org/wiki/Bayesian_approaches_to_brain_function



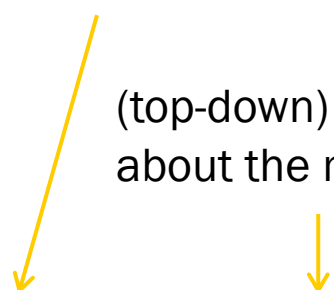
The Bayesian Brain:

- Represents beliefs as probability distributions, (not single-point estimates)
- Combines prior beliefs with sensory evidence according to Bayes' rule
- Continuously updates beliefs as new data arrive
- Uses uncertainty explicitly to guide learning, decision-making, and control

In formal terms:

Sensory Stimmuli

(top-down) prior information about the most likely causes

$$P(\text{state} \mid \text{data}) \propto P(\text{data} \mid \text{state}) \times P(\text{state})$$


The brain is not assumed to compute equations explicitly, but to implement *approximations* to Bayesian inference through neural dynamics and plasticity.



Doya's key contribution is to **map Bayesian learning variables onto neuromodulatory systems**, arguing that different brain chemicals signal different types of **uncertainty** or learning signals.

His central question

How does the brain control learning and inference under uncertainty?

Doya proposes that neuromodulators regulate **when, what, and how fast** the brain learns, in a way consistent with Bayesian principles.

Expected vs. unexpected uncertainty (A key conceptual distinction in Doya):

Type of uncertainty	Meaning	Neuromodulator
Expected uncertainty	Noise you know is there	Acetylcholine
Unexpected uncertainty	Sudden change in th world	Norepinephrine

This distinction is crucial in Bayesian learning:

- Expected uncertainty → **fine-tuning**
- Unexpected uncertainty → **model revision**



Bayesian Inference and Surprise



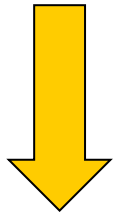
Bayesian Inference and Bayes Rules

A **generative model** is a construct from statistical theory that generates predictions about observations.

It may be formulated as the joint probability $P(x, y)$ of observations y and the world's hidden states x that generate these observations. $P(x, y) = P(x|y)P(y) = P(y|x) P(x)$

Likelihood: knowledge of how observations are generated from states

Prior: knowledge about the hidden states x of the world prior to seeing sensory data y .


$$P(x|y) = \frac{P(y|x) P(x)}{P(y)}$$

Posterior: probability of hidden states after receiving observations

Marginal likelihood
Normalization: makes the computation intractable

It is computed by marginalization over the unknown X :

$$\begin{aligned} p(Y=y) &= \text{SUM}_{\{x' \in X\}} p(X=x')p(Y=y | X=x') \\ &= \text{SUM}_{\{x' \in X\}} p(X=x', Y=y) \end{aligned}$$

Bayesian Inference and Bayes Rules

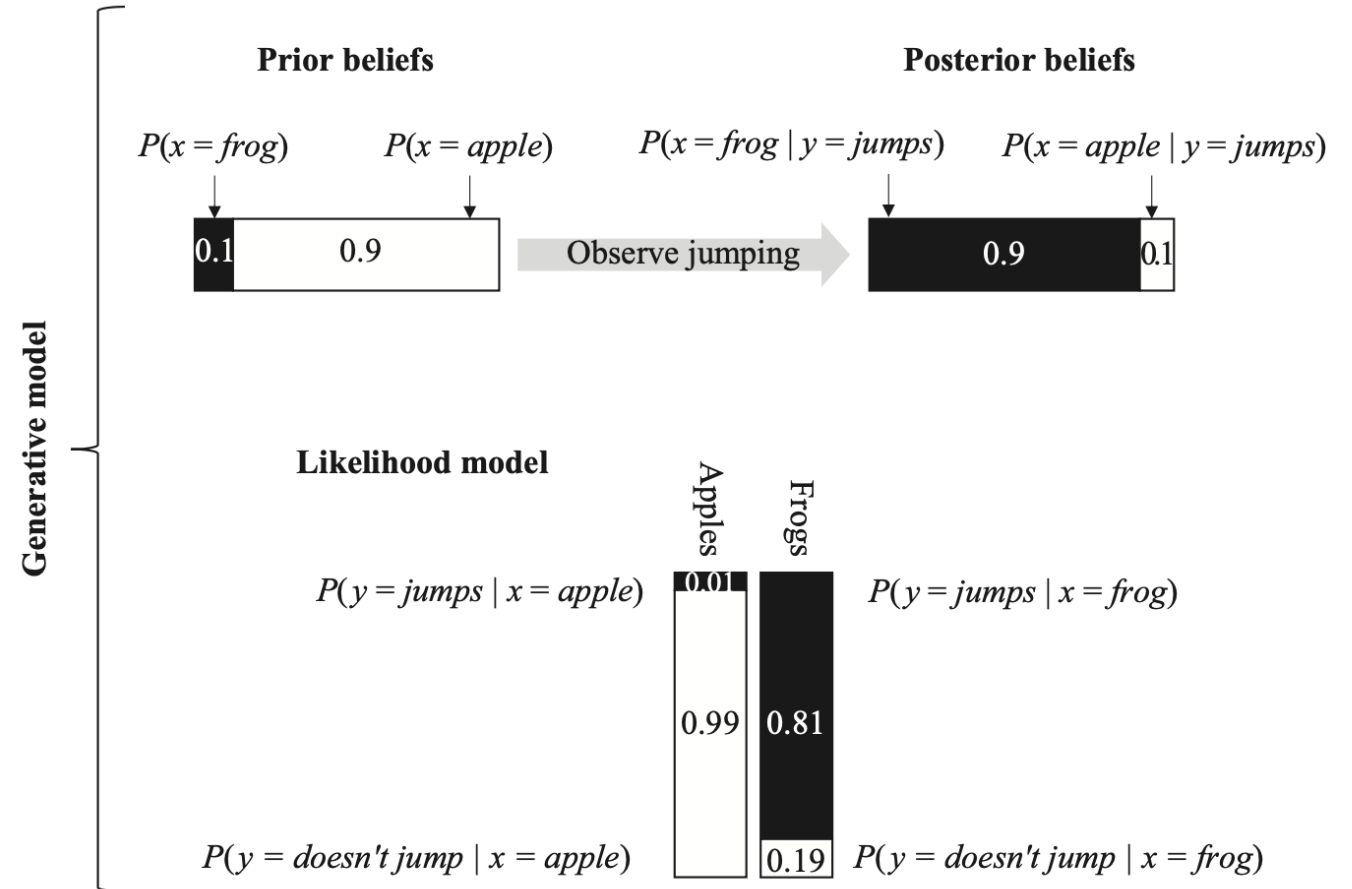
Example:

A Person will be confronted with an apple and a frog.

The person has prior knowledge and knows there are more apples than frogs:

$$\begin{aligned} P(x=\text{apple}) &= 0,9 \\ P(x=\text{frog}) &= 0,1 \end{aligned}$$

And knows that frogs jump and apples not.



Surprise

Surprise as the negative log evidence, where evidence is marginal probability of observations.

In our Example: - $\ln P(\text{„something is jumping“})$

$$\begin{aligned} P(\text{„something is jumping“}) &= \text{SUM}_{\{x = \text{apple, frog}\}} P(y=\text{„jumps“}, x) \\ &= P(y=\text{„jumps“}, x=\text{apple}) + P(y=\text{„jumps“}, x=\text{frog}) \\ &= P(y=\text{„jumps“} \mid x=\text{apple}) * P(x=\text{apple}) + P(y=\text{„jumps“} \mid x=\text{frog}) * P(x=\text{frog}) \\ &= 0.01 * 0.9 + 0.81 * 0.1 \\ &= 0.09 \end{aligned}$$

Under our model, we would only expect to observe „something is jumping“ about 9 times out of 100 observations. As such, we should be surprised to observe this if we subscribed to the model

→ Surprise as a measure of how poorly a model fits the data it tries to explain: $-\ln(0.09) = 2.4 \text{ nats}^1$

1 Like bits, nats are units of information. The choice of unit depends on whether we use a logarithm to the base 2 (bits) or a natural logarithm (nats).



Bayesian surprise is a measure of how much we have to update our beliefs following an observation.

→ Bayesian surprise quantifies the difference between a **prior** and a **posterior probability**.

This raises the question of how we quantify the dissimilarity of two probability distributions.

One answer, from information theory, is to use a Kullback-Leibler (KL) Divergence, defined as the **average difference between two log probabilities**:

$$D_{\text{KL}}(Q \parallel P) = \mathbb{E}_{x \sim Q} [\ln Q(x) - \ln P(x)]$$

$$D_{\text{KL}}(Q \parallel P) = \int q(x) (\ln q(x) - \ln p(x)) dx$$

$$D_{\text{KL}}(Q \parallel P) = \int q(x) \ln \frac{q(x)}{p(x)} dx$$

Bayesian Surprise

Example

$$D_{KL}(p(x|y) || p(x))$$

$$\begin{aligned} D_{KL}(p(x|y) || p(x)) &= p(x = \text{frog} | y = \text{jumps}) * [\ln p(x = \text{frog} | y = \text{jumps}) - \ln P(x = \text{frog})] \\ &\quad + p(x = \text{apple} | y = \text{jumps}) * [\ln p(x = \text{apple} | y = \text{jumps}) - \ln P(x = \text{apple})] \\ &= 0.9 [\ln(0.9) - \ln(0.1)] + 0.1 [\ln(0.1) - \ln(0.9)] \approx 1.8 \text{ nat} \end{aligned}$$

→ This scores the amount of belief updating, as opposed to simply how unlikely the observation was.



Surprise as a measure of how poorly a model fits the data it tries to explain:

1. Surprise as the negative log evidence, where evidence is marginal probability of observations
$$-\ln p(y)$$
2. Surprise as the difference between a prior and a posterior probability
$$D_{KL}(p(x|y) || p(x))$$



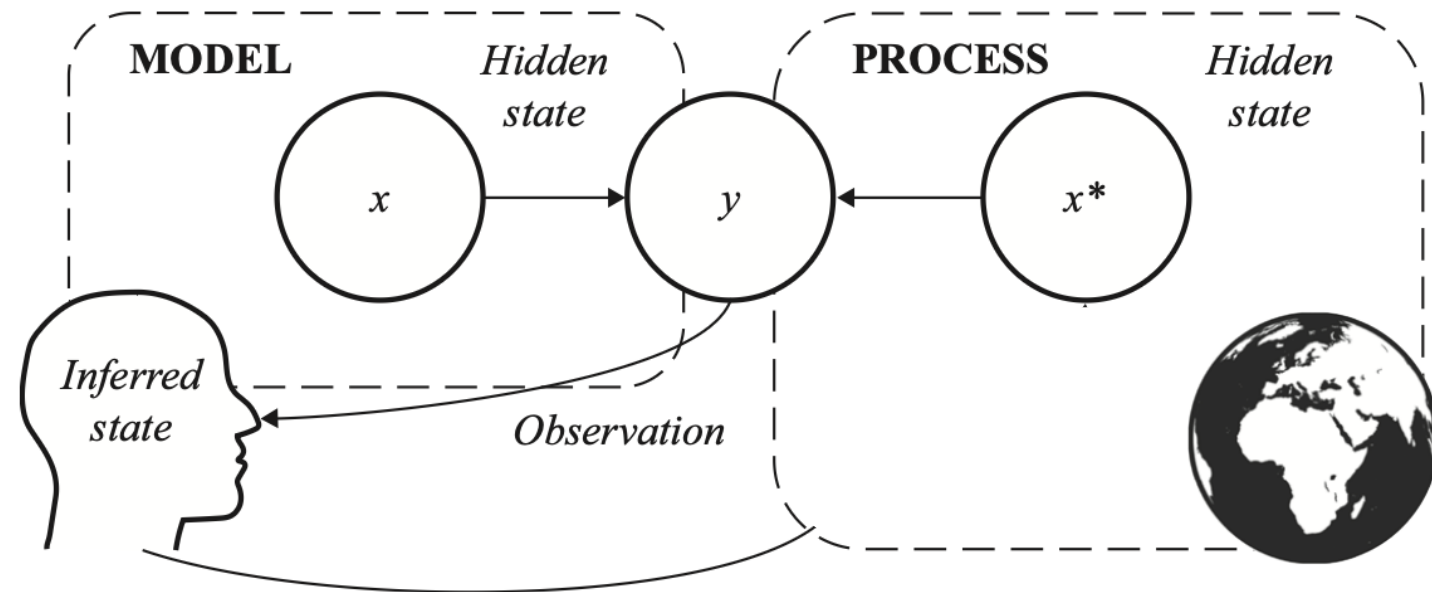
First Model of Biological Inference



Action-Perception Cycle

- Organisms operate on the basis of a subject's generative model.
- In the real world there is a real generative process that generates the observations of the organism.
- The generative model itself may be optimized as new experience is acquired.
- This may or may not converge to the generative process.

1. The generative process is in a true state x^* that is inaccessible to the organism.
2. The organism and world are reciprocally coupled, and x^* generates an observation y , which the organism senses.
3. The organism can use this observation y and Bayes' rule to infer the posterior probability $p(x|y)$ of some explanatory variable or hidden state in the generative model.
4. But there is always an error = $p(x|y) - x$



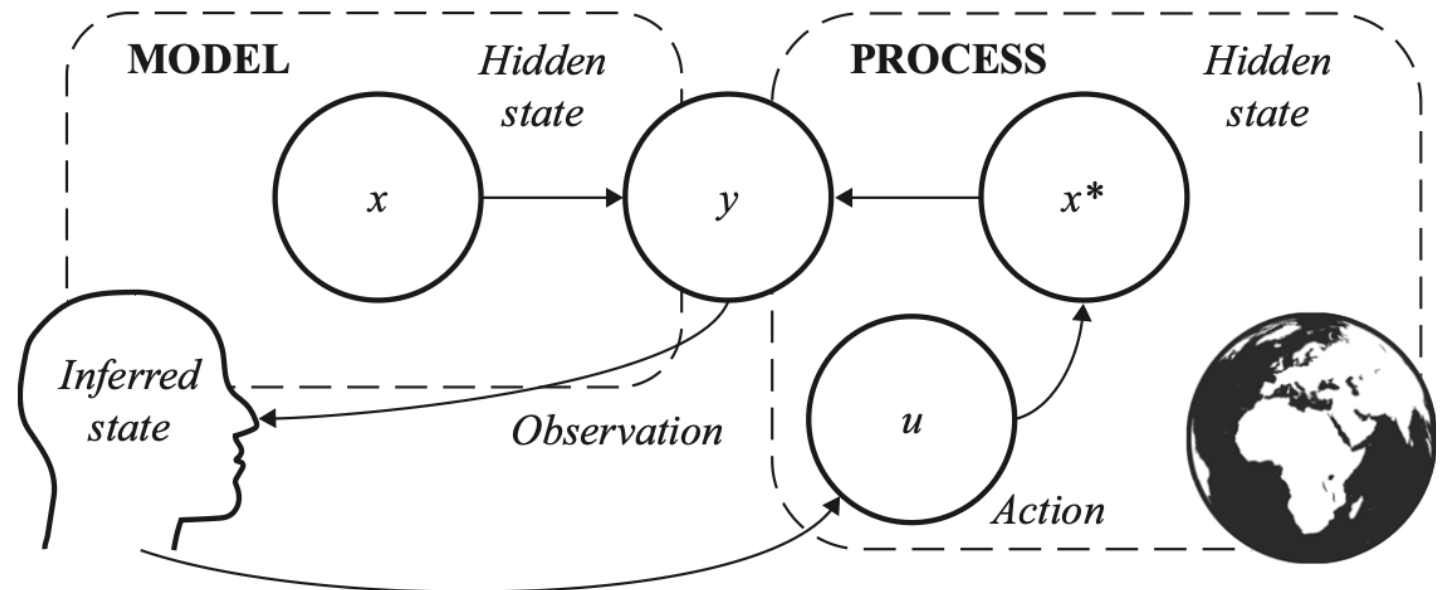
Action as Inference

Bayesian inference maximizes Bayesian model evidence or equivalently minimizes surprise.

There two ways of handling the error:

1. Adapt the generative model
2. Produce some action to change the enviroment

- that perception and action have the same (inferential) nature.
- perception and action cooperate to realize a **single objective** or optimize just **one function**:



Active Inference agents have just one objective

In the Active Inference literature, this common objective has been described in various (informal and formal) ways:

Minimization of

- surprise, → used in statistics
- entropy, → used in physics
- uncertainty, → used in information theory
- prediction error, → used in biological context
- (variational) free energy → used in machine learning context



Under some conditions, one can reduce variational free energy to other notions:

- the discrepancy between the generative model and the world, x versus x^*
- the difference between what one expects and what one observes (i.e., a prediction error). $p(y)$ versus y



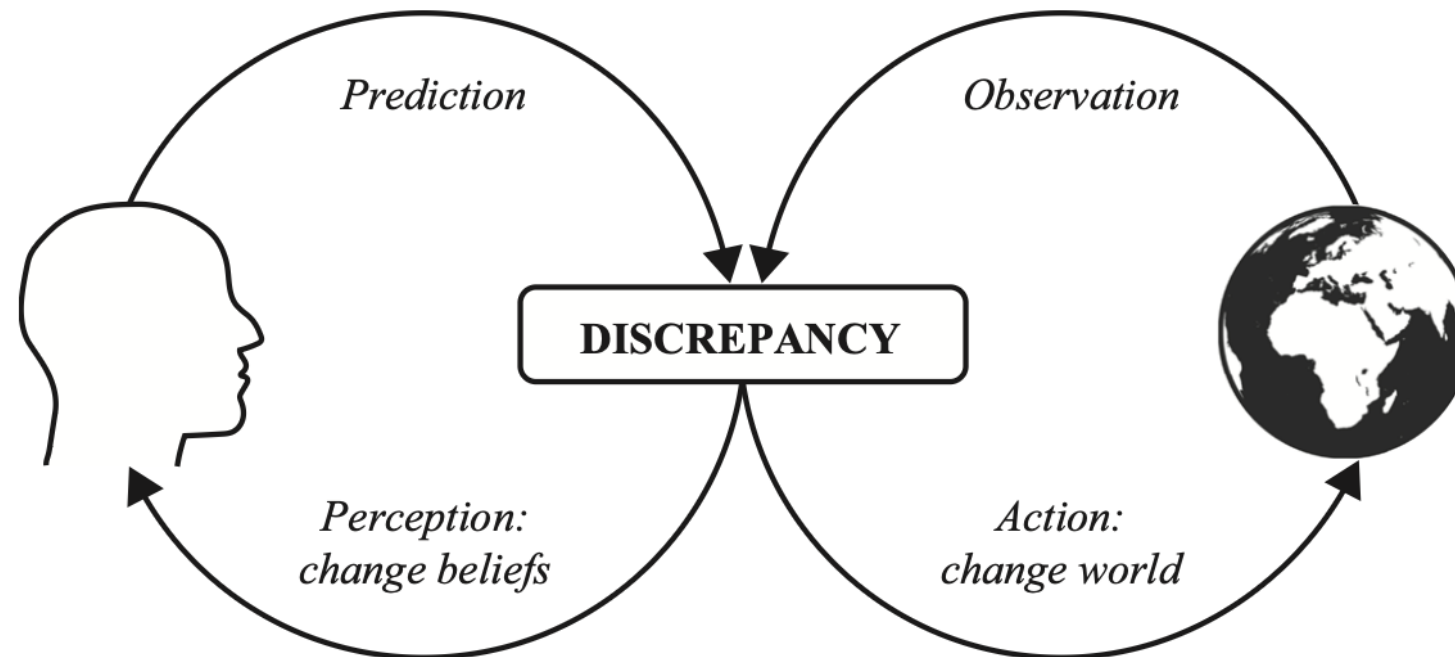
Variational Free Energy Minimization

or Minimizing the Discrepancy between Model and World



Minimize Discrepancy between model and world

- Active Inference agents minimize variational free energy (through perception and action)
- The common objective of perception and action can be formulated as a minimization of the discrepancy between the model and the world.



Minimize Variational Free Energy

Bayesian inference supporting perception and action is computationally intractable in most cases, because two quantities:

- the model evidence $P(y)$
- the posterior probability $P(x | y)$

cannot be computed for two possible reasons:

- complex models, there may be many types of hidden states that all need marginalizing out, making the problem computationally intractable.
- the marginalization operation might require analytically intractable integrals.

Solution: Active Inference appeals to a variational approximation of Bayesian inference that is tractable.

Most importantly, the problem of Bayesian inference now becomes a problem of optimization: the **minimization of variational free energy F .**



Our goal is to approximate the posterior with some tractable distribution $q(x)$:

$$Q(x) \approx P(x|y) = \frac{P(y|x) P(x)}{P(y)}$$

In terms of KL-Divergence:

$$\begin{aligned} D_{\text{KL}}(Q(x) || P(x|y)) &= \mathbb{E}_{Q(x)} [\ln Q(x) - \ln P(x|y)] \\ &= \mathbb{E}_{Q(x)} [\ln Q(x) - \ln P(x, y)] + \ln P(y) \end{aligned} \quad \begin{array}{l} \leftarrow P(x|y) = P(x, y) / P(y) \\ \ln P(x|y) = \ln P(x, y) - \ln P(y) \end{array}$$

Making $Q(x)$ as close as possible to $P(x|y)$ follows we have to minimize $\mathbb{E}_{Q(x)} [\ln Q(x) - \ln P(x, y)]$!

Rearranging: $\mathbb{E}_{Q(x)} [\ln Q(x) - \ln P(x, y)] = D_{\text{KL}}(Q(x) || P(x|y)) - \ln P(y)$



First Interpretation: Energy and Entropy

Variational free energy $F[Q, y]$ is a functional (function of a function) of the approximate posterior Q and a function of data y :

Approximate posterior: $Q \approx P(y|x)$

$$\begin{aligned} F[Q, y] &:= \mathbb{E}_{Q(x)} [\ln Q(x) - \ln P(y, x)] = \mathbb{E}_{Q(x)} [\ln Q(x)] - \mathbb{E}_{Q(x)} [\ln P(y, x)] \\ &= -\mathbb{E}_{Q(x)} [\ln P(y, x)] + \mathbb{E}_{Q(x)} [\ln Q(x)] \end{aligned}$$

Data (observation) y



$$F[Q, y] := - \mathbb{E}_{Q(x)} [\underbrace{\ln P(y, x)}_{< 0}] + \mathbb{E}_{Q(x)} [\ln Q(x)]$$

Shannon Entropy:

$$H(Q) = - \sum q_i \log(q_i) = - \mathbb{E}_{Q(x)} [\ln Q(x)]$$

In the absence of data or precise prior beliefs $p(x)$, we should adopt maximally uncertain beliefs about the hidden states of the world, in accordance with Jaynes's maximum entropy principle.

In statistical physics: Energy is defined as $E(x) = - \ln P(x)$

- States (x, y) that the model assigns high probability → low energy
- States (x, y) the model thinks are unlikely → high energy

Interpretation:

Energy measures how well the latent states explain the data under the model.

- low energy → latent variables that make the observed data likely
- high energy → poor explanations

Second Interpretation: Complexity and Accuracy

Use Bayes Rule again:

$$P(x, y) = P(y|x) * P(x)$$

$$\ln P(x, y) = \ln P(y|x) + \ln P(x)$$

$$\begin{aligned} F[Q, y] &= \mathbb{E}_{Q(x)}[\ln Q(x) - \ln P(x, y)] \\ &= \mathbb{E}_{Q(x)}[\ln Q(x) - \ln P(y|x) - \ln P(x)] \\ &= \mathbb{E}_{Q(x)}[\ln Q(x) - \ln P(x)] - \mathbb{E}_{Q(x)}[\ln P(y|x)] \\ &= \underbrace{D_{KL}(Q(x) || P(x))}_{\text{Complexity}} - \underbrace{\mathbb{E}_{Q(x)}[\ln P(y|x)]}_{\text{Accuracy}} \end{aligned}$$

Complexity is the KL-Divergence between approximated posterior and prior.

- Penalizes overly complex explanations

Accuracy is the expected log-likelihood.

- Measures how well latent states predict the data
- Rewards explanations that fit observations well.

We want to find the simplest (minimally complex) explanation that is able to accurately account for the data (Occam's razor).

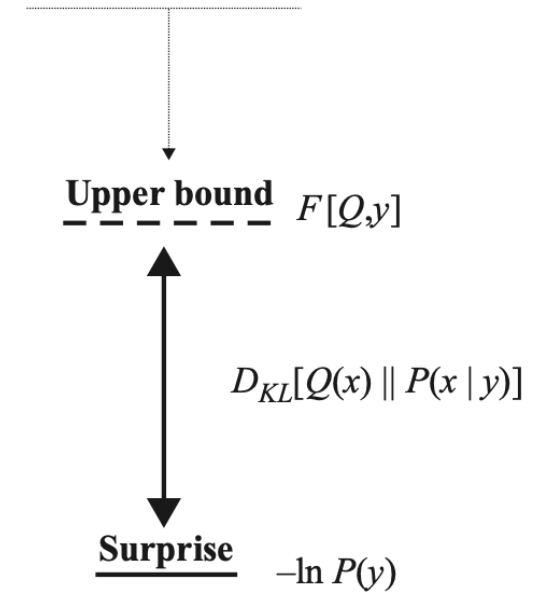
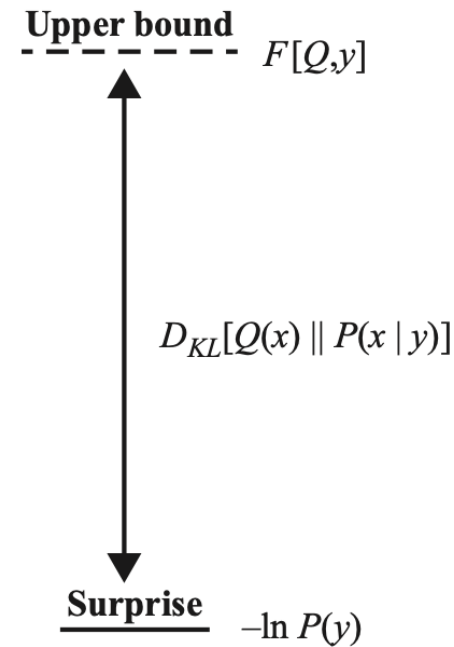
Third Interpretation: Divergence and Evidence

Now expresses the variational free energy as a bound on expected surprise: negative log evidence – $\ln P(y)$

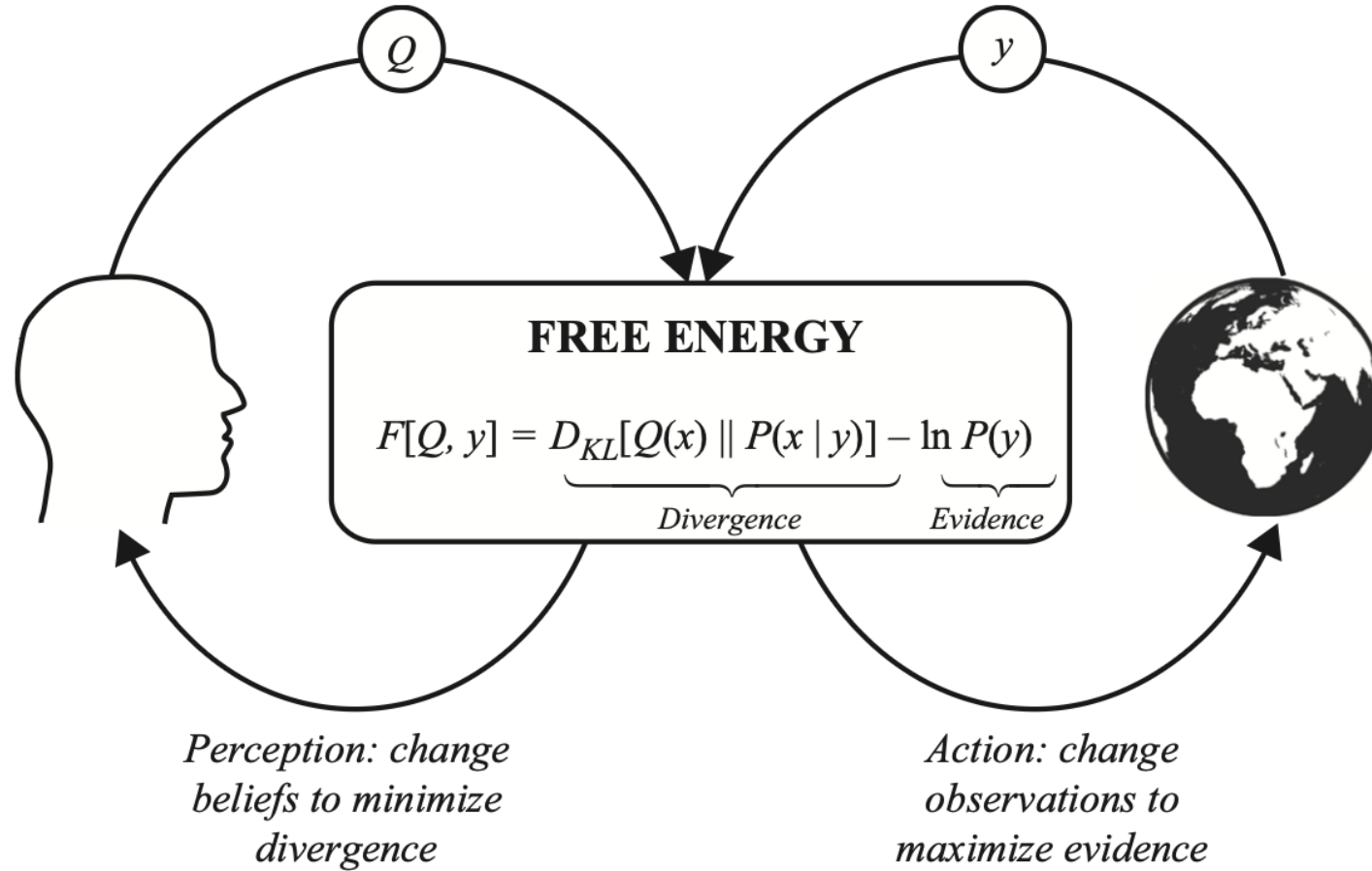
$$\begin{aligned} F[Q, y] &= \mathbb{E}_{Q(x)}[\ln Q(x) - \ln P(x, y)] \\ &= \mathbb{E}_{Q(x)}[\ln Q(x) - \ln P(x|y) - \ln P(y)] \\ &= \mathbb{E}_{Q(x)}[\ln Q(x) - \ln P(x|y)] - \mathbb{E}_{Q(x)}[\ln P(y)] \\ &= \underbrace{D_{KL}(Q(x) || P(x|y))}_{\text{Divergence}} \quad \underbrace{- \mathbb{E}_{Q(x)}[\ln P(y)]}_{\text{Evidence}} \end{aligned}$$

Interpretation:

- either use perceptual inference as one way to lower free energy by optimizing our approximate posterior Q as much as possible
- or change the log evidence term through acting to change sensory data



Minimizing Variational Free Energy



Variational free energy $F[Q, y]$ is a functional (function of a function) of the approximate posterior Q and a function of data y :

$$\begin{aligned} F[Q, y] &= \underbrace{-\mathbb{E}_{Q(x)}[\ln P(y, x)]}_{\text{Energy}} - \underbrace{H[Q(x)]}_{\text{Entropy}} \\ &= \underbrace{D_{KL}[Q(x) \parallel P(x)]}_{\text{Complexity}} - \underbrace{\mathbb{E}_{Q(x)}[\ln P(y | x)]}_{\text{Accuracy}} \\ &= \underbrace{D_{KL}[Q(x) \parallel P(x | y)]}_{\text{Divergence}} - \underbrace{\ln P(y)}_{\text{Evidence}} \end{aligned}$$



Backup

